

OdinSight

An eBPF-Based Client-Side Architecture for Kernel-Level Anti-Cheat on Linux

10th Semester Project





AALBORG UNIVERSITY
STUDENT REPORT

Department of Computer Science
<http://www.aau.dk>

Title:

OdinSight - An eBPF-Based Client-Side Architecture for Kernel-Level Anti-Cheat on Linux

Theme:

Specialization in Computer Science

Project Period:

February 2026 - June 2026

Project Group:

CS-IT10, Group 2

Participant(s):

Eske Klintø Larsen
Faustas Anulis

Supervisor:

Brian Nielsen

Page Numbers: 76

Appendix Page Numbers: 14

Date of Completion:

June 11, 2026

Abstract:

As Linux becomes a more viable platform for gaming, anti-cheat support remains an important challenge. Traditional kernel-level anti-cheat systems provide strong observability but raise concerns about privacy, stability, and user trust. This project presents **OdinSight**, an eBPF-based anti-cheat prototype inspired by the previous TyrSecure project. It addresses the limitations of relying solely on game-runtime monitoring by establishing protection before the game launches and maintaining it throughout the game process lifecycle. By combining eBPF, LSM hooks, cgroups, and kernel integrity validation, it provides monitoring and active policy enforcement against selected cheating techniques. The system is evaluated through compatibility, coverage, and performance tests. We conclude that eBPF is a promising foundation for Linux anti-cheat systems, while further work is needed before it can be considered a complete eBPF-based anti-cheat solution.

Summary

This report presents **OdinSight**, an **eBPF-based client-side anti-cheat prototype** for **Linux**. The project continues the work from **TyrSecure**, a previous project that showed eBPF could be used to detect selected cheating behavior during game execution, but also revealed limitations in a protection model that starts only after the game is already running.

The report first introduces the technical background required to understand the solution. It includes the Linux system model, the separation between **user-space** and **kernel space**, the boot process, Secure Boot, TPM, remote attestation, kernel lockdown, kernel modules, LSM, eBPF, cgroups, and IPC sockets. These concepts provide the basis for understanding the design and implementation of OdinSight.

The problem analysis identifies attacks that may occur before the game starts, during game load, during execution, or against the kernel environment itself. It motivates a protection model in which the anti-cheat establishes **trust before launch** and continues to enforce security policies while the game is running. The analysis suggests that observation alone is not enough, since a system that only reports suspicious behavior may react too late.

OdinSight is designed around a **daemon** and **launcher** architecture to support active enforcement through LSM hooks. The launcher provides an unprivileged interface to request protected game execution. At the same time, the daemon serves as the privileged component responsible for kernel integrity validation, eBPF program management, and controlled game startup.

The implementation shows two main **eBPF LSM** programs. The **daemon hardener** protects the OdinSight daemon from interference, since the anti-cheat itself must remain protected for the system to be reliable. The **game protection** program enforces restrictions on the game processes by blocking selected actions associated with cheating or process tampering.

The compatibility evaluation shows that OdinSight can successfully launch several Linux games, while some titles fail or require exceptions due to runtime behavior that conflicts with strict protection rules. The results show that the prototype is functional, but also that anti-cheat enforcement must balance security with the practical behavior of game engines and platform dependencies.

The coverage evaluation shows that OdinSight enforces its intended policies across the main trust boundaries considered in this project: the kernel environment, the anti-cheat daemon, and the game process. These results validate the implemented protections under controlled test cases, but they should not be interpreted as complete protection against all possible cheats or bypass strategies.

The performance evaluation uses both a synthetic benchmark and a real-game workload to measure whether OdinSight introduces visible performance overhead. The results do not show any clearly

observable degradation caused by OdinSight in the tested scenarios. It indicates that eBPF and LSM-based enforcement can provide useful security functionality with limited visible performance impact, although broader testing would be required to generalize this result.

The report concludes that eBPF combined with LSM hooks is a promising foundation for Linux anti-cheat enforcement. OdinSight provides a client-side protection architecture that validates parts of the kernel environment, starts enforcement before game execution, and restricts selected behaviors in the daemon and game processes. Future work should strengthen integrity verification, remote trust validation, game-specific policy handling, and server-side session validation. Overall, OdinSight demonstrates that eBPF is a viable direction for anti-cheat research on Linux, but further work is needed before such a system can be considered complete.

Contents

1	Preface	1
2	Introduction	2
2.1	Cheating and Fairness in Multiplayer Games	2
2.2	The Growth of Linux Gaming	3
2.3	Anti-Cheat on Linux	4
2.4	Motivation	5
2.5	Contribution	5
2.6	Report Overview	6
3	Background	7
3.1	Linux	7
3.1.1	System Architecture	8
3.1.2	Boot Sequence	9
3.2	Chain of Trust	10
3.2.1	Secure Boot	10
3.2.2	TPM	11
3.2.3	Remote Attestation	12
3.3	Kernel Lockdown	12
3.4	Kernel Modules	13
3.5	Linux Security Module	14
3.6	eBPF	15
3.7	Reactor Pattern	16
3.8	Event-Driven Kernel Monitoring	17
3.9	CGroup	18
3.10	IPC Socket	19
4	Problem Analysis	20
4.1	Previous Project: TyrSecure	20
4.1.1	Limitations of TyrSecure	21
4.2	Coverage of process execution	23
4.3	Active Policy Enforcement	24
4.4	Integrity	24
4.5	Threat Model	25
5	Problem Statement	26
5.1	Requirements	27
6	Design	29
6.1	Overview of OdinSight	29
6.2	Kernel Integrity Validator	32
6.3	ePoll Manager	33
6.4	eBPF Manager	34
6.5	Runner	35

6.5.1	Pre-Execution	36
6.5.2	Execution	36
6.5.3	Termination and Cleanup	37
7	Implementation	38
7.1	eBPF Module Configuration	38
7.2	Daemon Protection Hooks	40
7.3	Game Protection Hooks	42
8	Evaluation	46
8.1	Test Environment	46
8.1.1	Host System Configuration	46
8.1.2	Game Set	47
8.1.3	Tools	48
8.1.4	Custom Libraries	48
8.2	Methodology	50
8.2.1	Compatiblity Test	50
8.2.2	Coverage Test	50
8.2.3	Performance Overhead Test	54
8.3	Test Results	58
8.3.1	Compatibility	58
8.3.2	Coverage	59
8.3.3	Performance Overhead	62
9	Conclusions	68
9.1	Discussion	68
9.1.1	Lifecycle coverage	68
9.1.2	Security and Compatibility Trade-Off	69
9.1.3	Implications for eBPF-Based Anti-cheat	69
9.1.4	Limitations of evaluation	70
9.2	Future works	73
9.2.1	Integrity Verification System	73
9.2.2	Analytical Evaluation	74
9.2.3	Game-specific Policy Profiles	74
9.2.4	Virtualized Execution Environments	75
9.2.5	Extension of Current Evaluation	75
9.3	Conclusion	76
	Bibliography	77
	Appendices	81
A	Compatibility tests	82
B	Coverage tests	90

Chapter 1

Preface

This report was written as part of our 10th-semester project in Computer Science at Aalborg University. The project builds on our previous work in the distributed systems pre-specialization, where we investigated anti-cheat systems and explored the use of eBPF for security-related monitoring on Linux.

During that work, we found that Linux-based anti-cheat systems remain underexplored. It motivated the development of TyrSecure, a proof-of-concept anti-cheat solution based on eBPF. The project provided valuable experience in Linux security, kernel-level monitoring, and the challenges of protecting multiplayer games from cheating.

This project continues that work by addressing the limitations identified in TyrSecure through the design and implementation of OdinSight. Compared to the previous concept, OdinSight places greater emphasis on protecting the game process throughout its lifecycle, a focus that was not fully addressed in the pre-specialization phase.

The data regarding the performance overhead of our implementation is provided as a separate attachment to this report.

We would like to express our gratitude to our supervisor, Brian Nielsen, for his guidance, technical insights, and consistent feedback throughout the project.

AI-based chat tools were used in selected parts of the report to support grammar correction, language refinement, and readability improvements.

Chapter 2

Introduction

This chapter introduces the context and motivation for the project. It first explains why cheating is a serious problem in multiplayer games and how it affects fairness, trust, and the integrity of online competition. It then discusses the growing relevance of Linux as a gaming platform and the challenges that multiplayer games face when anti-cheat systems do not fully support or trust the Linux environment.

The chapter also presents the role of anti-cheat systems on Linux and explains why kernel-level observability is important for detecting and preventing local cheating techniques. This motivates exploring **eBPF** as a possible foundation for anti-cheat protection, since it can provide kernel-level monitoring and enforcement without relying on traditional kernel modules. Finally, the chapter introduces the project's main contribution, **OdinSight**, and provides an overview of the remaining chapters in the report.

2.1 Cheating and Fairness in Multiplayer Games

Multiplayer games rely on the assumption that players compete under the same rules and within the conditions intended by the game operator. **Cheating** in online games is defined as behavior that gives a player an advantage, or allows them to achieve a target that they are not supposed to have achieved according to the game rules or at the discretion of the game operator as described in the paper [50]. The outcome of a match is therefore expected to be determined by factors such as skill, strategy, reaction time, communication, and cooperation. When this assumption is violated, the game's outcome becomes less meaningful because players can no longer trust that opponents are subject to the same constraints.

In this report, cheating is defined as any unauthorized action that gives a player an unfair advantage or bypasses the game's intended rules. This can include modifying the game, observing information that should remain hidden, automating player actions, manipulating the execution environment, or interfering with systems that are intended to preserve fair play.

Cheats can appear in different forms and at different levels of a multiplayer game system. Some directly affect gameplay by improving player performance, for example, through automated aiming tools that improve a player's reaction or accuracy. Others provide unfair access to information that should remain hidden, modify game clients or server configurations, exploit bugs and design loopholes, or take advantage of weaknesses in the surrounding hardware, software, or network environment. Cheating may also involve player collusion, abuse of game procedures, manipulation of virtual assets, or exploitation of security weaknesses such as inadequate authentication and secrecy

mechanisms. Although these techniques differ in their implementation, they share the common objective of giving the cheating player an unfair advantage and undermining the game's fairness [9].

The impact of cheating is not limited to an individual match. It can reduce player trust, undermine fairness in competitive play, and make legitimate players less willing to participate, since online cheating can ruin games, cause users to give up, drive away new users, and affect player retention. This can also create an economic impact for game developers, since cheating may harm a game's reputation and threaten company revenue [9]. For game developers, cheating is therefore both a gameplay problem, a security problem, and a business concern, because the intended rules of the game depend on the integrity of the software and the environment in which it executes.

As multiplayer games become available on more platforms, they must also support mechanisms that protect game integrity. This is increasingly relevant for Linux, which has become a more practical gaming platform in recent years.

2.2 The Growth of Linux Gaming

Linux has historically been a smaller gaming platform than Windows, partly because many commercial games and supporting technologies were developed primarily for Windows. In recent years, this has changed significantly. Improvements in GPU drivers, Windows-to-Linux compatibility layers, and the Steam ecosystem have enabled the running of a growing number of games on Linux [45].

The Steam Deck has further increased the visibility of Linux gaming, while its Arch-based SteamOS environment has helped promote Linux as a viable gaming platform [5]. As a result, Linux is no longer only relevant to a small group of desktop users but also to a wider gaming audience.

However, running the game is only part of the problem. Multiplayer games also depend on supporting systems such as matchmaking, networking, platform services, and anti-cheat. As noted in the article, anti-cheat remains one of the main limitations for Linux gaming, since some games rely on systems that either do not support Linux or actively block it [45]. A game may therefore be technically compatible with Linux, but still unavailable or limited if its anti-cheat system does not support or trust the Linux execution environment.

Improved anti-cheat support could therefore contribute to the future growth of Linux gaming. If multiplayer games can be protected on Linux in a way that is practical for developers and acceptable for users, more games may be able to support Linux without excluding competitive or online features. This motivates further research into anti-cheat approaches to protect Linux games.

2.3 Anti-Cheat on Linux

Anti-cheat systems protect the integrity of multiplayer games by detecting, mitigating, or preventing behavior that gives players an unfair advantage. These systems can be implemented on the server side, on the client side, or through a combination of both. **Server-side anti-cheat** can validate game state and client input, identify behavior that violates the rules of the game, and analyze player activity over time. However, many cheat techniques are performed locally on the player's machine before their effects are visible to the server. Examples include memory scanning, debugger attachment, process inspection, code injection, including shared-library injection, and runtime code manipulation [1].

For this reason, **client-side anti-cheat** remains widely used for multiplayer game security. A client-side system can monitor the local execution environment and respond to suspicious behavior as it occurs. This is especially relevant when cheats target the game process itself or the operating-system interfaces around it. However, client-side anti-cheat also has an important limitation: it operates in a player-controlled environment [1]. This limitation is particularly important on Linux, where users often have greater visibility into and control over the operating system. On Linux, some players may have root or kernel-level privileges, or enough technical expertise to modify the local execution environment, inspect processes, load kernel modules, change kernel behavior, or deploy advanced cheats. If the operating system or kernel is compromised, then the information reported to the anti-cheat cannot be trusted [13].

Many competitive games address this problem by using kernel-level anti-cheat systems. These systems run with high privileges and can observe activity that may be hidden from ordinary user-space anti-cheat software. This allows them to detect or prevent attacks involving debugging, memory manipulation, process injection, and other low-level techniques. However, this approach introduces significant trade-offs. Traditional kernel-level anti-cheat systems often rely on kernel-level drivers or kernel modules that execute within the operating system with extensive privileges. These components can introduce privacy and security risks, leading to system instability, conflicts with drivers and security tools, and exposure of sensitive data if compromised [1].

This makes **Linux anti-cheat** design challenging because the system needs sufficient observability and enforcement capabilities to protect the game while avoiding unnecessary stability risks to the operating system [1]. eBPF provides a possible foundation for addressing this challenge because its programs must pass the **kernel verifier** before execution and run in a restricted execution environment. This limits unsafe memory access and other behavior that could affect the kernel [11], making eBPF less intrusive than fully privileged kernel modules while still allowing kernel-level monitoring and enforcement. Furthermore, eBPF simplifies maintenance by using Compile Once – Run Everywhere (CO-RE) [10], which allows the anti-cheat to adapt to different kernel configurations without breaking and supports frequent Linux kernel updates.

For anti-cheat designs on Linux, eBPF is relevant because it provides kernel-level observability without relying on traditional kernel modules. It enables monitoring and enforcement at important points in the game process lifecycle, including process execution, memory permission changes, debugging attempts, filesystem access, capability use, and other interactions with the operating system. As a result, eBPF offers a way to explore anti-cheat protection on Linux that balances observability, enforcement, system stability, and user trust.

2.4 Motivation

This project is motivated by our previous 9th-semester work on **TyrSecure** [19], which investigated whether the eBPF framework could serve as a foundation for an eBPF-based anti-cheat system on Linux. TyrSecure demonstrated that eBPF programs have potential in this area and can monitor the system and respond to specific cheating behavior without relying on kernel modules.

However, TyrSecure’s anti-cheat design was incomplete, preventing eBPF from providing its full protection. The anti-cheat system focused on monitoring a game after it had already started, which left parts of the game lifecycle unprotected. This allowed for the injection of cheats such as wallhacks and aimbots before the game starts. Additionally, the anti-cheat provided limited proactive blocking in response to suspicious behavior.

These limitations do not show that eBPF is incompatible for anti-cheat development. Instead, they show that area is largely unexplored. TyrSecure is a starting point in this area and highlights the need for further research to evaluate eBPF protection as part of a more complete anti-cheat design.

The motivation for this project is to continue that research. We are interested in building a more complete eBPF-based anti-cheat system that can establish trust before the game starts and maintain it throughout the game’s lifecycle, while providing stronger protection against cheats by proactively blocking suspicious behavior.

2.5 Contribution

The main contribution of this project is OdinSight, an eBPF-based anti-cheat prototype for Linux. OdinSight extends the previous TyrSecure approach by protecting a larger part of the game lifecycle.

The project contributes the following:

- **Daemon and Launcher Architecture:** A novel architecture consisting of a separate launcher process that requests protected game execution and a daemon process that orchestrates protections for both itself and game processes.
- **Lifecycle Protection:** A daemon-managed protection design that initializes anti-cheat protection before the protected game process starts and maintains it throughout the process lifespan.
- **Kernel Integrity Validation:** Validation of the Linux kernel environment to increase trust in the system on which OdinSight operates.
- **Empirical Evaluation:** An evaluation of OdinSight’s compatibility, security coverage, and performance overhead.

Together, these contributions evaluate whether eBPF can provide practical anti-cheat protection on Linux while reducing some of the risks associated with traditional kernel-module-based anti-cheat systems.

2.6 Report Overview

Chapter 3 introduces the technical background required for the project, including Linux system architecture, the Linux boot sequence, chain of trust, Secure Boot, TPM, remote attestation, kernel lockdown, kernel modules, Linux Security Modules, eBPF, event-driven kernel monitoring, cgroups, and IPC sockets.

Chapter 4 analyzes the problem area. It discusses the previous TyrSecure project, identifies limitations in its protection model, and motivates the need for stronger process lifecycle coverage, active policy enforcement, integrity validation, and a clear threat model.

Chapter 5 defines the problem statement and derives the requirements used to guide the design of OdinSight.

Chapter 6 presents the design of OdinSight, including the kernel integrity validator, eBPF manager, event handling architecture, and game runner.

Chapter 7 describes OdinSight's implementation and explains how the selected eBPF and LSM hooks are configured to protect the daemon and the game process.

Chapter 8 evaluates OdinSight using compatibility and coverage tests, as well as performance overhead measurements.

Chapter 9 discusses the results, identifies limitations, presents future work, and concludes the project.

Chapter 3

Background

This chapter introduces the technical background required for the project. It presents the necessary Linux concepts, security mechanisms, and design patterns for implementing the proposed solution.

3.1 Linux

Linux is an open-source, Unix-like kernel that forms the core of Linux-based operating systems. It manages hardware resources and exposes kernel services to user-space applications through defined kernel interfaces, primarily system calls [27].

For this project, Linux serves as the foundation for the proposed eBPF-based anti-cheat solution. The separation between user space and kernel space governs how applications interact with privileged system functionality, while the boot process establishes trust in the running system before user-space services start. These concepts provide the basis for understanding the use of Secure Boot, Kernel Lockdown, kernel module verification, and other mechanisms discussed later in this chapter.

3.1.1 System Architecture

The Linux system structure is shown in Figure 3.1. As described in the book [26], the user-space layer is where user programs and services execute, while the kernel provides the controlled interface to hardware resources and system services through mechanisms such as system calls.

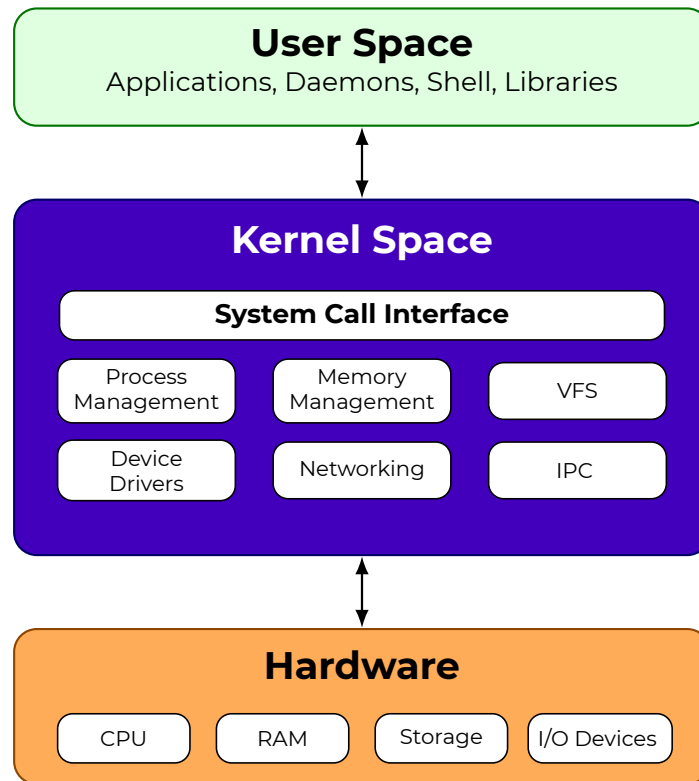


Figure 3.1: Kernel Layer and its Interaction with User Space and Hardware

The following sections describe each layer in more detail, starting with the hardware layer, then moving to kernel space, and finally to user space.

Hardware

As described in Operating System Concepts [38], the hardware layer provides the physical resources required for execution, including the CPU, memory, storage, and I/O devices. The CPU executes instructions, memory stores data currently in use, storage provides persistent data storage, and I/O devices allow the system to communicate with users and external devices.

Kernel-Space

As described in Linux Kernel Development [26], the kernel is the core component of the Linux system and provides the privileged layer between user-space programs and hardware resources.

The kernel is commonly divided into several subsystems. Process management handles process creation, scheduling, and termination, while memory management controls how system memory is allocated and used [26]. The Virtual File System (VFS) provides a filesystem interface to user-space programs and abstracts the details of specific filesystem implementations [43]. Other subsystems

provide inter-process communication, networking, and device-driver support, allowing processes to communicate, exchange data over networks, and interact with hardware devices.

User-Space

User-space applications and services run outside the kernel with limited privileges. As mentioned in the Linux manual page [34], system calls request operating-system services from the kernel, often via system libraries. The Linux manual pages describe system calls as the fundamental interface between an application and the Linux kernel.

User-space programs have different responsibilities within a Linux system. Command-line shells, such as bash, provide a text-based interface for interacting with the operating system [21]. Desktop applications, such as games, web browsers, and file managers, allow users to perform tasks through a graphical interface and are started through a desktop environment. Daemons are background programs that run without direct user interaction and provide services to the system or other applications [41].

3.1.2 Boot Sequence

The Linux boot sequence is the startup process of a Linux system. It begins with firmware initialization and continues through the loading of the bootloader, kernel, initramfs, root filesystem, and the main userspace init system [40]. Understanding the boot process is important because trust in the running system begins with the integrity of the early boot components. Therefore, this subsection provides the background for understanding how a **Chain of Trust** can be established, discussed in Section 3.2.

The Linux boot sequence consists of the following components:

- **Unified Extensible Firmware Interface(UEFI)** is the software stored on the motherboard. After the computer starts, it initializes the hardware and starts the boot process by loading and executing the bootloader from the EFI System Partition [47].
- **Bootloader** is a small program that loads the Linux kernel and initial RAM filesystem into memory and then starts the kernel [40].
- **Linux kernel** is loaded by the bootloader and performs the early boot steps needed before the main userspace system starts. During this stage, it initializes essential kernel functionality and starts the initramfs environment used for early boot preparation [44].
- **Initramfs** is an early userspace root filesystem that is loaded into memory during boot. It may contain an /init program and other files needed to bring the system up, including locating and mounting the real root filesystem. After this, it can switch to the real root filesystem and execute the main init program [44].
- **Root filesystem** is the real root filesystem that is located and mounted after the initramfs stage. Once it is mounted, control can be handed over from the initramfs environment to the main userspace system [44].
- **systemd** is commonly used as the main init system after the system switches to the real root filesystem. When run as PID 1, it acts as a system and service manager that brings up and maintains userspace services required for system operation [7].

Figure 3.2 illustrates an overview of this sequence.

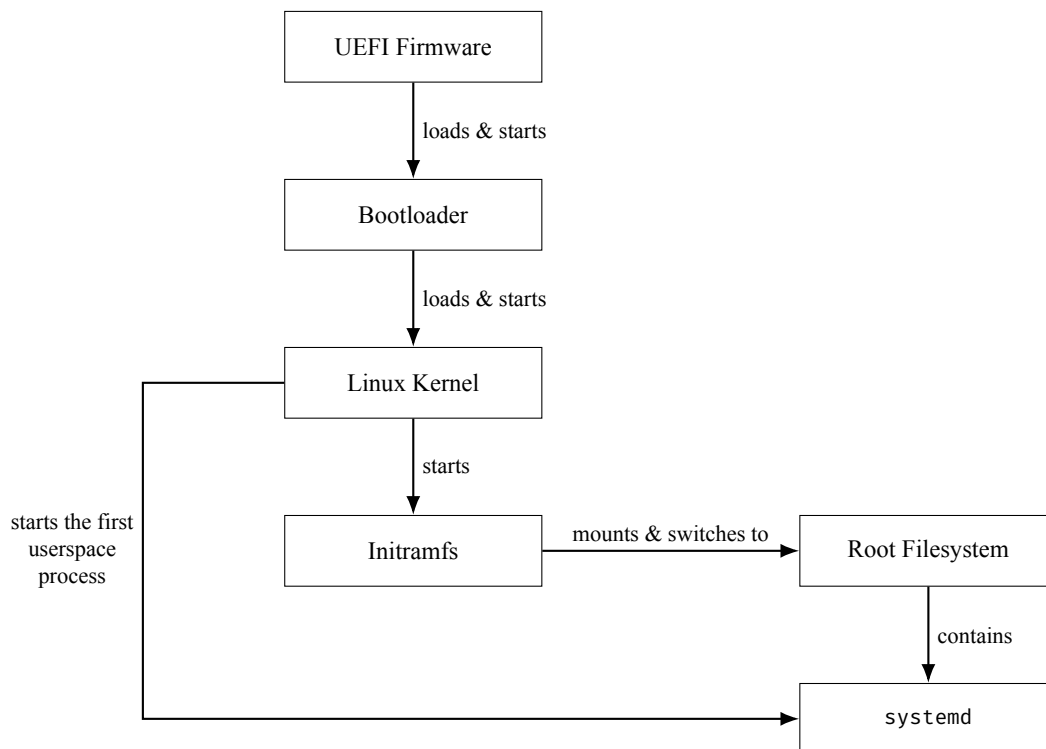


Figure 3.2: Boot sequence from UEFI firmware to userspace initialization.

3.2 Chain of Trust

The Chain of Trust establishes trust throughout the boot process, in which each component verifies the integrity of the next component before transferring execution [49]. This is useful for anti-cheat systems, as their reliability depends on whether the kernel itself can be trusted; if the early boot process has been compromised, anti-cheat may execute in an already untrusted environment.

In Linux systems, a Chain of Trust can be supported by security mechanisms such as Secure Boot, TPM and remote attestation. **Secure boot** enforces signature verification during the boot process, **TPM** stores cryptographic measurements of boot components, and **remote attestation** can use these measurements to compare the system's boot state against a trusted configuration. The following subsections explain these approaches and show how they help establish trust during the Linux boot process [32].

3.2.1 Secure Boot

To increase trust in a host system, its integrity should be protected from the earliest stages of boot. According to the paper [14], UEFI Secure Boot protects the boot process by verifying boot components before execution. Secure Boot uses cryptographic signature verification to ensure that only trusted boot components are allowed to run, helping to build trust that begins in firmware and extends into the early operating system boot process.

Figure 3.2 shows the normal Linux boot sequence from UEFI firmware to userspace initialization. Secure Boot does not change this execution order. Instead, it adds cryptographic verification before selected boot components are allowed to run. In a Linux Secure Boot configuration, the bootloader

stage shown in the figure may include shim as a first-stage bootloader. According to the paper [14], shim is signed by the Microsoft UEFI Certificate Authority and is therefore trusted by UEFI firmware when Secure Boot is enabled. Shim then validates and chain-loads GRUB, which can extend the chain of trust by verifying the Linux kernel before execution. If a boot component has been modified or is not signed by a trusted key, verification fails and the boot process can be stopped. This helps prevent unauthorized code from running before the operating system starts.

The keys used for Secure Boot verification are stored as UEFI firmware variables and determine which boot components are trusted. The Platform Key (PK) represents the root of trust for the platform and can be used to update the Key Exchange Key (KEK). The KEK protects updates to the Secure Boot databases. The signature database (DB) contains trusted certificates or hashes, while the forbidden signature database (DBX) contains revoked or blocked certificates and hashes. A boot component is allowed to execute only if it is trusted through the DB and is not blocked by the DBX. Many consumer devices include Microsoft-approved keys by default, allowing widely used operating system bootloaders to run without additional configuration [14].

For Linux systems, shim also supports the Machine Owner Key (MOK) mechanism. MOK can be created by the system owner and used to sign components such as GRUB and the Linux kernel. A certificate corresponding to this key is stored in the shim-controlled MOK list, allowing the owner to control which bootloaders or kernels are trusted without directly replacing the firmware Secure Boot keys [14].

Although Secure Boot establishes trust during the boot process, it does not prevent a privileged userspace process from modifying the running kernel. As a result, the trust established during boot may be compromised once the system is running.

3.2.2 TPM

A Trusted Platform Module is a dedicated security chip designed to securely store cryptographic keys. It supports operations such as hashing, signing and recording measurements of the system state during boot [29].

The TPM improves security by providing hardware-backed protection for cryptographic data. Because the TPM isolates key operations in dedicated hardware and is designed to resist software tampering, keys protected by the TPM are harder to extract through software attacks. This makes TPM useful for integrity verification, device authentication and protecting keys used by disk encryption [29].

Platform Configuration Registers (PCRs) are registers in a TPM used to store hash measurements of different system components, such as firmware, bootloader and kernel during the boot process. The resulting PCR values depend on both the new measurement and the previous PCR content to represent the state of the TPM's host platform at a given time [35].

In comparison to Secure Boot, a TPM does not prevent malicious software from executing. Rather, it stores cryptographic measurements, such as hashes of the boot components, in Platform Configuration Registers. This enables the remote verifier to later compare these measurements with known, trusted values to determine whether the system booted into an unmodified state.

TPM can also be used to protect sensitive data, so access is granted only if the system boots as expected. If the boot chain is modified, the measurements stored in TPM will change, and access to protected data can be denied [14].

3.2.3 Remote Attestation

Remote attestation is a security process in which a system proves its integrity to a verifier as described in the paper [2].

According to the paper [36], hardware-based attestation can be supported by a Trusted Platform Module (TPM). During the boot process, important software and firmware components are measured before execution. These measurements are extended into the TPM Platform Configuration Registers (PCRs), which provide a protected record of the system's boot state. During attestation, the TPM can sign selected PCR values using an attestation key, producing a TPM quote. This quote allows the verifier to check that the reported PCR values were produced by a genuine TPM and have not been modified in transit.

The verifier then compares the received measurements against expected values, trusted reference measurements, or an attestation policy. If the measurements match the expected configuration, the system can be considered to be in a known and trusted state. If the measurements differ, this may indicate that firmware, the bootloader, the kernel, or other measured runtime components have been changed, misconfigured, or compromised. In that case, the verifier may reject the attestation result and treat the system as untrusted [36].

Remote attestation is useful because it reduces the need to fully trust the local machine. Instead of relying only on claims made by the system itself, a remote verifier can evaluate cryptographic evidence about the system's measured state. This makes remote attestation particularly important in environments where machines may be shared between multiple users or exposed to potential compromise.

For client-side security systems, it provides an additional way to reason about whether the local environment can be trusted. If the operating system or kernel has been modified, then information reported by the client may no longer be reliable. By allowing a remote verifier to evaluate cryptographic evidence about the system's measured state, remote attestation can help establish whether the client environment is suitable before relying on local monitoring or enforcement mechanisms.

3.3 Kernel Lockdown

Kernel Lockdown is a Linux security feature designed to preserve the integrity of the running kernel. As discussed in Section 3.2.1, Secure Boot establishes trust during the boot process. Kernel Lockdown helps maintain that trust during runtime by restricting kernel interfaces and privileged operations that could be used to tamper with the running kernel [30].

These restrictions include preventing the loading of unsigned kernel modules [30]. According to the Arch Linux manual page [17], kernel lockdown also restricts access to interfaces that expose the running kernel image or kernel memory, such as `/dev/mem` and `/dev/kcore`, as well as mechanisms such as BPF and kprobes that can inspect or instrument kernel execution. These restrictions reduce privileged processes' ability to tamper with the running kernel.

Kernel Lockdown can be enabled in three modes: none, integrity, and confidentiality. In **integrity** mode, it prevents user-space from modifying the running kernel and blocks operations such as loading unsigned kernel modules [30], writing to kernel memory, and accessing low-level interfaces that could be used to modify the kernel at runtime [17]. **Confidentiality** mode is a superset of integrity mode; it includes all integrity restrictions and further restricts interfaces that could be used to read sensitive information from the kernel [30].

3.4 Kernel Modules

Kernel modules are `.ko` (kernel objects) that can be loaded into the Linux kernel to extend its functionality. They run in the kernel space, have the same privileges as the kernel itself, and can directly interact with kernel memory and hardware resources [25].

Kernel modules introduce a security risk. Malicious modules may be used to modify kernel behavior, hide processes, or disable security mechanisms. Because kernel modules can be loaded at runtime, they also introduce a risk of malicious modifications of the running kernel. Linux provides multiple ways to restrict kernel modules [39]:

1. Requiring kernel modules to be digitally signed using Module Signature Enforcement [3].
2. Preventing unsigned kernel modules from being loaded using Kernel Lockdown [30].
3. Disabling all module loading after system initialization [4].

Disabling all module loading after the system has finished the boot process prevents any further modules from being loaded during runtime. While it is an effective way to prevent attackers from loading malicious modules even with elevated privileges, it can be impractical on general systems because legitimate modules, such as hardware drivers and virtualization tools, may need to be loaded later.

Requiring kernel modules to be signed instead is a more practical approach. With **Module Signature Enforcement** enabled, the kernel verifies the module's digital signature before loading it. Kernel modules are signed with a private signing key by the kernel vendor, a Linux distribution, a third-party vendor, or a system administrator responsible for approving and signing locally managed modules.

When a module is loaded, the kernel checks whether its signature matches a trusted public key and whether the module has been modified. If the module is unsigned, has been altered after signing, or is signed by an untrusted key, the kernel can reject it. However, this protection depends on strict control of the signing keys and the trusted-key enrollment process. In systems where administrators can enroll their own trusted keys, they may be able to sign and load a malicious module. Therefore, module signing only protects against unauthorized modules when trusted keys are carefully managed, and key enrollment is restricted to approved signers.

3.5 Linux Security Module

According to the Linux kernel documentation [42], the **Linux Security Module** is a security framework within the Linux kernel that supports the implementation of various security models.

The default Linux security model controls access through file ownership and permission bits; however, it may not be sufficient for systems that require stronger security. LSMs improve this by enabling security policies in addition to the standard permission system.

It provides security hook placement at different points in the kernel, such as process creation, program execution, file access, and others. When such an operation occurs, the security model can decide whether to permit it.

For example, the LSM hook `task_kill` is invoked when one process attempts to send a termination signal, such as `SIGTERM` or `SIGKILL`, to another process. Before that signal is delivered, the LSM framework invokes registered security modules, which can decide whether to deliver it.

Multiple well-known Linux security systems, such as SELinux, AppArmor, and Landlock, are built on top of the LSM framework to enforce access control policies. As described in the Linux kernel administration guide [42], this framework serves as the foundation for these security subsystems.

SELinux provides mandatory access control policies using security labels that attach to network sockets, processes, files, and other system resources. A process is allowed to perform operations on resources only if those operations are permitted by the SELinux policy. It follows the principle of least privilege, denying operations by default to reduce the attack surface of the Linux environment.

AppArmor utilizes security profiles that can be assigned to individual programs. Once a program starts, its processes are restricted by the rules in the assigned profile. A security profile defines what resources and operations the program can use. If the program is trying to perform an action outside its security profile, AppArmor will deny the operation.

Landlock enables unprivileged processes to restrict their own access rights. It can be used to run a program with restricted permissions by limiting access to resources such as files, directories, and network operations. In comparison to SELinux or AppArmor, which are configured through system-wide policies, Landlock provides application driven way to enable restricted permissions, this means that using it requires a launcher or modifying application code to apply a ruleset before restrictions are enforced.

3.6 eBPF

eBPF (Extended Berkeley Packet Filter) is a Linux kernel technology that allows developers to build small, sandboxed programs that run in kernel space.

Its workflow and main components are illustrated in Figure 3.3.

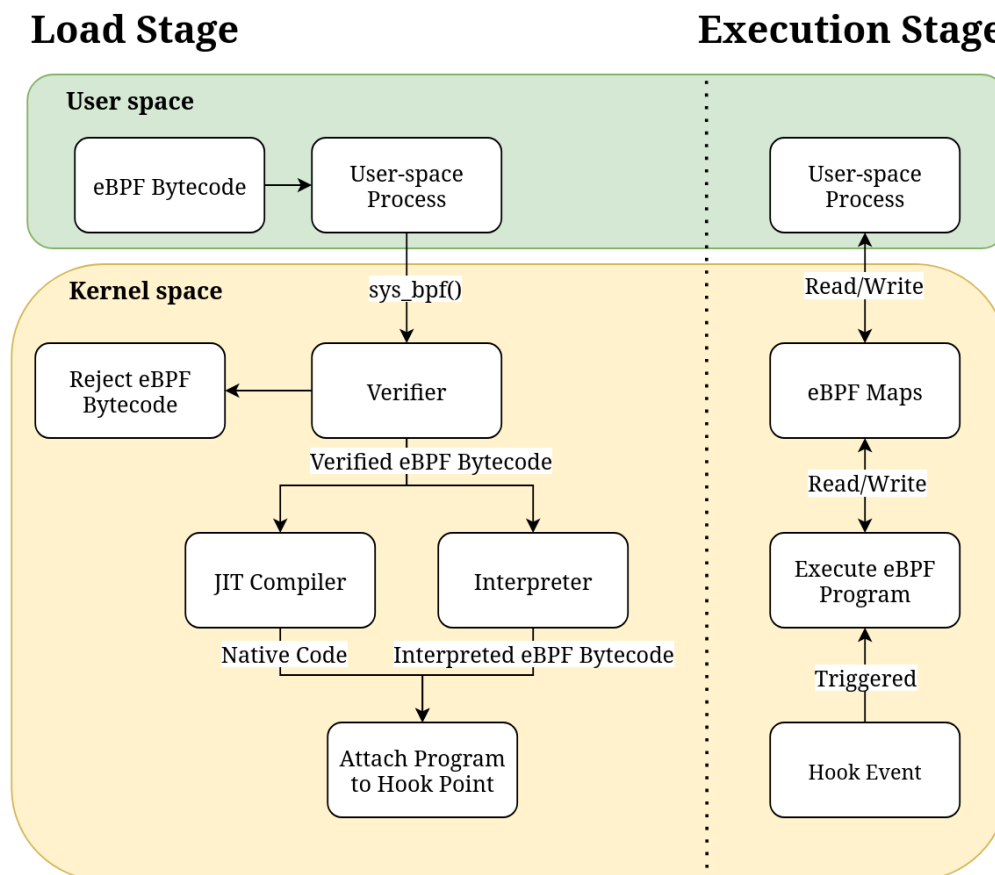


Figure 3.3: eBPF workflow

First, during the load stage, an eBPF program is compiled in user space into unverified bytecode. A user-space process loads the program into the kernel through the `bpf` system call [12]. At this point, the program is not yet trusted and cannot be executed.

Once it enters the kernel, the eBPF **verifier** performs a static analysis of the bytecode before it can be executed to ensure that the program is safe and terminates, so it doesn't perform invalid memory accesses, unsafe pointer operations, or other actions that could compromise kernel stability [48]. Kernel only accepts programs that pass such verification. After verification, the kernel stores an eBPF program as verified eBPF bytecode. If Just-in-Time (JIT) compiler is enabled, the kernel also generates and stores JIT-compiled native code for execution [31].

The **Interpreter** may be used; it provides an alternative execution path for eBPF programs when **JIT compilation** is disabled. As a result, verified eBPF programs can be executed either by the interpreter or as JIT-compiled native code. Safety comes from the kernel's checking of eBPF programs, while JIT execution is generally more efficient than interpretation [31].

After loading and verification, the eBPF program is attached to kernel hook points. These hook points represent locations where kernel or system events can be monitored or controlled. Depending on the

program type, they may be network events, system calls, or tracepoints. When the event occurs, the kernel executes the eBPF program [33].

Another important component of the eBPF architecture is **BPF maps**. These are data structures stored in the kernel and shared between kernel space and user space. They allow multiple eBPF programs and user-space processes to share and exchange data. For example, BPF maps can store counters, configuration data, policy rules and event information. User-space applications can read from and update BPF maps [6]. These maps may also contain references to eBPF programs, in which one eBPF program can transfer execution to another without returning via the normal call path. This allows larger eBPF systems to be constructed from multiple smaller programs [18].

eBPF programs are especially powerful when integrated with the LSM framework; they enable programmable access control policies, allowing enforcement logic to be implemented without modifying the kernel or developing traditional loadable kernel modules.

3.7 Reactor Pattern

The Reactor design pattern emerged in 1995. As described in paper [37], the design was a response to the scalability limitations of the "Thread-per-Connection" server model. This model failed to scale because the system incurred more overhead from context switching among thousands of threads than from processing data. Schmidt's paper identified that high-performance systems required a mechanism to handle multiple simultaneous service requests without the overhead of multi-threading. The Reactor pattern solves this by demultiplexing events, allowing a single thread to monitor multiple event sources and dispatch them to the appropriate handlers. These handlers are specialized components that encapsulate the application-specific logic required to process each request as it becomes available.

As illustrated in the sequence diagram (Figure 3.4), the simplified architecture follows a variant of the Reactor pattern, inspired by the implementation in [16]. This architecture is organized into five pillars: Client, Reactor, Dispatcher, EventHandler, and RequestHandler. Each component fulfills a specific role within the event-driven lifecycle:

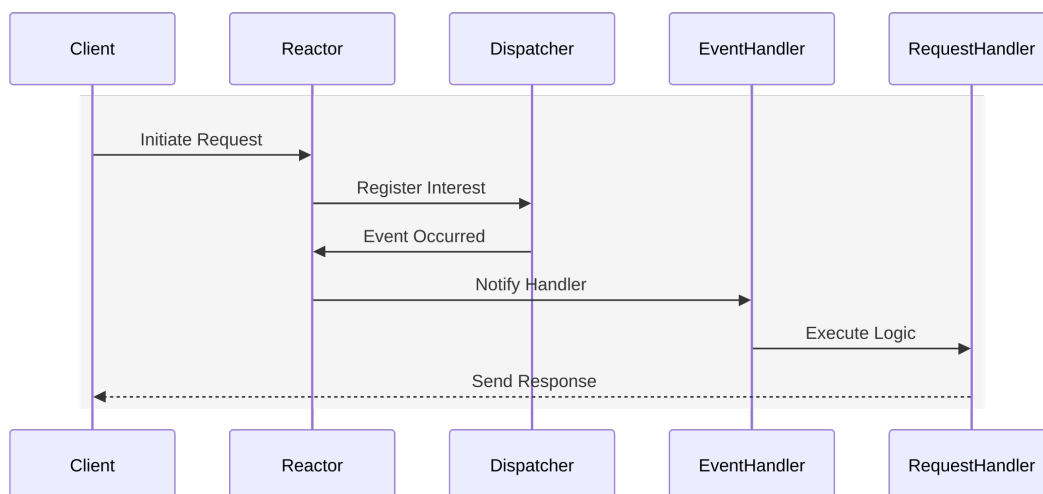


Figure 3.4: Reactor pattern

- **Initialization:** The process begins when the Client initializes the Reactor. The Reactor then registers the specific objects or resources that the Dispatcher must observe.

- **Observation:** The **Dispatcher** is designed with the responsibility of monitoring for events. When an event occurs, the Reactor is notified.
- **Execution:** Upon receiving the notification, the Reactor triggers the **EventHandler**, which is a specialized component containing the execution logic for an event.
- **Feedback and Delegation:** Once the logic execution is complete, the status is routed through the **RequestHandler** back to the Client. The Client then delegates this information back to the Reactor.
- **State Management:** From this point, the Reactor can determine the next course of action, such as registering new events with the Dispatcher, switching internal system states, or executing further internal logic.

3.8 Event-Driven Kernel Monitoring

Applications requiring the capability to monitor kernel events such as network socket activity, inter-process communication (IPC) (Section 3.10), or changes in process states face challenges related to CPU overhead and latency. To remain responsive while observing thousands of event sources, the event-based monitoring frameworks `poll` and `epoll` were designed.

The `poll` model [23] represents a stateless, pull-based approach to monitor kernel components. In this model, an application must actively query the status of events from the kernel by providing a list of file descriptors (FDs) along with a requested event mask. An event mask is a bitwise filter defining which state changes, such as data availability or process termination, an application wishes to monitor. A **file descriptor** is an integer index that references a specific kernel object. For each entry in the submitted list, the kernel performs a lookup using the FD to locate the corresponding **struct file** within the process's descriptor table [46]. This structure maintains the resource's live status, which the kernel represents as an internal bitmask for comparison against the requested event mask. If a match is found (indicating that the requested event is active), the kernel sets the corresponding bits in the return bitmask to notify the application. Because the kernel must manually iterate through and verify the entire list of FDs and their masks during every call, the overhead scales directly with the number of monitored resources.

The `epoll` framework [22] addresses these scaling limitations by making the kernel responsible for tracking the event masks. Instead of the application resubmitting the list of masks for every check, `epoll` utilizes a callback mechanism. When an application registers a resource through an FD and an event mask, the kernel attaches a callback function directly to the underlying **struct file**. This creates a "push" notification model: when an event occurs that matches the mask, the callback is triggered, automatically adding the corresponding FD to a "ready list."

When the application queries `epoll`, it no longer needs to wait for the kernel to scan all registered FDs; it simply retrieves the FDs already gathered in the ready list. This shift provides the foundation for the Reactor pattern (Section 3.7). In this context, `epoll` serves as the event demultiplexer/dispatcher. While the kernel gathers various events into a single ready list, the Reactor demultiplexes this list by fetching and identifying each ready FD and dispatching it to its associated **EventHandler**. This design reduces computational cycles by eliminating redundant lookups of file descriptors.

3.9 CGroup

Control Groups (cgroups) were introduced to the Linux kernel in 2008. As described in the original proposal by Menage [28] and the official system documentation [20], this kernel feature organizes processes into hierarchical groups. While the operating system typically manages processes as individual, isolated Process Identifiers (PIDs), the cgroup feature places these PIDs into managed groups where resource limitations such as maximum RAM and CPU usage can be enforced.

When a process is assigned to a cgroup, any child process it spawns is automatically tagged by the kernel and placed into the same group. This creates a hierarchy that prevents an application from escaping its encapsulated environment by launching new threads or background tasks.

Within the Linux filesystem, each cgroup is assigned a unique identifier (ID). This identifier provides a mechanism for system observation when combined with eBPF described in section 3.6. Because eBPF programs can retrieve the cgroup ID associated with a process, they can check whether a specific system event originates from a particular cgroup ID, allowing activity to be filtered based on the cgroup context.

When these eBPF programs are further integrated with the **Linux Security Module (LSM)** framework explained in section 3.5, they can enforce rules regarding which system calls are permitted within a specific cgroup. The combination of these three features, cgroups, eBPF observation, and LSM enforcement, provides a mechanism for process encapsulation, which we refer to as sandboxing.

3.10 IPC Socket

Inter-Process Communication (IPC) sockets allow processes running on the same machine to communicate by exchanging data. In Linux, they are called Unix domain sockets (form of IPC) [24].

These sockets rely on the kernel to support such communication and use a local filesystem path as their address rather than an IP address used for network sockets [24]. While network sockets can communicate over the network, IPC sockets communicate locally and as a result, they are faster and add less overhead when communicating within the same system [8].

Unix domain sockets support both stream and datagram communication modes. In stream mode, they provide stream-oriented, connection-based communication similar to TCP. In datagram mode, they provide message-oriented communication similar to UDP. Additionally, Unix domain sockets offer a security advantage because access to socket files is controlled by filesystem permissions, allowing restrictions on which processes can connect to a socket [24].

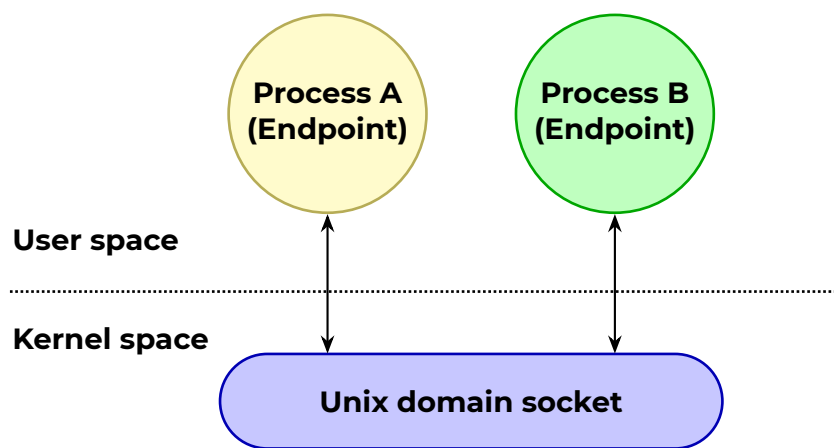


Figure 3.5: Unix Domain Socket

Chapter 4

Problem Analysis

This chapter analyzes the limitations of the TyrSecure project. By identifying the architectural gaps and security deficiencies in the initial prototype, it proceeds to define the broader technical challenges that must be addressed to develop an eBPF-based anti-cheat solution.

4.1 Previous Project: TyrSecure

TyrSecure is our 9th-semester student project aimed at determining whether eBPF programs can replace kernel modules in an anti-cheat solution on Linux [19]. The purpose of **TyrSecure** is to determine how much observability can be achieved through eBPF programs and whether they offer capabilities comparable to those of kernel modules with lower risk. This project was motivated by the risks of kernel-level anti-cheat systems, as kernel modules provide low-level observability and control over the operating system; however, they compromise stability, security, privacy, and user acceptance.

The project analyzed commonly used cheating techniques and their prevention. Based on the analysis, TyrSecure defined six requirements for an eBPF-based anti-cheat system to be considered a replacement for kernel modules. They include:

1. **Loadable Kernel Modules:** It should monitor the loading and unloading of kernel modules to detect suspicious modules.
2. **Detecting File Manipulation:** It should detect and prevent manipulation of the kernel's virtual file systems(VFS) such as /proc and /sys. The VFS provides kernel interfaces that attackers can manipulate.
3. **File Integrity:** It should detect malicious file changes. This can be done by setting up a known baseline of files by creating cryptographic hashes for them and checking the hashes periodically. Additionally, it can monitor system calls related to file integrity to detect suspicious behavior.
4. **Memory Scanning:** it should prevent common cheats, such as wallhacks and aimbots, that rely on game memory. This can be done by using eBPF at both the user and kernel levels, thereby preventing suspicious memory modifications.
5. **Virtual Machine:** it should detect whether a game is running inside a virtual machine, as virtual machines are commonly used to bypass anti-cheat systems. This can be done by identifying hardware-level behaviors due to the use of a hypervisor, such as differences in CPU

features, instruction behavior, execution timing, microarchitectural states, cache behavior, and performance counters. It could monitor these low-level artifacts to detect signs of virtualization.

6. **Secure Boot**: System should be booted in a cryptographically verified way to strengthen the security of the boot chain and the running kernel.

The anti-cheat prototype only fully implemented the memory scanning requirement and evaluated it by testing it on the AssaultCube game. Cheating behavior was simulated using memory scanning and debugging tools such as Scanmem and GDB. The evaluation showed that TyrSecure could detect and prevent runtime memory scanning and access. The results also revealed architectural limitations, as it had to attach its eBPF hooks after the game process had already started. This opened up an **attack surface** in which a cheat could already be injected before TyrSecure monitoring became active.

TyrSecure demonstrated that eBPF is a capable technology for implementing parts of a Linux anti-cheat system, such as runtime monitoring and the detection of suspicious behavior. It also demonstrated that eBPF programs provide low-level observability; however, the lack of implemented requirements and their architectural limitations showed that further development is needed.

4.1.1 Limitations of TyrSecure

The previous design of TyrSecure served as an initial exploration into eBPF-based anti-cheat mechanisms. However, evaluations of the architecture revealed limitations that prevented it from meeting its security objectives.

Architectural Limitations and Implementation Gaps

The design of TyrSecure, shown in Figure 4.1, required the game process to be active before monitoring could be attached. This architecture created an unmonitored window during process startup. Testing with **AssaultCube** verified that shared objects could be injected during this initialization phase, allowing execution of unauthorized scripts before detection became active. Consequently, these results prove that a runtime-only monitoring model provides insufficient detection, necessitating an architectural redesign to cover the pre-execution and execution phases.

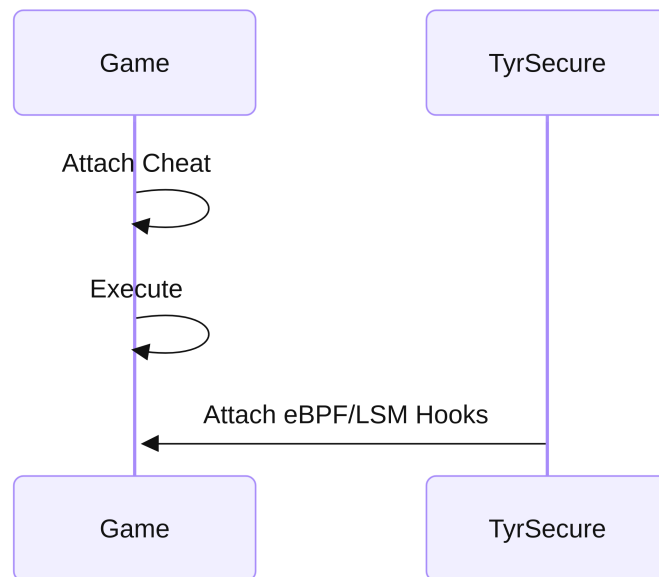


Figure 4.1: TyrSecure flow

Furthermore, while the design incorporated a single LSM hook, it relied on tracepoints and kprobes for data collection. While these are effective for observation, they are non-blocking and lack a mechanism for real-time policy enforcement. Without deeper integration with a server-side authority or a more extensive Linux Security Module (LSM) implementation, the system could only report suspicious activity rather than block it.

Requirements

As described earlier, TyrSecure defined six requirements that needed to be implemented in order to replace traditional kernel modules; however, as mentioned in Section 4.1, the implementation was significantly limited, addressing only a fraction of the requirements:

- **Memory Scanning:** This was the only requirement partially addressed. While the system could mitigate basic memory-tampering attacks, it remained vulnerable to techniques such as shared library injection. This indicates that the implementation focused on surface-level symptoms rather than on the process's underlying architectural security.
- **System-Level Protections:** The remaining five requirements covering file system integrity, kernel-level monitoring, and environment verification remained **unaddressed**. The complexity of the anti-cheat domain and the lack of an established Chain of Trust served as the primary obstacles.

Without a foundation to verify the integrity of the operating system, TyrSecure could not reliably determine whether the kernel had been compromised. If the kernel was altered, it could manipulate system calls, file paths, or other OS-level responses, making information reported by the operating system untrustworthy. The previous design identified critical attack surfaces but lacked the structural depth required to mitigate them, necessitating a more robust framework in the current iteration.

4.2 Coverage of process execution

As identified in TyrSecure’s limitations, the architecture failed to monitor the critical stages preceding and during process initialization. To better analyze these vulnerabilities, we divide the attack surfaces into four distinct phases: Kernel Space, Pre-Execution, Execution, and Post-Execution (see Figure 4.2).

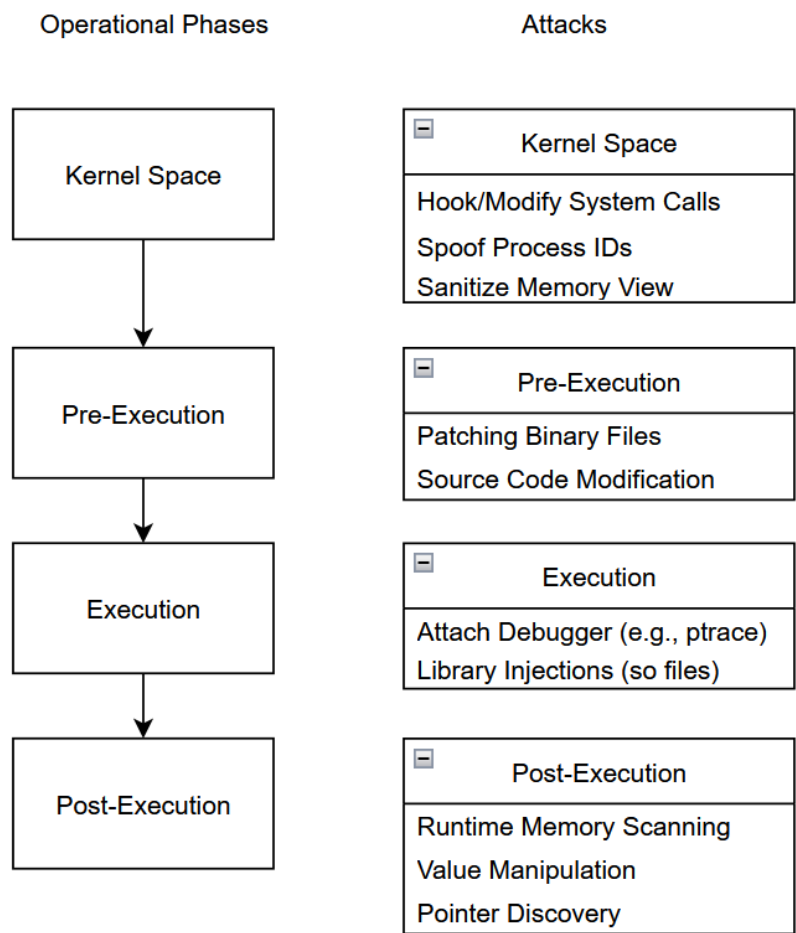


Figure 4.2: Operational phases and attacks

Phase 1: Kernel Space

The Kernel Space phase encompasses the underlying environment in which system calls are executed. If an attacker compromises this layer, they can tamper with the fundamental functions the anti-cheat relies on for data collection. By hooking or modifying system calls, the attacker can hide malicious files, spoof process IDs, or present a sanitized version of memory to the anti-cheat. In this scenario, the anti-cheat remains unaware that the game process is compromised because the information it receives from the OS is untrustworthy.

Phase 2: Pre-Execution

The pre-execution phase occurs before any binary data is loaded into volatile memory. Attacks at this stage typically involve directly modifying source code or patching binary files on disk. Without a File Integrity mechanism and an established Chain of Trust, the anti-cheat cannot verify the executable’s authenticity. This allows a tampered binary, potentially containing exploits or bypassing security checks, to be launched as if it were the original authorized version.

Phase 3: Execution

The execution phase is the window during which the operating system maps the executable and its dependencies into memory. This stage is a primary target for shared library injections and for attaching tools like ptrace for reverse engineering. Because TyrSecure's monitoring began only after this phase was complete, it was unable to detect malicious objects injected during the loading process, which is how the AssaultCube exploits were successfully deployed.

Phase 4: Post-Execution

The post-execution phase represents the active life of the process. This was the primary focus of the previous TyrSecure design, which targeted runtime attacks such as memory scanning and value manipulation. However, as the analysis shows, focusing solely on this phase is an "after-the-fact" approach. While the system could detect basic pointer discovery during runtime, it was unable to prevent or even identify the compromises that occurred in the three preceding phases.

4.3 Active Policy Enforcement

The analysis of TyrSecure revealed that its underlying architecture functioned primarily as a diagnostic tool rather than a defensive one. Because the implementation relied almost exclusively on non-blocking probes (kprobes and tracepoints) and utilized only a single LSM hook, the architecture was restricted to a "detect-and-report" model.

Observation alone does not prevent cheating. In the design of TyrSecure, there was a clear gap between detection and action. Since the monitoring mechanisms utilized probes, the system could log that a malicious system call had occurred, but lacked the authority to intervene. By the time an alert could be generated, the damage, such as an unauthorized shared library injection, had already been finalized.

Beyond these limitations, the system lacked a **policy engine** to determine which actions to terminate. Without a central "rulebook" to define and update what is allowed, there was no way to stop suspicious activity, whether it was a kernel event, a memory scan, or a new cheat technique. This confirms that without a way to enforce rules, an anti-cheat is just a witness to the cheating, not a defense against it.

4.4 Integrity

A prerequisite for **kernel-level security** is the ability to trust the underlying environment. The design of TyrSecure assumed that the kernel was "honest," yet it had no mechanisms to verify that assumption. This represents an architectural deficit: the lack of Kernel Integrity Verification.

In the context of an eBPF-based solution, the anti-cheat relies on the kernel to provide accurate data via hooks. However, if an attacker has already compromised the Kernel Space (Phase 1) for example, through a malicious Loadable Kernel Module (LKM) they can manipulate the system calls the anti-cheat uses for monitoring. Without a Chain of Trust or a verification process (such as Secure Boot and Remote Attestation to prove the system state to a server that contains the kernel source of truth), the anti-cheat cannot distinguish legitimate system responses from spoofed data provided by a rootkit.

Furthermore, this analysis identifies that integrity is a major requirement. To ensure a secure environment, three distinct components must be verified:

- The Kernel Space: To ensure the reliability of the hooks and the underlying OS primitives.

- The Anti-Cheat Software: To prevent stripping or patching of the security logic itself, which would allow a compromised client to report a false-positive clean state to the server.
- The Game Assets: To ensure that the process being monitored is the authentic, untampered version of the software.

If any one of these three parts is tampered with, the whole security system falls apart. You cannot rely on an anti-cheat to catch hackers if the very foundation it sits on is controlled by the hacker. This is the main problem we need to solve: ensuring every part of the chain is verified before we trust the data we see.

4.5 Threat Model

Attacker's Capabilities:

We assume the attacker is a privileged user with root access and the ability to leverage kernel features such as eBPF. In particular, loading/unloading eBPF programs, inspecting and modifying eBPF maps, and exploiting the kernel's ring buffer to monitor or manipulate communication between the kernel and user space. The attacker may have experience in reverse engineering and attempt to analyze our anti-cheat protections. Additionally, the attacker can modify or inject code into game files, disable our security components, and fake the environment in which the system operates.

Environment:

The host operates with Kernel Lockdown and Secure Boot enabled to enforce additional protection against kernel tampering and integrity violations during system initialization. It ensures the integrity of our system while the anti-cheat components are initializing. To validate the system environment, the anti-cheat mandates that all necessary components are active. If this verification fails, the software automatically terminates; furthermore, the server will refuse to establish a connection, preventing the cheater from launching the game.

Promise:

Our system is effective and functional, provided the Linux kernel remains uncompromised. Effectiveness is achieved through an event architecture. By using a single-threaded monitor driven by epoll, the system maintains a minimal resource footprint, leaving most CPU time for the game engine. Furthermore, the use of eBPF programs enables in-kernel filtering and rate limiting, ensuring the system remains resistant to overload from high-frequency security events. Additionally, the system's functionality is defined by proactively blocking core threat vectors via eBPF hooks using LSM. This includes preventing unauthorized memory manipulation, blocking the injection of untrusted libraries, denying the attachment of tracers or debuggers, and restricting unauthorized memory mappings at runtime.

Chapter 5

Problem Statement

The problem statement was derived from the limitations identified in the analysis of TyrSecure. The previous architecture relied on runtime-only monitoring, which left a blind spot during process startup. Results of TyrSecure showed that malicious shared objects could be injected before monitoring was attached, demonstrating that user-space observation initializes too late to provide reliable protection. Furthermore, the system primarily relied on non-blocking probes, which enabled detection but not prevention.

By analyzing the attack surface across the phases of Kernel Space, Pre-Execution, Execution, and Post-Execution, it became evident that securing only the active runtime of the game process was insufficient. The analysis also showed that monitoring cannot be treated independently of integrity: if the kernel, anti-cheat components, or game binaries cannot be trusted from boot onward, then the data collected later in runtime cannot be assumed to be reliable. This introduced a broader architectural requirement for a chain of trust rooted in system initialization, including boot-time protections such as Secure Boot, Kernel Lockdown, and verification of the anti-cheat and game environment before execution begins.

While TyrSecure provided a foundational exploration of eBPF-based monitoring, the problem analysis indicates that the client side requires additional functionality before it can benefit from a server-based integrity solution such as Remote Attestation. Specifically, the Linux Security Module (LSM) framework (see Section 3.5) is identified as the necessary mechanism to move beyond the observational limits of the previous work. This research, therefore, focuses on combining eBPF with LSM hooks to transition from passive detection to active enforcement. Server-based integrity mechanisms, such as those discussed in Section 4.4, are left outside the scope of this work. Instead, the scope is limited to establishing a stronger client-side foundation for protecting the game execution lifecycle.

The resulting problem statement reflects the challenges identified in the analysis:

How can a Linux client-side anti-cheat architecture use an eBPF-based solution to provide continuous monitoring and active enforcement across the game process lifecycle?

- Can the solution enforce protection from system initialization to post-execution of the game?
- Can eBPF detect the required events and use LSM hooks to enforce the corresponding policies?
- Can the LSM policies function with the game execution?

5.1 Requirements

To address the problem statement, we categorize the requirements into the three domains defined in the Integrity Analysis (Section 4.4): the kernel space, the anti-cheat software, and the game assets. These domains represent the architectural components that must be protected. Furthermore, these requirements are designed to provide coverage across the Game Execution Lifecycle (Section 4.2), ensuring that integrity is maintained from kernel space to post-execution.

Kernel Space

To ensure reliability, the underlying kernel must serve as a Root of Trust. A compromised kernel could potentially disable the eBPF framework and kernel features used by the anti-cheat. Establishing kernel integrity is therefore a prerequisite, necessitating the following requirement:

- **Kernel integrity.** The solution must provide a trusted kernel environment, supported by mechanisms such as Secure Boot, Kernel Lockdown, and Module Signature Enforcement to reduce the risk of kernel tampering.

Anti-Cheat Software

While the anti-cheat protects the game's integrity, its privileged position makes it a target for attackers. To ensure the system cannot be bypassed, the architecture must satisfy the following self-protection requirements:

- **Execution Precedence:** It must initialize and establish an active monitoring state before the game process begins its loading sequence to prevent early-stage injection.
- **Anti-Tampering and Integrity Verification:** It must implement mechanisms to verify the integrity of the anti-cheat components themselves, ensuring that kernel probes and user-space agents have not been modified or suppressed.
- **Privilege Isolation:** It must provide eBPF and kernel-level protections to isolate the security logic from user-space manipulation, even if the game process is compromised.
- **Resilience Against Termination:** It must detect and prevent unauthorized attempts to terminate the monitoring agent or detach kernel hooks during the game's lifecycle.

Game Assets and Process Lifecycle

The process execution analysis defined four critical stages in section 4.2. Since the first stage (Kernel Space) is already addressed, the solution must provide coverage for the remaining three stages to ensure protection across the entire game process lifecycle:

- **Pre-execution verification.** The system must verify the integrity of the game executable, anti-cheat components, and relevant runtime dependencies before execution begins.
- **Execution-phase monitoring.** The system must monitor the process loading stage to detect or prevent attacks such as shared library injection, unauthorized memory mappings, or debugger attachment during initialization.

- **Post-execution runtime monitoring.** It must deliver real-time protection against unauthorized debugging, memory inspection and tampering, process observation, modification of the protected game environment, and other suspicious process interactions to prevent runtime tampering.

Chapter 6

Design

This chapter presents the design of OdinSight, our anti-cheat system. OdinSight consists of two components: a daemon and a launcher. The following sections describe the functionality of both, including a breakdown of the individual components within the daemon.

6.1 Overview of OdinSight

This section presents a high-level overview of the OdinSight daemon and launcher. The **daemon** runs in the background as a system service and is responsible for launching and protecting games. The **launcher** provides an interface for requesting protected game launches from the daemon.

The daemon startup flow is shown in figure 6.1. First, the daemon checks the kernel environment using the **Kernel Integrity Validator**. The validator returns the result of this check to the daemon, which then decides whether the environment satisfies the required security conditions. If the validation succeeds, the daemon initializes the OdinEngine.

OdinEngine is the main coordination component of the daemon. It connects the daemon's subsystems and controls the overall execution flow after the kernel environment has been validated. Rather than performing protection, launching, or event handling directly, OdinEngine delegates these responsibilities to the specialized components described in the following sections. This keeps the daemon organized around a central control point while preserving clear responsibilities between the individual managers and handlers.

After initialization, the OdinEngine subscribes to relevant asynchronous events through the epoll manager. The epoll manager then notifies the engine when registered events occur. Once the engine is running, it can request game execution through the runner, command listener, and eBPF manager components, which return the execution result back to the engine.

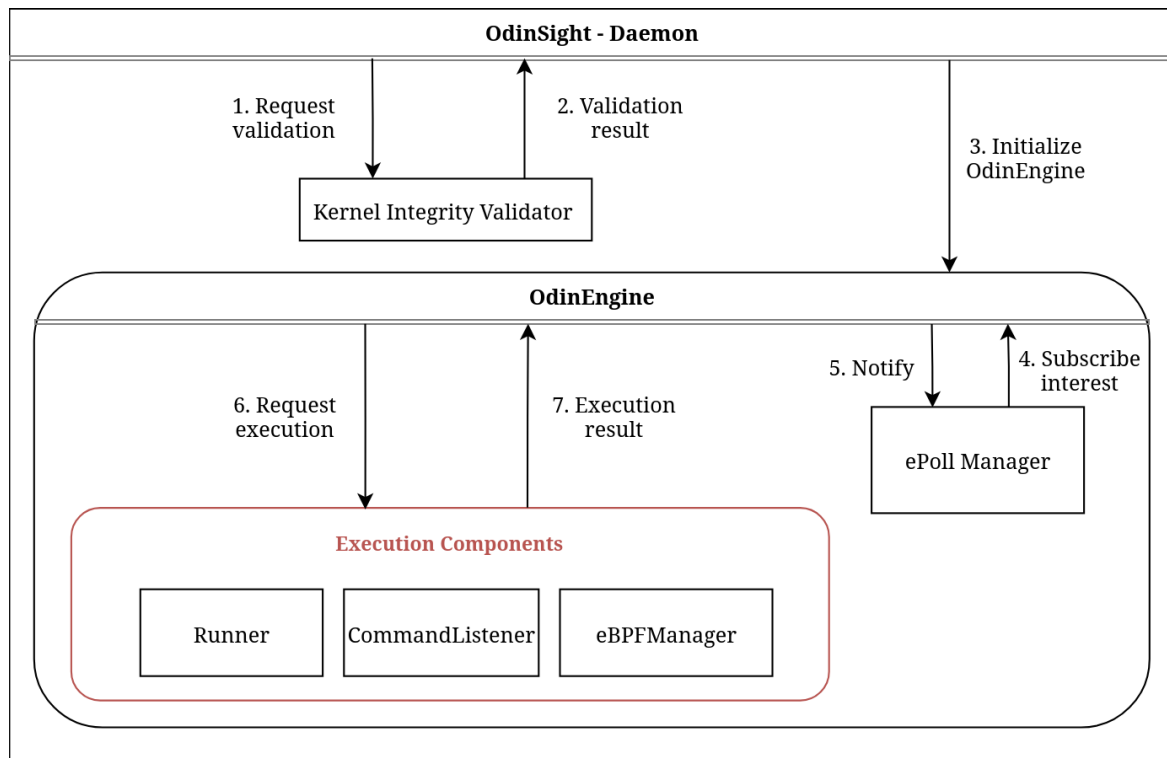


Figure 6.1: High-level overview of the OdinSight daemon

The **CommandListener** creates the local command socket used by the launcher and receives launch requests through it. The **Runner** is responsible for preparing and starting the protected game process. The **eBPF Manager** loads and manages protection hooks, while the **ePoll Manager** coordinates asynchronous events, including launcher requests, eBPF notifications, and game process terminations.

Launcher

The launcher communicates with the daemon using a local command socket. At a high level shown in figure 6.2, the daemon listens for launch requests, the launcher sends a launch packet, and the daemon validates the request before preparing the protection environment and starting the game. The launcher does not directly apply protection or make trusted enforcement decisions.

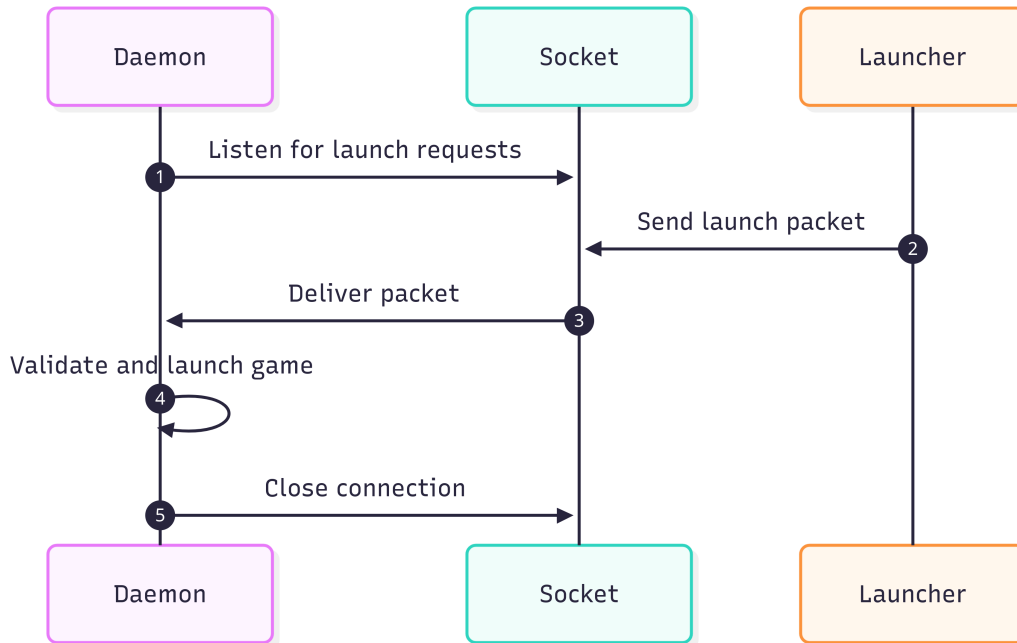


Figure 6.2: High-level interaction between the daemon, socket, and launcher

This separation of responsibilities is an intentional design choice in OdinSight, where sensitive security operations are isolated within the privileged daemon, while the launcher remains an unprivileged interface for user interaction. This architecture minimizes the attack surface by limiting the amount of trusted code exposed to the user environment, reduces the risks associated with launcher tampering, and establishes a clear boundary between user interactions and core protection enforcement.

6.2 Kernel Integrity Validator

The Kernel Integrity Validator is designed as the first stage of the OdinSight daemon initialization process. Its purpose is to prevent the daemon from starting unless the underlying kernel environment satisfies the security assumptions defined in the Kernel Integrity Requirements section 5.1.

Figure 6.3 shows the first stage of the validation process. The validator reads **kernel-exposed security interfaces** to determine whether Secure Boot, Kernel Lockdown, and Kernel Module Signature Enforcement are active.

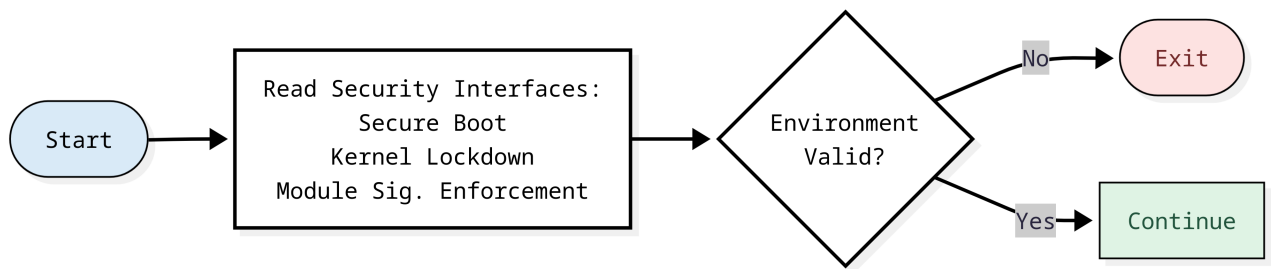


Figure 6.3: Environment Validation

In addition to reading the reported kernel state, the validator performs an active enforcement test by attempting to load an **unsigned kernel module**, as shown in Figure 6.4. This prevents the validator from relying solely on reported configuration values. Instead, it verifies that the expected security mechanisms are enforced in practice.

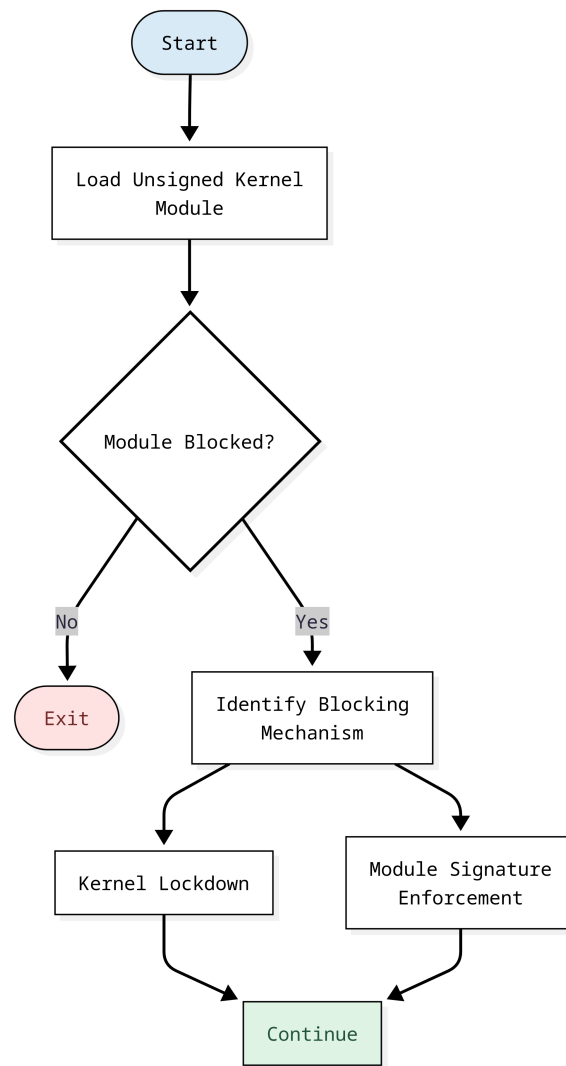


Figure 6.4: Load Unsigned Kernel Module

If the required kernel security settings are missing, or if the unsigned module can be loaded successfully, the environment is treated as invalid and the daemon exits. This ensures that OdinSight only continues when the host meets the kernel integrity assumptions.

6.3 ePoll Manager

The daemon uses an event-driven architecture based on the **Reactor Pattern** described in Section 3.7. This design allows the daemon to react to operating system events without continuously polling each subsystem individually. Instead, components register the events they are interested in, and the event system dispatches notifications when those events occur.

The ePoll Manager is responsible for the low-level interaction with the Linux **epoll** interface described in the background section 3.8. It maintains the set of file descriptors the daemon needs to observe, such as those associated with eBPF event streams, process exit state, and the launch socket. These file descriptors represent communication channels exposed by the operating system. When one of the registered descriptors becomes ready, it triggers handlers, such as the eBPFManager component described in section 6.4.

As shown in Figure 6.5, the OdinEngine registers the relevant file descriptors with the ePoll Manager during initialization. Once an event occurs, the ePoll Manager reports it back to the OdinEngine, which serves as the central coordination point. The OdinEngine then forwards the notification to the appropriate EventHandler. The OdinEngine then forwards the notification to the appropriate EventHandler.

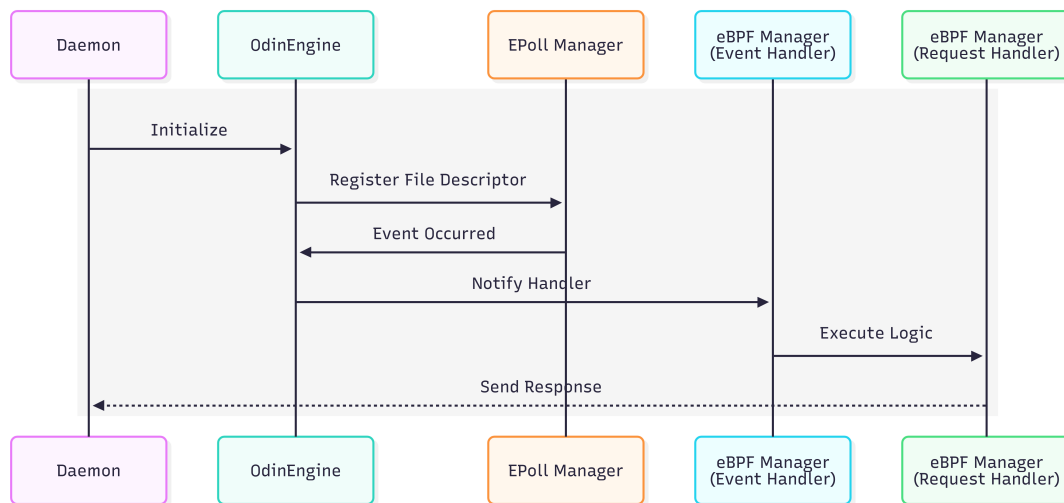


Figure 6.5: Event System

For example, the eBPF Manager can be viewed as both an event handler and a request handler in this design. As an event handler, it reacts to readiness notifications from eBPF ring-buffer file descriptors and consumes kernel events emitted through those buffers. As a request handler, it can be invoked by OdinEngine when the daemon needs to attach, detach, or update eBPF protection modules.

This same separation applies to other daemon components that interact with the ePoll Manager, such as the CommandListener and the Runner. The ePoll Manager only detects that a registered file descriptor is ready, while OdinEngine and the components decide how their events should affect the daemon's state.

6.4 eBPF Manager

The eBPF Manager forms the boundary between the daemon and the eBPF programs. It provides an interface for managing **protection modules**, while hiding lower-level details of libbpf, eBPF skeletons, and maps. In the design, the eBPF Manager has two main responsibilities: controlling the lifecycle of eBPF modules and collecting the events those modules emit and returning them to user space. This makes it the central component connecting **kernel-side enforcement** to the daemon's event-driven architecture.

As shown in Figure 6.6, the eBPF Manager owns the user-space lifecycle of the eBPF modules. It loads modules into the kernel, attaches their hooks, detaches them when they are no longer needed, and consumes events emitted through the shared ring buffer.

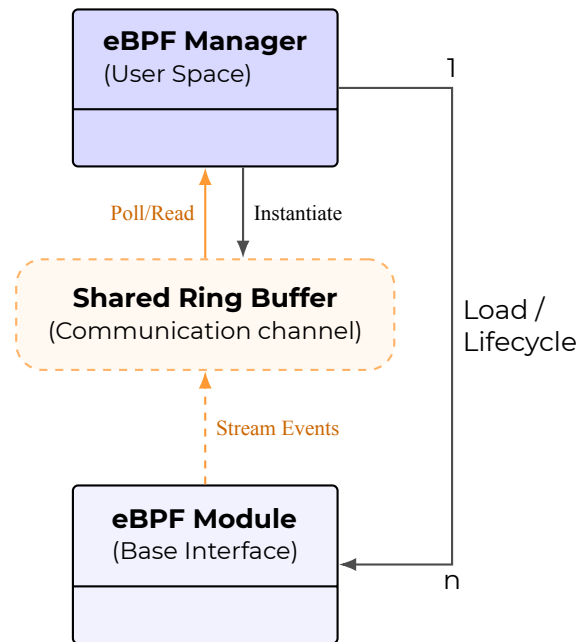


Figure 6.6: Architecture of eBPF Manager.

During daemon initialization, the eBPF Manager loads the default **daemon protection hooks**. These hooks protect the daemon itself and are active before any game process is launched. Later, when the Runner prepares a game execution environment and creates the corresponding cgroup context, OdinEngine invokes the eBPF Manager to load the **game protection hooks** for that cgroup. This means that game-specific enforcement is only attached once the daemon has a concrete game process context to protect.

All loaded modules reuse the same shared ring buffer rather than creating separate buffers. The eBPF Manager's ring buffer's polling file descriptor is exposed to the ePoll Manager, so the buffer can participate in the daemon's event loop. When the ring buffer becomes readable, the ePoll Manager reports the readiness event, and the eBPF Manager consumes the pending records.

Each event contains a **module identifier**. After consuming an event from the shared ring buffer, the eBPF Manager uses this identifier to dispatch the event to the module that produced it. The module then handles the event through its own `processEvent` implementation. This keeps the shared transport generic while allowing modules such as the daemon protection module and the game protection module to define their own event-specific handling logic.

6.5 Runner

The Runner is responsible for the **game execution lifecycle**. Together with the eBPF Manager and the game protection eBPF hooks described in the implementation chapter, it supports the pre-execution, execution, and post-execution phases described in Section 4.2. It transforms a validated launch request into a controlled game session by preparing the execution context, starting the game inside a CGroup, and cleaning up the session after the process exits.

The Runner is reached through the daemon's command flow. The `CommandListener` receives a launch packet from the launcher and extracts the requested `GameID`. OdinEngine then uses this identifier to prepare the Runner. The Runner checks the identifier against the **game whitelist**, which defines the games that the daemon is allowed to launch, their executable paths, working directories, and any required environment variables.

6.5.1 Pre-Execution

The pre-execution stage (Figure 6.7) involves the coordination between the Runner component and the eBPF Manager. The Runner initiates the process by instantiating a new CGroup. Once created, the unique CGroup ID is passed to the eBPF Manager. This ID is essential for the eBPF Manager to deploy targeted hooks that specifically monitor and protect the designated sandbox environment.

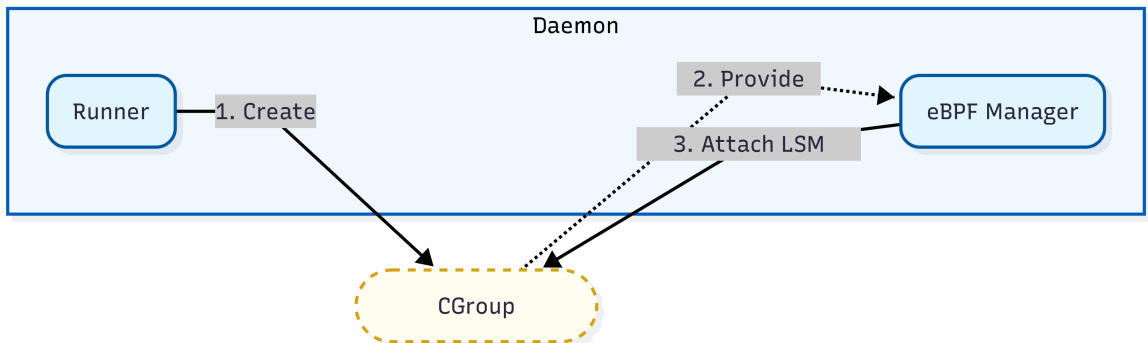


Figure 6.7: Pre-execution design and CGroup initialization.

6.5.2 Execution

In the execution stage (Figure 6.8), the Runner triggers a system call to spawn a child process. Because this process is born within the previously initialized CGroup, it immediately inherits all security constraints and resource limits. This child process then executes the game binary, ensuring that it is protected by the eBPF hooks from its first instruction onward.

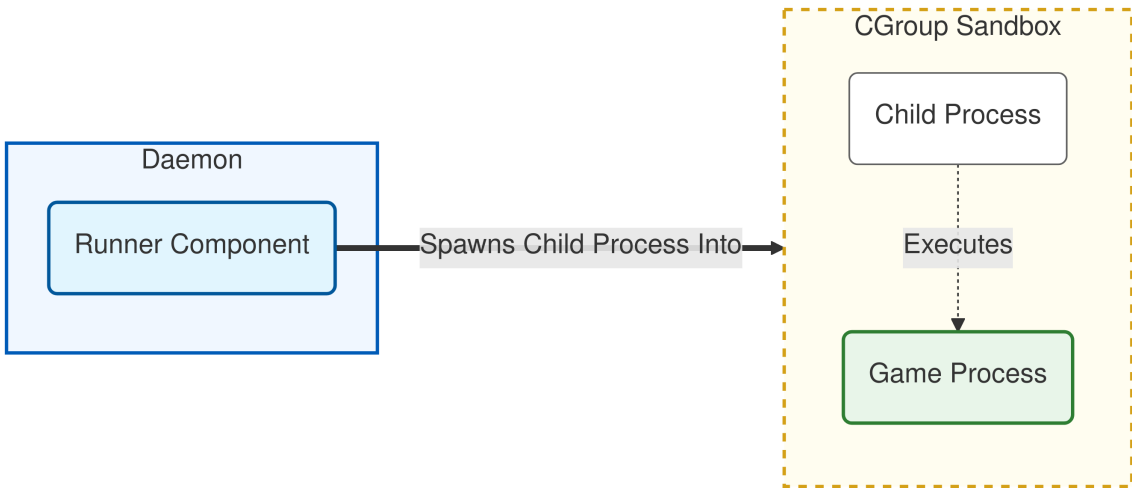


Figure 6.8: Execution stage and process spawning.

6.5.3 Termination and Cleanup

The final stage handles the decommissioning of the sandbox upon game exit (Figure 6.9). When the game process terminates either through a standard shutdown or a forced exit the Event System (detailed in Section 6.3) catches the signal. The EPollManager notifies the Runner, which executes a cleanup routine. This routine reaps all remaining processes within the CGroup to prevent "zombie" or "ghost" processes from persisting. Finally, the CGroup is removed, resetting the system for the next execution cycle.

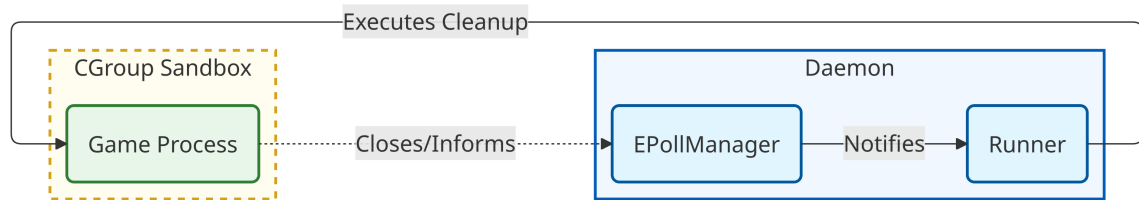


Figure 6.9: Termination and environment cleanup.

Chapter 7

Implementation

This chapter presents the implementation of OdinSight's eBPF-based protection system. The implementation consists of configuration and two eBPF programs: `daemon_hardener.bpf.c`, which protects the OdinSight daemon, and `game_protection.bpf.c`, which protects the launched game processes. Both programs are implemented as eBPF LSM modules and enforce their policies through kernel-level security hooks.

The two modules are configured via their respective eBPF module implementations before being loaded into the kernel. The **daemon protection module** is configured with the daemon process ID through the `DAEMON_PID` constant. The **game protection module** is configured with both the daemon process ID and the target cgroup ID, allowing it to identify trusted daemon activity and protected game processes. As described in Section 6.4, the daemon protection module is loaded during initialization of the eBPF manager, while the game protection module is loaded later, when the Runner launches a game, and the target cgroup has been created.

7.1 eBPF Module Configuration

Before OdinSight loads its eBPF programs into the kernel, it configures each module with the runtime identifiers required for enforcement. These identifiers are not known at compile time, because they depend on the running daemon process and the cgroup created for the protected game process.

Figure 7.1 shows how OdinSight configures the two eBPF modules before they are loaded into the kernel. Each module first opens its generated libbpf skeleton. After the skeleton has been opened, the module writes runtime-specific identifiers into the skeleton's read-only data section. These values become constants for the eBPF program once it is loaded.

```

1  Odin::Result<void> DaemonModule::open() {
2      m_skel.reset(daemon_hardener__open());
3      if (!m_skel) {
4          return std::unexpected(Error::System(ctx, "daemon_hardener__open",
5          },
6
7      uint32_t current_pid = static_cast<uint32_t>(::getpid());
8
9      m_skel->rodata->DAEMON_PID = current_pid;
10     return {}; // Success
11 }
12
13 Odin::Result<void> GameModule::open() {
14     m_skel.reset(game_protection__open());
15     if (!m_skel) {
16         return std::unexpected(Error::System(ctx, "daemon_hardener__open",
17         },
18
19     uint32_t current_pid = static_cast<uint32_t>(::getpid());
20
21     m_skel->rodata->TARGET_CGROUP = m_cg_id;
22     m_skel->rodata->DAEMON_PID    = current_pid;
23     return {}; // Success
24 }

```

Figure 7.1: Configuration of `daemon_hardener` and `game_protection`.

During OdinSight initialization, `DaemonModule::open()` reads the current daemon process ID with `getpid()` and writes it to `m_skel->rodata->DAEMON_PID`. The daemon hardening hooks use this value to determine whether an operation targets the OdinSight daemon and whether the caller is the daemon itself or an external process.

In `GameModule::open()`, OdinSight performs a similar configuration step for the game protection program. This step is triggered during the pre-execution phase highlighted in Figure 6.7, after the daemon has received a game launch request from `OdinSight_launcher`. The module writes the current daemon process ID to `DAEMON_PID` and the protected cgroup identifier to `TARGET_CGROUP`, as provided by the Runner component. The daemon PID is used to recognize trusted daemon activity, while the cgroup ID is used to identify the game processes that should be protected by the hooks.

This configuration occurs before the eBPF manager calls each module's `load()` and `attach()` methods. These methods are not shown here, since they mainly perform the standard libbpf loading and attachment procedure. The important detail is that the runtime identifiers are written before the programs enter the kernel, so the hooks begin enforcement with the correct daemon and cgroup identifiers already available.

7.2 Daemon Protection Hooks

The daemon hardener program is loaded through the `DaemonModule` as described in the previous section 7.1 and contains the LSM hooks responsible for protecting the OdinSight daemon from external interference. These hooks enforce restrictions on operations that could be used to inspect, terminate, deprioritize, or otherwise weaken the daemon while it is managing OdinSight's protection mechanisms.

Two implemented LSM hooks are shown in this section. The first shows the `lsm/bpf` hook, which restricts access to the `bpf()` system call. The second shows the `lsm/ptrace_access_check` hook, which prevents unauthorized debugging and tracing of the daemon. The remaining hooks listed in Table 7.1 follow the same general structure, but apply it to different kernel operations.

Hook	Purpose	Status
<code>lsm/bpf</code>	Restricts <code>bpf()</code> syscalls	Implemented
<code>lsm/task_setscheduler</code>	Prevents scheduling changes	Implemented
<code>lsm/task_setnice</code>	Prevents priority tampering	Implemented
<code>lsm/task_setioprio</code>	Prevents I/O throttling	Implemented
<code>lsm/task_setrlimit</code>	Prevents resource limit drops	Implemented
<code>lsm/inode_permission</code>	Blocks <code>procfs</code> inspection	Implemented
<code>lsm/ptrace_access_check</code>	Blocks debugging	Implemented
<code>lsm/task_kill</code>	Prevents unauthorized signals	Implemented

Table 7.1: Daemon Protection Hooks and Implementation Status.

The `lsm/bpf` hook implementation is shown in Figure 7.2. This hook allows BPF operations only when they originate from the configured daemon process. This is important because OdinSight relies on eBPF programs and maps as part of its enforcement mechanism. If another privileged process were allowed to interact freely with BPF objects, it could inspect, replace, unload, or otherwise interfere with the protection logic. Restricting BPF access to the daemon reduces this attack surface and ensures that only OdinSight's trusted control process can manage the eBPF components.

```
1  SEC("lsm/bpf")
2  int BPF_PROG(restrict_bpf_to_self, int cmd, union bpf_attr *attr,
3              unsigned int size)
4  {
5      if (DAEMON_PID == 0) return 0;
6
7      __u32 current_pid = bpf_get_current_pid_tgid() >> 32;
8      if (current_pid != DAEMON_PID) {
9          return -EPERM;
10     }
11     return 0;
12 }
```

Figure 7.2: LSM hook that controls access to the `/proc` file system. Edited for readability.

The `lsm/ptrace_access_check` hook implementation is shown in Figure 7.3. This hook prevents unauthorized processes from using `ptrace` against the OdinSight daemon. Linux provides `ptrace`

so that one process can inspect or control another process, which is commonly used by debuggers and tracing tools. For the daemon, this access must be restricted because an external process could read its memory, observe its behavior, or interfere with its execution.

The hook uses a shared PID-based helper, which is also used by other hooks in the daemon protection list, to verify that the target process is the configured OdinSight daemon. If `DAEMON_PID` has not been set, the operation is allowed. Otherwise, operations targeting unrelated processes are ignored, while operations targeting the daemon are allowed only when they originate from the daemon itself. All other attempts are rejected with `-EPERM`.

```
1 static __always_inline int is_unauthorized_external_actor(struct
   task_struct *p)
2 {
3     if (DAEMON_PID == 0) return 0;
4
5     if (BPF_CORE_READ(p, tgid) == DAEMON_PID) {
6         __u32 current_pid = bpf_get_current_pid_tgid() >> 32;
7
8         if (current_pid == DAEMON_PID || current_pid == 0) {
9             return 0;
10        }
11        return -EPERM;
12    }
13    return 0;
14 }
15
16 SEC("lsm/ptrace_access_check")
17 int BPF_PROG(ptrace_proc, struct task_struct* child, unsigned int
   mode) {
18     return is_unauthorized_external_actor(child);
19 }
```

Figure 7.3: LSM hook that prevents unauthorized ptrace access to the daemon.

The remaining daemon protection hooks apply the same authorization model to other operations that could weaken or disrupt the daemon.

The `lsm/task_kill` hook prevents external processes from sending signals to the daemon, including termination signals. The daemon is still allowed to signal itself, and kernel-originated operations are allowed, so normal cleanup behavior is not blocked.

The scheduling and resource hooks protect the daemon from indirect interference. The `lsm/task_setscheduler`, `lsm/task_setnice`, and `lsm/task_setioprio` hooks prevent external processes from changing the daemon's scheduling policy, CPU priority, or I/O priority. Without these checks, another process could reduce the daemon's ability to run reliably by lowering its priority or throttling its I/O. Similarly, the `lsm/task_setrlimit` hook prevents external changes to the daemon's resource limits, which could otherwise be used to restrict memory, file descriptors, or other resources required for normal operation.

The `lsm/inode_permission` hook protects daemon-related entries in the `/proc` file system. It first checks whether the accessed inode belongs to `procfs`, and then determines whether the `procfs` entry refers to the configured daemon PID. If an external process attempts to access the daemon's `procfs` information, the operation is rejected with `-EPERM`. This reduces information leakage through files such as process status, memory maps, and other runtime metadata exposed through `/proc`.

The hardener also includes `lsm/ptrace_traceme`, which covers a related `ptrace` case. While `ptrace_access_check` blocks external processes from attaching to the daemon, `ptrace_traceme` prevents the daemon from being started or controlled under an unauthorized tracing parent. Together, these hooks reduce both direct and indirect access for debugging the daemon.

7.3 Game Protection Hooks

The game protection program is loaded via the `GameModule`, as described in the `eBPF Module Configuration` section 7.1, and includes the LSM hooks that protect game processes from external interference. These hooks enforce restrictions on operations that could be used to inspect, modify, trace, or otherwise tamper with protected game processes while `OdinSight` is active.

Two implemented LSM hooks are shown in this section. The first shows the `lsm/mmap_file` hook, which blocks suspicious executable memory mappings in protected game processes. The second shows the `lsm/bprm_check_security` hook together with the related execution tracepoints, which blocks `LD_PRELOAD` based library injection. The remaining hooks listed in Table 7.2 follow the same general structure, but apply it to different kernel operations.

Hook	Purpose	Status
<code>lsm/ptrace_access_check</code>	Blocks tracing of protected game processes	Implemented
<code>lsm/ptrace_traceme</code>	Blocks protected processes from requesting parent tracing	Implemented
<code>lsm/mmap_file</code>	Blocks W+X/anonymous exec; scaling issues	Partial
<code>lsm/file_mprotect</code>	Blocks suspicious memory transitions	Implemented
<code>lsm/inode_permission</code>	Blocks <code>cgroupfs</code> and <code>procfs</code> access	Implemented
<code>lsm/sb_mount</code>	Blocks mount operations targeting the protected <code>cgroup</code> path	Implemented
<code>lsm/move_mount</code>	Blocks moving mounts into or out of the protected <code>cgroup</code> path	Implemented
<code>lsm/bprm_check_security</code>	Blocks execution when <code>LD_PRELOAD</code> is detected	Implemented
<code>lsm/capable</code>	Limits capability use in protected game processes	Partial
<code>lsm/perf_event_open</code>	Blocks performance-event inspection	Implemented

Table 7.2: Game Protection Hooks and Implementation Status.

The `lsm/mmap_file` hook implementation is shown in Figure 7.4. This hook prevents protected game processes from creating suspicious executable memory mappings. It first respects any earlier LSM decision by returning `ret`. It then allows the `OdinSight` daemon and ignores processes outside the protected game `cgroup`. For protected game processes, the hook checks both the permissions requested by the caller and the permissions that would be applied to the mapping. If either set of permissions would make the mapping both writable and executable, the operation is rejected with `-EPERM`. The hook also rejects anonymous executable mappings because they can be used to insert code directly into the process address space.

This hook is marked as partially implemented because the current policy does not distinguish between malicious executable mappings and all legitimate mappings that some games or runtimes may require.

```

1 SEC("lsm/mmap_file")
2 int BPF_PROG(restrict_mmap_file, struct file* file, unsigned long
   reqprot, unsigned long prot,
3         unsigned long flags, int ret) {
4     if (ret) return ret;
5
6     caller_ctx caller = get_caller_ctx();
7
8     if (is_daemon_process(caller.pid) || !is_protected_cgroup(caller.
   cgid)) return 0;
9
10    const bool is_wx_mapping = ((reqprot & PROT_WRITE) && (reqprot &
   PROT_EXEC)) ||
11                                ((prot & PROT_WRITE) && (prot &
   PROT_EXEC));
12
13    const bool is_anonymous_exec = !file && ((reqprot & PROT_EXEC) ||
   (prot & PROT_EXEC));
14
15    if (is_wx_mapping) {
16        return -EPERM;
17    }
18
19    if (is_anonymous_exec) {
20        return -EPERM;
21    }
22
23    return 0;
24 }

```

Figure 7.4: LSM hook that prevents unauthorized signal delivery to the daemon.

The `lsm/bprm_check_security` hook implementation is shown in Figure 7.5. This hook is used together with the `execve` and `execveat` tracepoints to block LD_PRELOAD based library injection in protected game processes.

The tracepoint hooks run when `execve()` or `execveat()` is entered, at which point the environment pointer is available. They pass this environment pointer to `detect_ld_preload_from_envp()`, which inspects the environment and stores the current process ID in the bpf map if LD_PRELOAD is present. Later, when `bprm_check_security` is called for the same execution, the hook checks this map. If no stored entry exists, the execution is allowed. If an entry exists, it is removed, and the execution is rejected with `-EPERM`.

```

1 SEC("tracepoint/syscalls/sys_enter_execve")
2 int detect_ld_preload_execve(struct trace_event_raw_sys_enter* ctx)
3 {
4     detect_ld_preload_from_envp((const char* const*) ctx->args[2]);
5     return 0;
6 }
7 SEC("tracepoint/syscalls/sys_enter_execveat")
8 int detect_ld_preload_execveat(struct trace_event_raw_sys_enter* ctx
9 ) {
10     detect_ld_preload_from_envp((const char* const*) ctx->args[3]);
11     return 0;
12 }
13 SEC("lsm/bprm_check_security")
14 int BPF_PROG(block_ld_preload_exec, struct linux_binprm* bprm, int
15     ret) {
16     __u8* mark;
17
18     if (ret) return ret;
19
20     caller_ctx caller = get_caller_ctx();
21
22     if (!is_protected_cgroup(caller.cgid)) {
23         bpf_map_delete_elem(&blocked_exec_pids, &caller.pid);
24         return 0;
25     }
26
27     mark = bpf_map_lookup_elem(&blocked_exec_pids, &caller.pid);
28     if (!mark) return 0;
29
30     bpf_map_delete_elem(&blocked_exec_pids, &caller.pid);
31
32     return -EPERM;
33 }

```

Figure 7.5: Tracepoints and LSM hook that block LD_PRELOAD-based execution in the protected game cgroup.

The remaining game protection hooks apply the same cgroup-based enforcement model to other operations that could be used to inspect, modify, or interfere with protected game processes.

The `lsm/ptrace_access_check` hook blocks external processes from tracing protected game processes. It allows the OdinSight daemon and ignores targets outside the protected cgroup. If the target is inside the protected game cgroup, the hook only allows the process to inspect itself; other tracing attempts are rejected with `-EPERM`. The related `lsm/ptrace_traceme` hook blocks protected game processes from requesting that their parent process trace them.

The `lsm/file_mprotect` hook complements `mmap_file` by checking memory permission changes after a mapping already exists. It is used to detect suspicious transitions such as writable and executable memory or anonymous executable memory, which may indicate runtime code injection.

The `lsm/inode_permission` hook restricts access to cgroupfs and procfs entries related to the protected game cgroup. For cgroupfs, it prevents external processes from writing to protected cgroup files. For procfs, it prevents external processes from inspecting procfs entries that belong to protected game processes.

The `lsm/sb_mount` and `lsm/move_mount` hooks protect the cgroup path used to identify protected game processes. The `sb_mount` hook blocks mount operations that target the protected cgroup path, while `move_mount` blocks mounts from being moved into or out of that path. This helps preserve the integrity of the cgroup boundary used by OdinSight.

The `lsm/capable` hook limits capability use inside the protected game cgroup. In the implementation, setup-related capabilities such as `CAP_SETUID` and `CAP_SETGID` are allowed, while other capability checks from protected game processes are rejected. It is marked as partially implemented because a more complete implementation would define a more detailed allowlist based on the exact capabilities required by the protected game and runtime environment.

The `lsm/perf_event_open` hook blocks performance event monitoring, reducing the ability to observe protected game processes through profiling or sampling interfaces.

Chapter 8

Evaluation

This chapter presents the performance and functional evaluation of OdinSight. The evaluation is structured around three pillars: functional coverage, which verifies that OdinSight correctly identifies and blocks prohibited actions; compatibility analysis, which assesses OdinSight's ability to operate across diverse game titles without preventing execution; and performance benchmarking, which quantifies the overhead introduced by the OdinSight daemon.

To ensure reproducible results, all tests were conducted in a standardized environment. This chapter first details the host configuration and tools employed, followed by the specific methodologies for our coverage, compatibility, and performance tests. The final results of these evaluations are presented in the subsequent sections of this chapter.

8.1 Test Environment

To ensure reproducibility and consistency, a controlled test environment was established. The host system configuration was standardized across all tests defined in the methodology section 8.2, ensuring that the coverage, compatibility, and performance tests were executed on the same hardware. Within this environment, specific tools were selected based on the objectives of each test category. In addition, a defined set of game titles was selected to evaluate compatibility with OdinSight, while custom-developed libraries were used to inject code into the protected game process for specific coverage test scenarios.

8.1.1 Host System Configuration

The evaluations were performed on Arch Linux using the same hardware and software configuration to ensure comparable results. Table 8.1 summarizes the system configuration used during testing.

Category	Details
OS	Arch Linux (Kernel: 7.0.9-arch1-1)
Environment	Hyprrland 0.55.2 (Wayland)
CPU	AMD Ryzen 7 5800X (16 threads) @ 4.85 GHz
GPU	AMD Radeon RX 6900 XT (Discrete)
Memory	Corsair Vengeance 8GB DDR4-3600 RAM (x4) - 31.25 GiB Total
PSU	Corsair RM1000e 1000W ATX PSU Fully Modular (CP-9020297-EU)
Motherboard	Asus TUF Gaming GT301 VG-Edition
Displays	1x 2560x1440 (240Hz), 2x 1920x1080 (144Hz)
Storage	Kingston 1TB A2000 M.2 NVMe SSD

Table 8.1: System configuration used for testing

8.1.2 Game Set

The five games selected for testing are presented in Table 8.2. This table includes four attributes for each game: title, genre, platform, and version. These attributes are included because they may influence compatibility with OdinSight.

Game Title	Genre	Platform	Version
AssaultCube	First-Person Shooter	Open Source	1.2.0.2
SuperTuxCart	Racing	Open Source	1.5
A Story About My Uncle	Adventure/Platformer	Steam	BUILD V. 5188
Baldur's Gate 3	RPG	Steam	4.1.1.7209685
Valheim	Survival	Steam	1-0.221.12 (n-36)
Unturned	Survival	Steam	3.26.3.2

Table 8.2: Games used in testing

These games were selected to cover different execution environments, game engines, distribution platforms, and multiplayer properties. SuperTuxKart was included because it is an open-source, native Linux game with multiplayer support, making it useful for testing compatibility with another freely available game. A Story About My Uncle was selected to represent a native Linux game distributed through Steam, despite being single-player, in order to evaluate whether OdinSight remains compatible with Steam-launched games. Baldur's Gate 3 was included as a modern commercial Steam game with multiplayer functionality, providing a more complex test case than the open-source titles. Finally, Valheim and Unturned were selected because they are Unity-based multiplayer games, allowing us to evaluate compatibility with games built on a widely used commercial game engine.

8.1.3 Tools

The tools utilized during the evaluation are detailed in Table 8.3. This table provides the name, version, and a description for each tool, ensuring the reproducibility of the test environment.

Tool name	Version	Description
GDB	17.2	Memory scanner and debugger tool
Scanmem	0.17	Memory scanner and debugger tool
bpftool	7.7.0 (libbpf 1.7)	Helper tool to manage bpf hooks
ac_rhack	2021-06-04	Open-source cheat program
Unigine Superposition	1.1	Benchmarking tool for PC hardware
MangoHud	0.8.3	Linux FPS and GPU/CPU overlay tool

Table 8.3: Tools used in testing

GDB and **Scanmem** provide with external runtime inspection techniques. **GDB** provides debugging functionality and can attach to a running game process to inspect memory, registers, and execution state, while **Scanmem** complements this by providing a more focused form of memory scanning. Rather than acting as a full debugger, it searches a process address space for specific or changing values, such as in-game health and ammunition. Together, these tools represent common approaches for observing and manipulating a game process during runtime.

For eBPF management, we use **bpftool** for interfacing with the eBPF framework. It can attach/detach eBPF hooks, monitor eBPF maps, and collect diagnostic logs via `bpf_printk`, which provides the data necessary to validate eBPF hooks.

A GitHub project called **ac_rhack** is a cheat developed for Assault Cube. It uses a cheat technique known as library injection via `LD_PRELOAD`, which allows additional functionality to be loaded into the game. This library enables hacks such as aimbot and wallhack. Importantly, the project follows ethical boundaries and is intended to operate strictly in single-player mode to prevent misuse and disruption in multiplayer environments.

Unigine Superposition serves as a benchmark tool for evaluating graphics performance and system stability under a heavy GPU workload. It provides controlled, repeatable workloads by rendering the same 3D scene across test runs while allowing the graphical difficulty to be customized. This makes it possible to stress the system under consistent conditions and reliably compare performance results, as each benchmark follows the same rendering process.

MangoHud is a performance monitoring overlay for Linux that tracks real-time metrics, such as frame rates, frame times, and CPU/GPU usage. In addition to its on-screen display, it automatically logs these metrics to CSV files. These logs generate the precise data points needed to identify performance bottlenecks and reliably compare results across test runs.

8.1.4 Custom Libraries

For the coverage tests detailed in Section 8.2.2, certain test cases cannot be executed via standard shell commands. To perform these evaluations, we developed three custom shared libraries to be injected into the game process. These libraries are used to trigger parent-based tracing, anonymous executable memory allocation, and changes to writable-executable memory permissions. Each library utilizes the `__attribute__((constructor))` specifier, ensuring the code executes automatically upon library injection via the `LD_PRELOAD` environment variable.

```

1  #include <sys/ptrace.h>
2
3  extern "C" __attribute__((constructor)) void run_ptrace_traceme_test
4      () {
5      ptrace(PTRACE_TRACEME, 0, nullptr, nullptr);
6  }

```

Figure 8.1: Custom library to trigger the parent-based tracing

The code snippet in Figure 8.1 represents a custom library that calls `ptrace` with the `PTRACE_TRACEME` request. This causes the process to attempt to declare itself as traceable by its parent process, simulating parent-based debugging.

```

1  #include <cstdlib>
2  #include <sys/mman.h>
3
4  extern "C" __attribute__((constructor)) void
5      run_anonymous_exec_mmap_test() {
6      void* mem = ::mmap(nullptr, 4096, PROT_READ | PROT_EXEC,
7      MAP_PRIVATE | MAP_ANONYMOUS, -1, 0);
8  }

```

Figure 8.2: Custom library to trigger creation of the anonymous executable memory.

The code snippet in Figure 8.2 represents a custom library that calls `mmap` to create an anonymous memory mapping with read and execute permissions. This simulates the allocation of executable memory that is not backed by a file.

```

1  #include <cstdlib>
2  #include <sys/mman.h>
3
4  extern "C" __attribute__((constructor)) void run_wx_mprotect_test()
5      {
6      void* mem = ::mmap(nullptr, 4096, PROT_READ, MAP_PRIVATE |
7      MAP_ANONYMOUS, -1, 0);
8      ::mprotect(mem, 4096, PROT_WRITE | PROT_EXEC);
9  }

```

Figure 8.3: Custom library to trigger the change of the anonymous readable memory to be both executable and writable.

The code snippet in Figure 8.3 represents a custom library that first creates anonymous memory mapping with read permission and then attempts to change the mapping permissions to be both writable and executable by using `mprotect`.

8.2 Methodology

8.2.1 Compatibility Test

The compatibility test is designed to evaluate whether OdinSight can protect games with different technical characteristics without preventing normal game execution. While the coverage tests focus on individual security mechanisms, this test examines whether the complete anti-cheat system remains functional when applied to a broader set of real games.

AssaultCube was used as the baseline title because it is open-source, has an existing open-source cheat implementation called `ac_rhack 8.3`, and was previously used in the TyrSecure project. This made it suitable for validating OdinSight against a known and reproducible cheating scenario. However, relying solely on AssaultCube would provide limited evidence of compatibility, since it is a relatively small, older native Linux game. Therefore, the test set was expanded with four additional games, as shown in Table 8.2.

For each game, the compatibility assessment follows the standardized procedure below:

1. Define the root directory of the game.
2. Specify the path to the game executable.
3. Configure the path to the required game libraries.
4. Execute the launch command from the launcher.
5. Monitor the daemon logs to identify potential execution/runtime errors.
6. Analyze `bpf_printk` output to diagnose eBPF-related blocking events.

Together, these games provide a more diverse compatibility test set than the original AssaultCube baseline. The purpose of this test is not to evaluate cheat detection coverage for each individual game, but rather to verify that OdinSight can be applied across different types of Linux games without disrupting normal launch, gameplay, or multiplayer functionality.

8.2.2 Coverage Test

The coverage test is designed to evaluate whether the proposed eBPF-based anti-cheat solution, OdinSight, enforces the protection goals defined in the problem statement 5. In this work, test coverage does not refer to the percentage of source code executed by unit tests. Instead, it describes whether the selected test cases cover the main protection areas of the defined requirements and verify that the expected security restrictions are enforced.

The test cases are organized around three protection areas. The first area concerns the integrity of the host kernel environment, since OdinSight depends on kernel-level trust assumptions before its eBPF-based restrictions can provide meaningful protection. The second area concerns the OdinSight daemon, which must remain available and protected from external interference while the game is running. The third area concerns the protected game process and its surrounding execution environment, where the tests evaluate whether common inspection, modification, and abuse techniques are blocked.

This evaluation considers both allowed and denied behavior. Allowed behavior confirms that valid configurations and unrelated host activity continue to function normally. Denied behavior

confirms that operations violating the anti-cheat policy are blocked by the appropriate eBPF security hooks. This distinction is important because OdinSight must enforce security restrictions without unnecessarily interfering with benign processes on the host system.

Kernel Space

The first group of tests evaluates the protection of the kernel space

Test name	Test description	Test expectations
KI-1	Start OdinSight on a system with Secure Boot, kernel lockdown, and module signature enforcement enabled, where unsigned kernel modules cannot be loaded.	OdinSight is expected to verify the system state as valid and allow the anti-cheat process to continue.
KI-2	Secure Boot disabled: Start OdinSight on a system with Secure Boot disabled.	OdinSight is expected to reject the system state and stop the anti-cheat process.
KI-3	Kernel lockdown disabled: Start OdinSight on a system where Kernel Lockdown is disabled.	OdinSight is expected to reject the system state and stop the anti-cheat process.
KI-4	Module signature enforcement disabled: Start OdinSight on a system where Module Signature Enforcement is disabled.	OdinSight is expected to reject the system state and stop the anti-cheat process.
KI-5	Unsigned kernel module loading: Start OdinSight on a system where module signature enforcement and kernel lockdown are disabled, and an unsigned kernel module can be loaded.	OdinSight is expected to reject the system state and stop the anti-cheat process.

Table 8.4: Kernel Integrity Test Cases

Table 8.4 presents the kernel integrity test cases. These tests verify that OdinSight starts only when the underlying platform meets the required assumptions. Since eBPF programs execute inside the kernel and enforce policy at security-relevant hooks, the kernel itself must be protected from unauthorized modification. Secure Boot, kernel lockdown, and module signature enforcement are therefore treated as prerequisite protections. If any of these mechanisms are disabled, an attacker may be able to modify kernel behavior, load unsigned kernel modules, or bypass the anti-cheat logic entirely. The expected result is that OdinSight starts only when the system meets these integrity requirements.

Anti-cheat Software

The next group of tests evaluates the protection of the OdinSight daemon. The daemon is responsible for maintaining the anti-cheat policy and coordinating protection of the game process. As a result, it is a high-value target for attackers. If an attacker can terminate the daemon, attach a debugger to it, lower its scheduling priority, or modify its resource limits, the anti-cheat system may become ineffective or easier to bypass. These tests, therefore, verify that OdinSight protects its own lifecycle and execution environment.

Test name	Test description	Test expectations
DP-BPF-01	Start OdinSight and inject eBPF programs from an external process.	The request is expected to be rejected by the bpf restriction.
DP-IPC-01	Start OdinSight and trigger a termination signal to the OdinSight daemon using the Linux kill command from an external administrative process.	The signal is expected to be rejected by the task_kill restriction.
DP-RES-01	Lower the CPU scheduling priority of the protected daemon process.	The request is expected to be rejected by the task_setnice restriction.
DP-RES-02	Change the real-time scheduling policy of the protected daemon process.	The request is expected to be rejected by the task_setscheduler restriction.
DP-RES-03	Change the I/O priority of the protected daemon process.	The request is expected to be rejected by the task_setioprio restriction.
DP-RES-04	Change the resource limits of the protected daemon process.	The request is expected to be rejected by the task_setrlimit restriction.

Table 8.5: OdinSight Lifecycle and Resource Test Cases

Table 8.5 presents test cases that cover attempts to interfere with the OdinSight daemon through process control and resource manipulation. The BPF test checks whether an external process can load or inject eBPF programs that may conflict with, observe, or bypass OdinSight. The IPC and resource tests verify that the daemon cannot be terminated or weakened by changing its scheduling parameters, I/O priority, or resource limits. Together, these tests evaluate whether the anti-cheat component remains available and stable while the protected game is running.

Test name	Test description	Test expectations
DP-DBG-01	Start OdinSight and attempt to attach a debugger to the daemon process using GDB from an external process.	The request is expected to be rejected, preventing external processes from inspecting or modifying OdinSight's memory layout.
DP-FS-01	Start OdinSight and attempt to read runtime layout configurations or memory metadata within its procfs structures using <code>cat /proc/DAEMON_PID/maps</code> from an external shell.	The file read query is expected to be rejected with a permission-denied error, thereby cloaking active memory mappings from discovery scanners.
DP-FS-02	Start OdinSight and execute standard system inspection routines on non-protected host processes or on global filesystem paths such as <code>/proc/cpuinfo</code> .	The file access operations are expected to complete successfully, thereby verifying that the protection logic introduces no side effects on unrelated host processes.

Table 8.6: OdinSight Debugging and Filesystem Test Cases

Table 8.6 presents the OdinSight daemon debugging and filesystem test cases to evaluate whether OdinSight can resist inspection of the daemon from external tools. Debuggers and procfs-based inspection are commonly used during reverse engineering and cheat development because they expose memory mappings, process state, and runtime behavior. Blocking access to sensitive daemon metadata reduces the attacker's ability to understand or manipulate the anti-cheat process. At the same time, the inclusion of a non-protected filesystem access test ensures that the policy is scoped correctly and does not unnecessarily restrict unrelated system activity.

Game Assets and Process Lifecycle

The final group of tests evaluates the protection of the game process. By default, these tests will be performed using AssaultCube as the protected game process, since it provides a lightweight and reproducible test target. In some cases, additional games were used where their runtime behavior was relevant to the specific protection mechanism being evaluated. The game process is the primary asset protected by OdinSight. The tests focus on attack techniques commonly used by game cheats, including debugger attachment, memory scanning, runtime code injection, allocation of executable memory, and unauthorized access to process metadata. These techniques are especially relevant because they may allow an attacker to inspect the game state, modify memory, inject cheat logic, or bypass integrity checks.

Test name	Test description	Test expectations
GP-DBG-01	Attach GDB to the game process.	The ptrace request is expected to be rejected by the ptrace_access_check restriction.
GP-DBG-02	Attach Scanmem to the game process.	The procfs memory access request is expected to be rejected by the inode_permission restriction.
GP-DBG-03	Attempt to initiate parent-based tracing inside the game process.	The ptrace_traceme request is expected to be rejected by the ptrace_traceme restriction.
GP-DBG-04	Attempt to access performance monitoring events from outside the daemon or the game.	The perf_event_open request is expected to be rejected by the perf_event_open restriction.
GP-MEM-01	Create a memory with both write and execute permissions inside the protected game cgroup.	The mmap request with PROT_WRITE and PROT_EXEC is expected to be rejected by the mmap_file restriction.
GP-MEM-02	Create an anonymous memory with execute permission inside the protected game cgroup.	The request is expected to be rejected due to the mmap_file restriction.
GP-MEM-03	Change existing memory to have both write and execute permissions inside the protected game cgroup.	The memory permission change request is expected to be rejected by the file_mprotect restriction.
GP-INJ-01	Execute the game process with a preloaded shared library inside the protected game cgroup.	The execve request with LD_PRELOAD in the process environment is expected to be rejected by the bprm_check_security restriction.

Table 8.7: Game process Debugging, Memory and Injection Test Cases

Table 8.7 presents test cases that cover several major categories of runtime attacks. The debugger tests verify that external tools cannot attach to or trace the protected game process. The memory tests verify that the process cannot create or modify memory regions in ways that enable runtime code generation or shellcode execution, particularly through writable and executable memory mappings. The injection test verifies that the game cannot be launched with a preloaded shared library, which is a common method for inserting unauthorized code into a process before normal execution begins.

Test name	Test description	Test expectations
GP-FS-01	Access procfs entries of the protected game process from outside the protected game cgroup.	The access request is expected to be rejected by the <code>inode_permission</code> restriction.
GP-FS-02	Write to the protected cgroupfs directory from outside the protected game cgroup.	The write request is expected to be rejected by the <code>inode_permission</code> restriction.
GP-MNT-01	Move a mount into the protected game cgroup path.	The request is expected to be rejected by the <code>move_mount</code> restriction.
GP-MNT-02	Move a mount out of the protected game cgroup path.	The request is expected to be rejected by the <code>move_mount</code> restriction.
GP-CAP-01	Use a restricted Linux capability inside the protected game cgroup.	The request is expected to be rejected by the capability restriction while allowing capabilities such as <code>CAP_SETUID</code> and <code>CAP_SETGID</code> .

Table 8.8: Game Process Filesystem, Mount and Capability Test Cases

Table 8.8 presents the filesystem, mount, and capability test cases to evaluate whether the protected game environment can be modified or inspected through Linux kernel interfaces outside the normal process execution flow. Procfs and cgroupfs are important because they expose process metadata and control interfaces that can be abused to inspect or influence protected processes. Mount operations are tested because namespace and filesystem manipulation can alter the runtime view of files or introduce unexpected resources into the protected environment. Finally, the capability test verifies that privileged operations are restricted within the protected game cgroup while still allowing capabilities required for normal execution.

8.2.3 Performance Overhead Test

The performance overhead test is designed to evaluate the runtime cost introduced by OdinSight during graphics-intensive workloads. Since OdinSight relies on eBPF hooks to monitor and restrict security-sensitive kernel operations, it is important to determine whether these protections introduce measurable performance degradation during normal gameplay. The goal of this test is therefore to compare system performance with and without OdinSight enabled under both isolated and representative desktop workloads. This evaluation is intended to identify whether OdinSight introduces clearly observable performance degradation. It is not designed to determine the exact performance overhead of the system or to quantify minor performance differences between test scenarios.

To evaluate the performance impact of OdinSight’s eBPF-based monitoring hooks, two distinct types of gaming workloads are utilized to collect performance metrics during system execution:

- **Unigine Superposition:** A synthetic, graphics-heavy benchmark chosen because it provides a highly repeatable, consistent, and identical rendering sequence across multiple test runs.
- **Baldur’s Gate 3:** A real-world gaming workload chosen to complement the synthetic test by evaluating typical game engine behavior, asset streaming, and complex CPU/GPU interactions.

Note on Workload Variance: It should be noted that while Unigine Superposition provides a highly controlled and consistent workload across test runs, the Baldurs Gate 3 test involves non-identical gameplay loops between runs and therefore greater workload variation. Consequently, neither

workload is intended to provide a precise quantification of OdinSight’s runtime overhead. Instead, the purpose of the evaluation is to determine whether OdinSight causes observable performance degradation under controlled, representative gaming conditions. The Unigine workload provides greater sensitivity to performance deviations due to its consistent execution pattern, while the Baldur’s Gate 3 workload demonstrates the same benchmarking methodology in a practical gameplay scenario.

MangoHud is used for data collection and is configured at its default 100 ms sampling interval to collect runtime performance metrics throughout the benchmark. This data includes frame rate, frametime, CPU and GPU utilization, temperatures, clock speeds, power consumption, RAM usage, VRAM usage, and swap usage, providing a complete record of system behavior during the test.

For **Unigine Superposition**, the same benchmark configuration is used across all scenarios, as shown in Figure 8.4. The workload is intentionally configured to be highly GPU-bound through aggressive graphics settings and near-saturated GPU utilization.

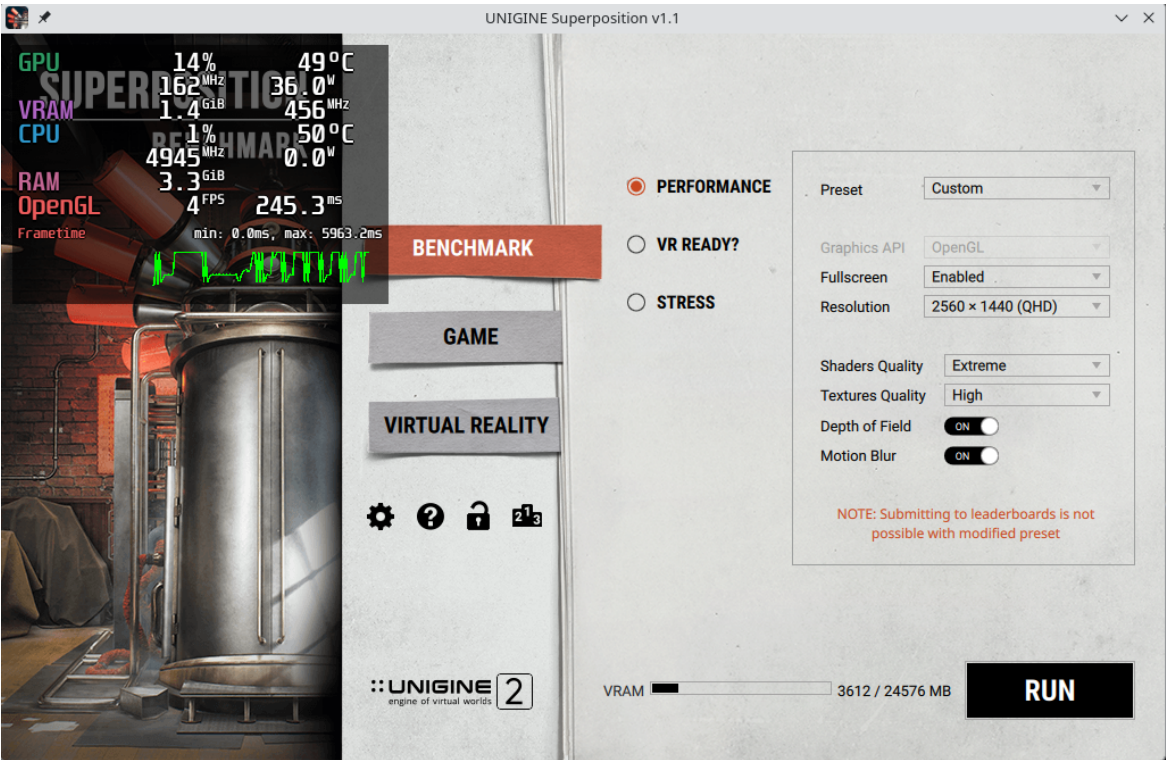


Figure 8.4: Unigine Benchmark Settings

For Baldur’s Gate 3, the same graphical configuration is used across all scenarios, as shown in Figure 8.5. The game is configured with maximum graphics settings in order to create a demanding real-world gameplay workload.



Figure 8.5: Baldur's Gate 3 graphics settings

The tests consist of four scenarios, shown in Table 8.9. The first scenario establishes a baseline measurement by running the workload without OdinSight and without additional background applications. The second scenario repeats the same benchmark with OdinSight attached, allowing the standalone overhead of the anti-cheat system to be measured. The third scenario introduces a representative user workload by enabling Discord screen sharing and YouTube playback in a web browser while OdinSight is disabled. This scenario is included to measure the effect of common background activity on benchmark performance. Finally, the fourth scenario combines the same background workload with OdinSight protection enabled, allowing the impact of background applications and the anti-cheat system to be evaluated together.

Test name	Test scenario
PERF-01	The selected workload is used as the benchmark with MangoHud enabled. No additional background applications are active, and OdinSight is not attached. This scenario is designed to establish the baseline performance measurement.
PERF-02	The selected workload is used as the benchmark with MangoHud enabled. No additional background applications are active, and OdinSight is attached. This scenario is designed to measure the standalone performance overhead introduced by OdinSight.
PERF-03	The selected workload is used as the benchmark with MangoHud enabled while Discord screen sharing and a web browser playing YouTube are active, and OdinSight is not attached. This scenario is designed to measure performance under a representative background workload.
PERF-04	The selected workload is used as the benchmark with MangoHud enabled while Discord screen sharing and a web browser playing YouTube are active, and OdinSight is attached. This scenario is designed to measure the combined overhead of background activity and OdinSight protection.

Table 8.9: Performance Overhead Test Scenarios

For each scenario, both workloads are tested under the same hardware and software configuration described in Table 8.1. The benchmark settings are kept constant across all runs to ensure comparability. To warm up the hardware, an initial workload run is performed before the four test scenarios begin. Between tests, a cooldown period of approximately 5 minutes is used to allow the system to stabilize before the next run begins. The scenarios are executed in consecutive order from PERF-01 to PERF-04. The primary comparison is made between the scenarios without OdinSight and their corresponding scenarios with OdinSight enabled. In particular, PERF-01 is compared against PERF-02 to estimate OdinSight’s standalone overhead, while PERF-03 is compared against PERF-04 to estimate its overhead under a more realistic desktop workload.

The Unigine Superposition workload uses its built-in benchmark sequence. Each run is started from the benchmark menu. After the benchmark is initiated, it runs until the measurement period is complete without any further user input. Since Unigine Superposition follows a fixed rendering sequence, each run presents the same scenes and camera positions. This makes the benchmark highly repeatable and allows performance differences between scenarios to be attributed primarily to OdinSight and the selected background workload conditions.

The Baldur’s Gate 3 workload runs in co-op mode over a peer-to-peer connection between two clients, with two real players controlling the characters to simulate a multiplayer environment. To ensure test reliability, each run starts from the same saved game at the introduction. From this save point, the same gameplay route is followed until the measurement period is complete. The route and player actions are kept as consistent as possible across all scenarios to minimize variation due to differences in scene complexity, asset loading, or gameplay events. This allows the Baldur’s Gate 3 measurements to approximate a realistic gameplay workload while still maintaining sufficient repeatability for comparison.

These tests do not aim to evaluate cheat detection functionality. Instead, they focus on whether OdinSight can provide protection while maintaining acceptable runtime performance. A low difference between the OdinSight-enabled and OdinSight-disabled scenarios would indicate that the eBPF-based monitoring approach introduces limited overhead during high-load graphical workloads.

8.3 Test Results

8.3.1 Compatibility

This section presents the compatibility results for the selected game titles when executed under the OdinSight daemon. The purpose of this test is to evaluate whether OdinSight can launch various Linux games without preventing normal execution, gameplay, or multiplayer functionality. The detailed launch configurations, runtime logs, eBPF trace outputs, and screenshots for each game are provided in Appendix A.

Game	Result	Description
AssaultCube	Pass	The game launched successfully under OdinSight, with no runtime errors or eBPF-related denials. Multiplayer functionality was verified through a public server session.
SuperTuxKart	Pass	The game launched successfully under OdinSight, with no runtime errors or eBPF-related denials. Multiplayer functionality was verified with two connected clients.
A Story About My Uncle	Pass	The game reached a playable state despite SteamAPI warnings and a denied CAP_SYS_ADMIN request. The denied capability did not prevent normal gameplay.
Baldur's Gate 3	Partial Pass	Initial Steam runtime execution triggered LD_PRELOAD and namespace-related restrictions. When the Steam client dependency was removed from the protected launch path, the game launched successfully, and multiplayer functionality was verified.
Valheim	Fail	The game failed to launch because OdinSight denied writable-executable memory mappings requested during startup. This may be related to Unity engine behavior or bundled native plugins.
Unturned	Fail	The game failed to launch with the same writable-executable memory-mapping denials observed in Valheim, indicating a likely compatibility issue with Unity-based runtime behavior under the current OdinSight memory policy.

Table 8.10: Compatibility test results for games executed under OdinSight protection

Table 8.10 shows that OdinSight was fully compatible with three of the six tested games, while additionally Baldur's Gate 3 was considered as partially compatible.

AssaultCube and SuperTuxKart launched without runtime errors and eBPF-related rejections, and both were functioning as intended in multiplayer sessions. A Story About My Uncle also launched successfully despite SteamAPI-related warnings and a denied CAP_SYS_ADMIN request. Baldur's Gate 3 initially produced Steam runtime errors related to LD_PRELOAD and namespace creation, but the game still ran successfully when launched without the Steam client interfering with the protected execution environment.

The main compatibility issues were observed in cases where the game or its runtime environment required behavior that conflicts with OdinSight's enforced security. Steam-related initialization can rely on mechanisms such as LD_PRELOAD and namespace creation, both of which are treated as sensitive because they may also be used for library injection, process manipulation, or sandbox escape

techniques. Similarly, the Unity-based titles Valheim and Unturned failed to launch because they attempted to create writable-executable memory mappings. OdinSight blocks this behavior because memory that is both writable and executable can enable runtime code modification, just-in-time code generation, or code injection. However, these memory mapping requests do not necessarily prove that writable-executable memory is required for correct execution. They may instead result from overly permissive implementation choices in the game engine or from a developer's choice to request such permissions even when they are not actively used. Therefore, the failures observed for Valheim and Unturned may reflect a limitation of OdinSight's current enforcement, since it rejects writable-executable mappings based on the requested permissions rather than whether those mappings are actually used for runtime code generation or modification.

These results suggest that OdinSight is well-suited to games that follow a more conventional execution model and do not require privileged capabilities, runtime library injection, or writable-executable memory regions. However, the results also demonstrate a clear trade-off between strict security enforcement and compatibility with complex game runtimes. Some legitimate engines, launchers, or middleware components may rely on behavior that resembles techniques commonly used by cheats or malware, leading to false positives or blocked launches.

8.3.2 Coverage

This section evaluates the security coverage provided by OdinSight across the main attack surfaces considered in this work. The evaluation is organized into three areas: kernel-space integrity, anti-cheat daemon protection, and game process protection. The kernel-space tests verify that OdinSight only operates when the required kernel security features are enabled. The anti-cheat software tests evaluate whether the OdinSight daemon itself is protected against external interference. Finally, the game asset and process lifecycle tests examine the protections applied to the protected game process. The detailed command inputs, outputs, error messages, and `bpf_printk` traces for the coverage tests are provided in Appendix B.

For some coverage tests, the `LD_PRELOAD` protection was intentionally disabled. This was done to allow custom test libraries to be injected into the protected game process, enabling specific runtime behaviors to be evaluated. The output of the results in appendix A

Kernel Space

Table 8.11 shows that OdinSight correctly enforced its kernel integrity requirements. The valid platform configuration was accepted, while each invalid configuration was rejected, including disabled Secure Boot, disabled Kernel Lockdown, disabled module signature enforcement, and a system state where an unsigned kernel module could be loaded. These results confirm that OdinSight requires the underlying kernel environment to satisfy the required trust assumptions.

Test name	Result	Description
KI-01	Pass	OdinSight accepted the system state when Secure Boot, Kernel Lockdown, and kernel module signature enforcement were enabled. Loading an unsigned kernel module was denied by the kernel's signature policy.
KI-02	Pass	OdinSight detected that Secure Boot was disabled and treated the platform state as invalid.
KI-03	Pass	OdinSight detected that Kernel Lockdown was disabled and treated the platform state as invalid.
KI-04	Pass	OdinSight detected that Kernel Module Signature Enforcement was disabled and treated the platform state as invalid.
KI-05	Pass	The Unsigned Kernel Module was successfully loaded in the test environment and treated the platform state as invalid.

Table 8.11: Kernel Space Test Results

Anti-cheat Software

Table 8.12 shows that OdinSight protected the daemon against external interference. Unauthorized attempts to load eBPF programs, terminate the daemon, modify scheduling or resource parameters, attach a debugger, and inspect daemon-specific procfs entries were all denied. At the same time, access to unrelated system-wide procfs information was still allowed, indicating that the procfs restriction was scoped to protected targets rather than applied system-wide.

Test name	Result	Description
DP-BPF-01	Pass	An attempt to load an unauthorized eBPF program from an external process was denied.
DP-IPC-01	Pass	An attempt to send a termination signal to the OdinSight daemon was denied.
DP-RES-01	Pass	An attempt to modify the daemon's CPU priority using <code>renice</code> was denied.
DP-RES-02	Pass	An attempt to modify the daemon's scheduling policy using <code>chrt</code> was denied.
DP-RES-03	Pass	An attempt to modify the daemon's I/O priority using <code>ionice</code> was denied.
DP-RES-04	Pass	An attempt to modify the daemon's file descriptor resource limit using <code>prlimit</code> was denied.
DP-DBG-01	Pass	An attempt to attach <code>gdb</code> to the OdinSight daemon using <code>ptrace</code> was denied.
DP-FS-01	Pass	An attempt to access the daemon's process-specific procfs entry was denied.
DP-FS-02	Pass	An attempt to access unrelated system-wide procfs information using <code>/proc/cpuinfo</code> remained allowed.

Table 8.12: Anti-cheat Software Test Results

Game Assets and Process Lifecycle

Table 8.13 shows that OdinSight enforced the expected restrictions around protected game processes. Debugging and inspection attempts using `gdb`, `scanmem`, parent-based tracing, and performance monitoring were denied. Memory manipulation attempts, including writable-executable mappings, anonymous executable memory, and permission changes to writable-executable memory, were also

blocked. In addition, OdinSight denied LD_PRELOAD-based library injection, protected game-specific procfs entries, restricted cgroup filesystem modification, prevented mount relocation involving the protected cgroup path, and blocked restricted capability use inside the protected game cgroup.

Test name	Result	Description
GP-DBG-01	Pass	An attempt to attach GDB was denied.
GP-DBG-02	Pass	An attempt to attach Scanmem was denied.
GP-DBG-03	Pass	An attempt to initiate parent-based tracing from inside the game process was denied.
GP-DBG-04	Pass	An attempt to access performance monitoring events was denied.
GP-MEM-01	Pass	An attempt to create a memory mapping with both write and execute permissions was denied.
GP-MEM-02	Pass	An attempt to create anonymous executable memory was denied.
GP-MEM-03	Pass	An attempt to change existing memory permissions to writable and executable was denied.
GP-INJ-01	Pass	An attempt to inject a shared library through LD_PRELOAD was denied.
GP-FS-01	Pass	An attempt to access the game process-specific procfs entry was denied.
GP-FS-02	Pass	An attempt to create a file inside the game cgroup directory was denied.
GP-MNT-01	Pass	An attempt to move an external mount point into the game cgroup path was denied.
GP-MNT-02	Pass	An attempt to move the protected game cgroup path out to another mount location was denied.
GP-CAP-01	Pass	An attempt to use restricted capability inside the game cgroup was denied.

Table 8.13: Game Assets and Process Lifecycle Test Results

The coverage results show that OdinSight enforces protection across the main attack surfaces evaluated in this work. At the kernel level, OdinSight validates that required platform protections are enabled before continuing execution, including Secure Boot, Kernel Lockdown, and kernel module signature enforcement. This reduces the risk of running the anti-cheat in an environment where unsigned kernel modules or weakened kernel-integrity settings could undermine its trust assumptions.

For the anti-cheat daemon itself, the results show that OdinSight can prevent external processes from interfering with its execution. Unauthorized attempts to load additional eBPF programs, send termination signals, modify scheduling or resource limits, attach a debugger, or access daemon-specific procfs entries were all denied. These protections help preserve the availability and integrity of the OdinSight daemon during runtime.

For the protected game process, OdinSight blocked several common inspection, injection, and tampering scenarios. The tests show that debugger attachment, procfs-based memory inspection, parent-based tracing through PTRACE_TRACEME, performance-counter monitoring, LD_PRELOAD-based library injection, suspicious memory mappings, cgroup filesystem modification, mount relocation, and privileged capability use were all restricted. In particular, OdinSight addresses the LD_PRELOAD injection gap by detecting and rejecting the environment variable before the protected game process is executed. This prevents cheat libraries, such as ac_rhack, from being loaded into the game process, where they could enable cheats such as aimbots, wallhacks, or other runtime manipulation techniques. Overall, these results demonstrate that OdinSight covers multiple stages of a potential attack, including pre-execution injection, runtime process inspection, memory permission abuse, and filesystem or cgroup manipulation.

However, the coverage results should not be interpreted as complete protection against all possible cheats or attack techniques. The tests focus on a selected set of Linux attack surfaces and

manually constructed scenarios. Therefore, the results demonstrate practical coverage of the tested mechanisms, but they do not prove that OdinSight can block every possible bypass or cheat implementation.

8.3.3 Performance Overhead

This section presents the results of the performance overhead evaluation described in the methodology. Following established principles for computer systems performance analysis, the evaluation compares equivalent workload conditions with OdinSight enabled and disabled to isolate the effect of the system under test [15]. For consistency, each scenario was evaluated over a 150-second measurement period. During this period, performance data were recorded using MangoHud at 100 ms intervals, resulting in 1440 samples, each representing a recorded measurement point. Two workloads are tested: Unigine Superposition as a controlled synthetic benchmark and Baldur's Gate 3 as a representative real-game workload.

Unigine Benchmark

The Unigine Superposition benchmark serves as the controlled **synthetic workload** to evaluate OdinSight's performance overhead. As defined in Chapter 8.2.3, the analysis considers both the baseline condition and the representative workload condition with background applications active.

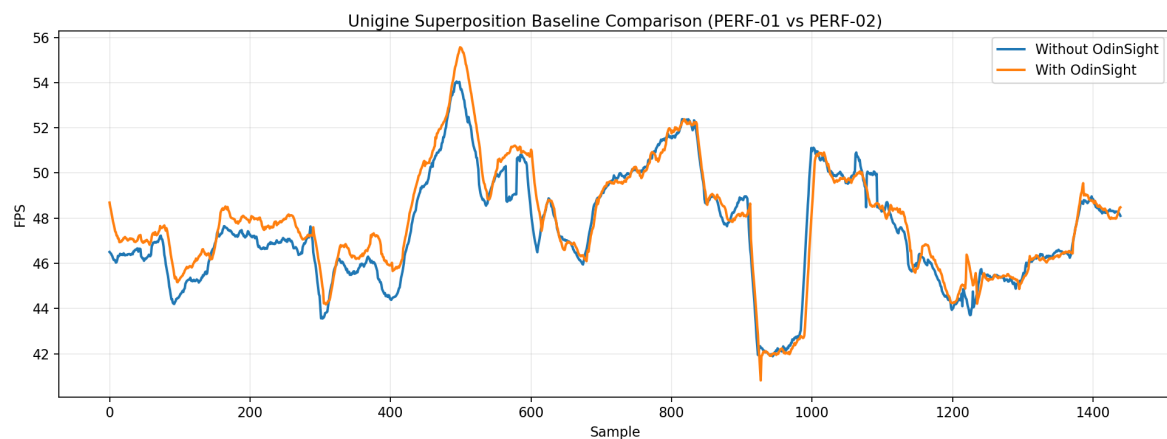


Figure 8.6: Baseline Comparison of Unigine Superposition

Figure 8.6 presents the FPS over time for the Unigine Superposition baseline benchmark. The y-axis shows FPS, and the x-axis shows the collected samples, where each sample represents a data point recorded at 100 ms intervals. The comparison includes one run with OdinSight enabled and one without, both performed with no background applications running.

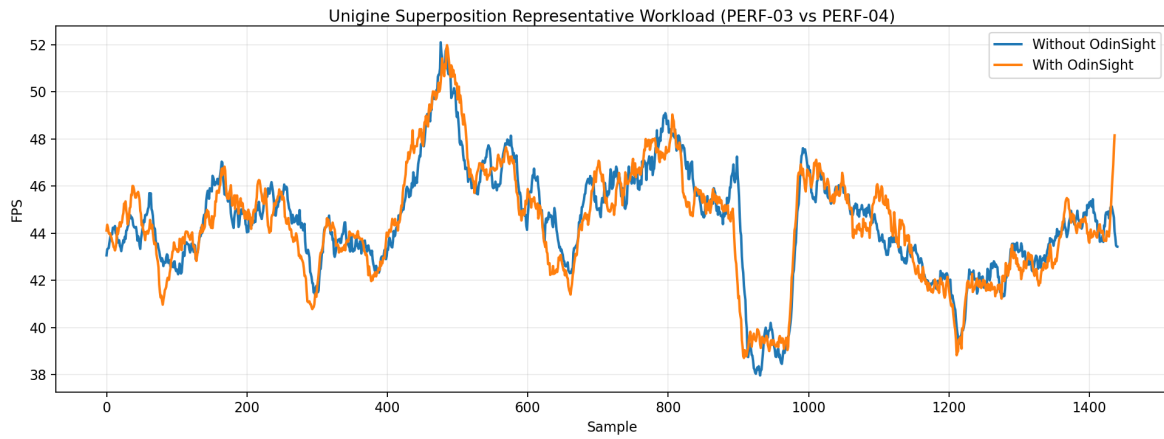


Figure 8.7: Representative Workload Comparison

Figure 8.7 presents the FPS over time for the Unigine Superposition under the representative workload condition, where background applications were active during the benchmark. The y-axis shows FPS, and the x-axis shows the collected samples, where each sample represents a data point recorded at 100 ms intervals. The comparison includes a run with OdinSight enabled and one without, both performed while background applications were active.

Test	OdinSight	Avg FPS	1% Low FPS	Avg Frametime (ms)
PERF-01	No	47.46	41.59	21.12
PERF-02	Yes	47.82	41.56	20.99
PERF-03	No	44.50	36.71	22.61
PERF-04	Yes	44.40	36.77	22.67

Table 8.14: Unigine Superposition Performance Metrics Summary

Table 8.14 summarizes the Unigine Superposition performance results for the four test scenarios as defined in Methodology Chapter 8.2.3. The table reports average FPS, 1% low FPS, and average frametime.

Test	CPU Load(%)	GPU Load(%)	CPU Temp (°C)	GPU Temp (°C)	RAM (GB)	VRAM (GB)	GPU Power (W)
PERF-01	3.68	98.99	51.60	70.16	3.24	4.65	271.00
PERF-02	3.63	98.98	57.77	72.57	3.46	4.73	270.96
PERF-03	13.82	98.99	65.91	71.72	5.65	5.11	270.99
PERF-04	13.81	98.99	66.10	71.92	5.54	5.12	270.99

Table 8.15: Unigine Superposition Resource Utilization Summary

Table 8.15 summarizes CPU, GPU, and RAM-related measurements for each test scenario.

Performance Difference Analysis

As shown in Table 8.14, the PERF-01 vs PERF-02 comparison isolates the standalone OdinSight overhead. Average FPS changes from 47.46 FPS to 47.82 FPS (+0.75% due to run-to-run variance), while 1% low FPS remains effectively unchanged. Frametime metrics show negligible variation, indicating no clearly observable rendering degradation attributable to OdinSight.

The PERF-03 vs PERF-04 comparison evaluates OdinSight under representative workload condition.

Average FPS changes from 44.50 FPS to 44.40 FPS (-0.22%), indicating no clearly observable performance degradation attributable to OdinSight.

Cross-scenario comparison shows that the representative background workload has a substantially greater performance impact than OdinSight itself.

System Overhead Interpretation

As shown in table 8.15. Under isolated benchmark conditions (PERF-01 vs PERF-02), OdinSight does not introduce clearly observable rendering degradation, although differences are observed in selected system-level metrics. CPU temperature increased from 51.60 °C to 57.77 °C (+6.17 °C), GPU temperature increased from 70.16 °C to 72.57 °C (+2.41 °C), RAM usage increased from 3.24 GB to 3.46 GB (+0.22 GB), and VRAM usage increased from 4.65 GB to 4.73 GB (+0.08 GB). CPU and GPU utilization remain effectively unchanged, indicating that OdinSight does not introduce an observable increase in computational load sufficient to affect rendering throughput.

Under representative workload conditions (PERF-03 vs PERF-04), the differences become substantially smaller. CPU temperature changes by only +0.19 °C, GPU temperature by +0.20 °C, while RAM usage decreases slightly by 0.11 GB. This suggests that ordinary concurrent workload dominates total system utilization, making any additional OdinSight-related impact difficult to distinguish from normal workload variation under loaded conditions.

Baldur's Gate 3 Benchmark

The Unigine Baldur's Gate 3 is used as the real-game workload for evaluating OdinSight's performance overhead. As defined in Chapter 8.2.3, the analysis considers both the baseline condition and the representative workload condition with background applications active.

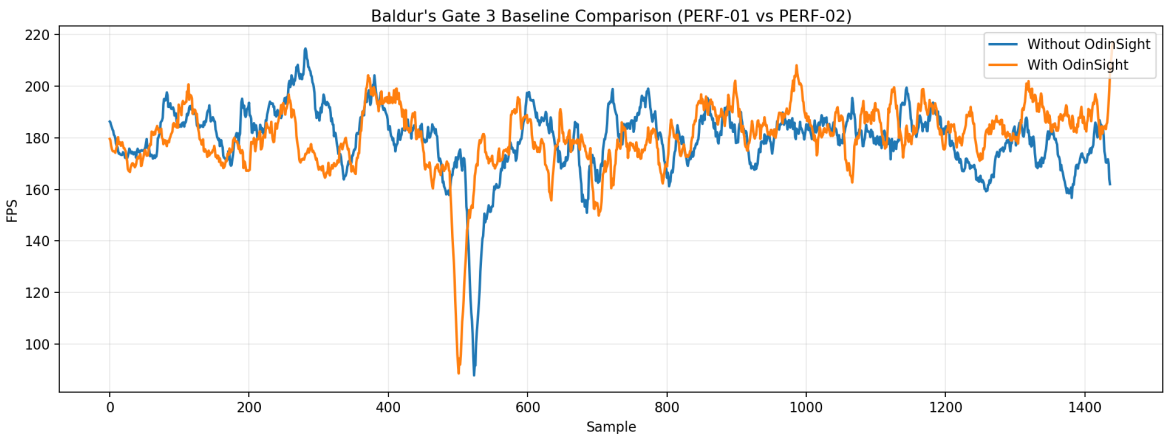


Figure 8.8: Baseline Comparison

Figure 8.8 presents the FPS over time for the Baldur's Gate 3 baseline benchmark. The y-axis shows FPS, and the x-axis shows the collected samples, where each sample represents a data point recorded at 100 ms intervals. The comparison includes a run with OdinSight enabled and one without, both performed with no background applications running.

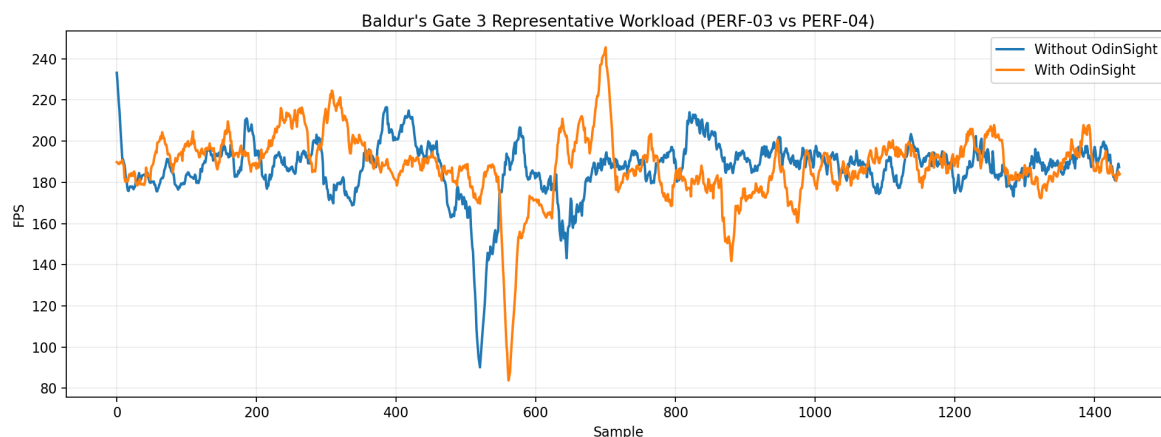


Figure 8.9: Representative Workload Comparison

Figure 8.9 presents the FPS over time for Baldur’s Gate 3 under the representative workload condition, where background applications were active during the benchmark. The y-axis shows FPS, and the x-axis shows the collected samples, where each sample represents a data point recorded at 100 ms intervals. The comparison includes a run with OdinSight enabled and one without, both performed while background applications were active.

Test	OdinSight	Avg FPS	1% Low FPS	Avg Frametime (ms)
PERF-01	No	179.54	92.51	5.71
PERF-02	Yes	180.23	97.70	5.69
PERF-03	No	186.83	80.64	5.53
PERF-04	Yes	188.09	82.52	5.50

Table 8.16: Baldur’s Gate 3 Performance Metrics Summary

Table 8.16 summarizes the Baldur’s Gate 3 performance results for the four test scenarios as defined in Methodology Chapter 8.2.3. The table reports average FPS, 1% low FPS, and average frametime.

Test	CPU Load(%)	GPU Load(%)	CPU Temp (°C)	GPU Temp (°C)	RAM (GB)	VRAM (GB)	GPU Power (W)
PERF-01	36.22	97.87	78.20	78.90	6.07	5.39	269.05
PERF-02	36.69	97.99	78.89	77.30	6.15	5.48	269.20
PERF-03	39.27	97.37	77.06	76.55	8.38	5.74	268.56
PERF-04	38.35	97.17	75.90	76.14	8.46	5.81	268.17

Table 8.17: Baldur’s Gate 3 Resource Utilization Summary

Table 8.17 summarizes CPU, GPU, and RAM-related measurements for each test scenario.

Performance Difference Analysis

The observed patterns in both the baseline and representative workload comparisons, shown in Figures 8.8 and 8.9, are affected by the variations in human gameplay described in the methodology section 8.2.3. The FPS drop between samples 400 and 600 is due to rendering a resource-intensive cutscene.

Based on the performance metrics in Table 8.16, the PERF-01 vs PERF-02 comparison isolates standalone OdinSight overhead. Average FPS changes from 179.54 FPS to 180.23 FPS (+0.38%),

while 1% low FPS increases from 92.51 FPS to 97.70 FPS (+5.61%). These differences indicate that OdinSight does not introduce any clearly observable rendering degradation under isolated benchmark conditions.

The PERF-03 vs PERF-04 comparison evaluates OdinSight under the representative workload condition with background applications active. Average FPS changes from 186.83 FPS to 188.09 FPS (+0.67%), while 1% low FPS increases from 80.64 FPS to 82.52 FPS (+2.33%), also indicating no clearly observable performance degradation attributable to OdinSight.

Cross-scenario comparison based on the resource utilization measurements in Table 8.17 shows that the representative background workload affects system resource usage more clearly than OdinSight itself. In particular, the RAM usage increases substantially between the isolated and representative workload scenarios, while rendering performance remains broadly stable. The CPU load also increases during the representative workload scenario. This suggests that the additional workload increases overall system demand, while no clearly observable performance degradation attributable to OdinSight is evident in the collected measurements.

System Overhead Interpretation

Under isolated benchmark conditions (PERF-01 vs PERF-02), OdinSight does not introduce clearly observable rendering degradation. CPU load increases slightly from 36.22% to 36.69% (+0.47%), while GPU load remains effectively unchanged. CPU temperature increases from 78.20 °C to 78.89 °C (+0.69 °C), whereas GPU temperature decreases from 78.90 °C to 77.30 °C. RAM usage increases from 6.07 GB to 6.15 GB (+0.08 GB), and VRAM usage increases from 5.39 GB to 5.48 GB (+0.09 GB). GPU power changes only marginally, from 269.05 W to 269.20 W.

Under representative workload conditions (PERF-03 vs PERF-04), the observed differences remain small. CPU load decreases from 39.27% to 38.35%, while GPU load decreases slightly from 97.37% to 97.17%. CPU temperature decreases from 77.06 °C to 75.90 °C, and GPU temperature decreases from 76.55 °C to 76.14 °C. RAM usage increases slightly from 8.38 GB to 8.46 GB (+0.08 GB), and VRAM usage increases from 5.74 GB to 5.81 GB (+0.07 GB). GPU power decreases slightly from 268.56 W to 268.17 W.

Comparison of Benchmark Results

Comparing Unigine Superposition and Baldur's Gate 3 shows a consistent overall pattern: OdinSight does not introduce any clearly observable rendering performance degradation in either benchmark, while the representative background workload has a more noticeable effect on system resource usage and frame consistency.

Unigine Superposition provides a more controlled and repeatable benchmark environment. Since the workload follows a fixed rendering sequence, comparisons between test runs are more stable. In this benchmark, the difference between OdinSight-enabled and disabled remains very small in both the isolated and representative workload scenarios. This suggests that OdinSight does not introduce any clearly observable performance degradation during a consistent GPU-intensive workload.

Baldur's Gate 3 shows greater variation between runs. This is expected because real game benchmarks are harder to reproduce exactly. Small differences in camera movement, scene timing, world state, background simulation, and player actions can affect the measured FPS and 1% low FPS values, even when the same test procedure is followed. As a result, small positive or negative differences between Baldur's Gate 3 runs should be interpreted cautiously. Consequently, this real-game benchmark serves primarily as a practical demonstration of the evaluation methodology rather

than a definitive performance baseline.

Across both benchmarks, the representative background workload has a clearer impact than OdinSight itself. This is most visible in the increased RAM usage and CPU load when background applications are active. In Baldur's Gate 3, the lower 1% of FPS values under the representative workload suggest that background applications affect frame consistency more than OdinSight does. However, average FPS remains broadly stable, and the differences between OdinSight-enabled and OdinSight-disabled runs remain small.

Overall, both Unigine Superposition and Baldur's Gate 3 indicate that no clearly observable rendering degradation attributable to OdinSight was identified under the tested conditions. The larger observed differences are primarily attributable to the additional background workload and, in the case of Baldur's Gate 3, to natural run-to-run variability in a real-world game environment. However, these results should be interpreted within the limitations of the evaluation, since they are based on a limited number of workloads, one hardware and software configuration, and a small number of repeated measurements, as discussed further in Section 9.1.4.

Chapter 9

Conclusions

This chapter concludes the project by discussing the results, limitations, and future work of OdinSight. It presents the main findings from the evaluation, identifies the remaining challenges, and summarizes how the project answers the problem statement.

9.1 Discussion

9.1.1 Lifecycle coverage

OdinSight's **lifecycle-oriented** design is one of the project's main strengths. By preparing protection before the protected game process starts, OdinSight reduces the risk that enforcement only becomes active after the process is already in an untrusted state. This is important because attacks such as library injection or early process manipulation can occur during the transition from launch to runtime. The use of a protected cgroup also gives OdinSight a more stable protection boundary than tracking only a single process identifier, since the policy can apply to the game process and child processes that remain inside the same protected context.

Another strength is that OdinSight includes protection for the anti-cheat daemon itself. This is important because the reliability of the protected game environment depends on the component that manages and enforces the policy. If the daemon could be terminated, inspected, or manipulated without resistance, the game-facing protections would become less reliable. By protecting both the game process and the daemon, OdinSight provides a more coherent lifecycle model.

However, OdinSight's lifecycle coverage still has clear **limitations**. The system depends on local trust assumptions, since the kernel validation step can verify that expected security settings are enabled but cannot prove that the running kernel is uncompromised. If an attacker already controls the kernel or can falsify the information exposed to the daemon, OdinSight's enforcement decisions may no longer be reliable. This means that OdinSight improves the client-side protection boundary, but does not solve the deeper trust problem of proving that the local machine is trustworthy.

Pre-execution protection is also incomplete. OdinSight can prepare enforcement before the game starts, but it does not provide full integrity verification of game binaries, assets, runtime libraries, or the anti-cheat binary itself. This leaves a gap between preparing a protected execution context and proving that the files entering that context are authentic.

Overall, OdinSight demonstrates that **lifecycle-aware protection** is a useful direction for Linux client-side anti-cheat. Its main strengths are early enforcement preparation, cgroup-based protection,

and daemon protection. Its main weaknesses are the remaining trust assumptions, the lack of file integrity verification, and the risk that strict policies reduce compatibility. Therefore, OdinSight should be understood as a stronger client-side lifecycle foundation, but not as a complete lifecycle trust solution.

9.1.2 Security and Compatibility Trade-Off

The evaluation also shows a trade-off between strict security enforcement and compatibility. OdinSight blocks behavior commonly associated with cheats or process manipulation, such as LD_PRELOAD-based injection, writable-executable memory mappings, namespace-related capability use, and cgroup or mount manipulation. From a security perspective, blocking these operations is desirable because they can be used to inject code, alter the runtime environment, or inspect protected processes.

However, some legitimate game runtimes and launchers may rely on similar mechanisms during normal execution. For example, Steam-related initialization may use LD_PRELOAD or namespace creation, while some Unity-based games attempted to create writable-executable memory mappings during startup. OdinSight correctly blocked these operations in line with its policy, but this reduced compatibility with some games. This shows that operations that are suspicious from an anti-cheat perspective are not always malicious in practice.

This trade-off is important for practical anti-cheat deployment. A strict policy reduces the attack surface but increases the risk of false positives and blocked launches. A more permissive policy may improve compatibility but could introduce attack surfaces. OdinSight therefore demonstrates the need for policy refinement, in which enforcement can distinguish between legitimate runtime behavior and behavior that poses an unacceptable security risk. This may require game-specific configuration, allowlists, or more context-aware enforcement to reduce compatibility problems without weakening the overall protection model.

9.1.3 Implications for eBPF-Based Anti-cheat

The evaluation suggests that eBPF LSM hooks are a promising foundation for Linux anti-cheat enforcement. Regarding the daemon and game process protection requirements defined in Section 5.1, OdinSight shows that eBPF can cover most of the required protection mechanisms while avoiding the need for a traditional custom kernel module.

However, the value of eBPF depends on the policies and architecture built around it. It provides the kernel-level hooks needed to observe and restrict suspicious behavior, but the surrounding anti-cheat system must decide which actions to allow or deny. The effectiveness of an eBPF-based anti-cheat, therefore, depends on how accurately its policies distinguish malicious behavior from legitimate behavior originating from game engines, launchers, or runtime dependencies. While stricter policies can reduce the attack surface, they also increase the risk of blocking normal game behavior.

A further limitation is that eBPF depends on the integrity of the kernel. If the kernel is compromised, the anti-cheat can no longer trust kernel-reported state, hook execution, or enforcement decisions. eBPF also does not solve pre-execution integrity, game asset verification, or trust in the anti-cheat binary itself. Therefore, eBPF should be understood as a strong local enforcement layer, but not as a complete anti-cheat solution without integrity verification, server-side validation, and game-specific policy refinement.

9.1.4 Limitations of evaluation

Compatibility

The compatibility evaluation provides an initial indication of how OdinSight behaves across different Linux games. The test set only includes a limited number of manually configured native Linux binaries. These include standalone games installed outside Steam, such as AssaultCube and SuperTuxKart, as well as Steam-installed games launched directly through their game executables. This provides more evidence than testing only AssaultCube, but the sample is still too small to generalize to the wider Linux gaming ecosystem.

The selected games may also introduce selection bias. The titles were chosen because they were available, configurable in the test environment, and suitable for manual testing. As a result, the evaluation may not represent games with different launchers, Proton or Wine-based execution, anti-cheat integrations, runtime dependencies, or engine-specific behaviors. For example, the compatibility results showed that some Unity-based games triggered writable-executable memory mappings, while Steam-related initialization introduced issues related to LD_PRELOAD and namespace creation. These cases show that game runtimes can behave differently, and a larger sample would be needed to determine whether such conflicts are isolated cases or common compatibility issues.

Another limitation is that all compatibility tests were performed on a single hardware-and-software configuration. The evaluation was conducted on one Arch Linux system with a specific kernel version, desktop environment, CPU, GPU, driver configuration, and library setup. Compatibility may differ on other user systems. Therefore, the results demonstrate compatibility only for the tested environment and cannot be assumed to hold across all user systems.

The compatibility test was also primarily based on whether each game could be launched and whether basic gameplay or multiplayer functionality appeared to work. This provides useful practical evidence, but it does not cover long play sessions, updates, all gameplay situations, or interactions with external services. Some compatibility problems may only appear after longer runtime, during specific gameplay events, when loading certain assets, or when using online platform features.

Coverage

The coverage evaluation shows that OdinSight enforces the implemented policies for the selected test cases, but it does not evaluate all possible cheat techniques or bypass strategies. The tests focus on selected Linux attack surfaces relevant to OdinSight's design and useful for validating the implemented eBPF LSM hooks. Therefore, the results should be interpreted as evidence that OdinSight enforces its intended policy for the evaluated mechanisms, rather than as proof of complete anti-cheat coverage.

For kernel space protection, the evaluation provides evidence that OdinSight checks whether Secure Boot, Kernel Lockdown, and kernel module signature enforcement are enabled, and whether unsigned kernel modules can be loaded. These checks confirm that the host environment satisfies the expected configuration assumptions before OdinSight continues execution. However, these tests only validate whether the expected configuration is present and do not prove that the kernel is uncompromised. Therefore, the kernel protection tests confirm that OdinSight rejects invalid configurations, but they do not provide complete assurance of kernel integrity.

For daemon protection, the evaluation shows that OdinSight can withstand selected external interference attempts, including termination signals, debugger attachment, procfs inspection, unauthorized eBPF loading, and resource manipulation. However, the evaluation does not include

event flooding, repeated policy violations, or attempts to exhaust daemon resources. As a result, the tests provide evidence that the daemon is protected against the evaluated interference techniques, but they do not fully evaluate how robust the daemon remains under sustained adversarial pressure.

For game process protection, the evaluation provides evidence that OdinSight blocks relevant execution and post-execution attacks against the protected game. However, these tests only cover the selected attack surfaces implemented in the evaluation. They do not cover every possible cheating technique, such as input automation, graphics overlay abuse, network-level manipulation, or attacks that operate outside the tested Linux security interfaces. Therefore, the results show that OdinSight enforces its intended policy for the evaluated mechanisms, but they should not be interpreted as complete protection against all possible cheats.

Another limitation is that the coverage evaluation focuses mainly on whether selected actions are allowed or denied. This confirms that specific operations are blocked, but it does not systematically measure policy precision. In particular, the evaluation does not systematically classify policy outcomes across a broad set of legitimate and malicious behaviors, making it difficult to quantify false positives, false negatives, true positives, and true negatives. The anti-cheat enforcement must block malicious behavior without unnecessarily disrupting the legitimate functionality of the game, launcher, engine, or runtime. The compatibility results already show that some legitimate behavior, such as Steam runtime initialization and Unity memory mappings, can conflict with OdinSight's current policy. A broader evaluation would therefore be needed to determine how accurately OdinSight distinguishes malicious behavior from legitimate runtime behavior.

The evaluation primarily validates OdinSight against controlled, reproducible test cases. This makes the results easier to verify, but it does not fully reflect how real attackers may behave. Therefore, the evaluation should be understood as a validation of the implemented protections, rather than a complete evaluation of real-world cheating techniques.

Performance Overhead

The performance overhead evaluation indicates that OdinSight does not introduce any clearly observable reduction in rendering performance in the tested scenarios. Across both the controlled Unigine Superposition benchmark and the Baldur's Gate 3 gameplay workload, the differences between OdinSight-enabled and OdinSight-disabled runs are small and remain within a range that could plausibly be affected by normal runtime variation. This suggests that OdinSight's eBPF-based monitoring approach has a limited observable impact on frame rate and frametime under the evaluated conditions.

However, the benchmark results should be interpreted with several limitations in mind. First, the evaluation was conducted using a single hardware-and-software configuration. As a result, the findings may not generalize to systems with different CPUs, GPUs, memory capacities, driver versions, or kernel configurations. Additional hardware setups would be needed to determine whether OdinSight's overhead remains similarly low across lower-end systems, CPU-limited systems, and different GPU vendors.

Second, the number of test repetitions is limited. Although each scenario uses a fixed number of MangoHud samples, repeated benchmark runs would provide stronger evidence by enabling more accurate estimation of run-to-run variance. This is important because small differences in average FPS, frametime, temperatures, and memory usage may be caused by measurement noise, background operating system activity, thermal variation, or normal benchmark variability rather than OdinSight itself.

Third, the selected workloads primarily represent GPU-heavy gaming scenarios. Unigine Superpo-

sition is useful as a repeatable synthetic benchmark, while Baldur's Gate 3 provides a more realistic game workload. However, this combination does not fully cover all types of game performance behavior. In particular, the evaluation does not include strongly CPU-bound games, simulation-heavy titles, competitive multiplayer games with high frame rates, or games with frequent asset streaming. These workloads could expose different overhead characteristics, especially if OdinSight's monitoring activity interacts more heavily with CPU-side execution or system calls.

Another limitation is that the benchmark focuses primarily on game-side performance rather than directly measuring the OdinSight daemon. Future evaluations should include dedicated measurements of the daemon's CPU usage, memory footprint, event-processing rate, and logging overhead. This would make it possible to separate the overhead observed in game performance from OdinSight's internal resource cost.

The current results support the conclusion that no clearly observable performance degradation attributable to OdinSight was identified in the tested workloads. Nevertheless, the evaluation should be viewed as an initial performance assessment rather than a complete characterization. More repeated runs, additional hardware configurations, daemon-level profiling, and a broader selection of CPU-bound and GPU-bound games would be required to make stronger general claims about OdinSight's performance impact.

9.2 Future works

This section presents possible directions for continuing the development of OdinSight. Since these improvements address different parts of the system and cannot all be implemented at once, they are ranked by priority. Table 9.1 summarizes this order, where priority 1 represents the highest priority, and priority 5 represents the lowest priority.

Priority	Objective	Rationale
1	Integrity Verification System	Establishes trust in the kernel, daemon, game binary, and required libraries before runtime enforcement can be reliable.
2	Analytical Evaluation	Identifies relevant Linux cheating techniques and helps determine which eBPF hooks and enforcement policies to prioritize.
3	Game-specific Policy Profiles	Reduces false positives by allowing security rules to be adapted to legitimate game-specific behavior.
4	Virtualized Execution Environment Detection	Addresses hypervisor-level attack scenarios where the protected environment may be observed or manipulated externally.
5	Extension of Current Evaluation	Extends evaluation across more games, engines, hardware configurations, and performance conditions.

Table 9.1: Prioritization of future development directions.

9.2.1 Integrity Verification System

As established in the Integrity Section 4.4, the current architecture lacks a reliable mechanism for verifying the integrity of the execution environment. This means that an attacker could potentially modify the kernel, game-dependent libraries, the daemon, or the game binary itself to bypass security controls. To address this, we propose a multi-layered strategy that combines integrity verification before execution with continuous validation during runtime.

To secure the kernel, we propose **remote attestation** using the system’s Trusted Platform Module (TPM). The client performs a measured boot, in which the boot sequence and kernel image are measured into the TPM’s Platform Configuration Registers (PCRs). The client can then provide a signed quote of these registers to the server, which compares the measurements against a known-good baseline. This establishes a hardware-backed chain of trust and helps ensure that the kernel has not been tampered with.

For user-space components, including the daemon, game binaries, and required libraries, we propose a **challenge-response hash verification system**. The server maintains cryptographic hashes of authorized files, organized using a **Merkle tree** to support efficient verification. During verification, the server sends a random nonce to the client, which calculates a nonce-bound hash of the target file and returns it. If the result does not match the server’s baseline, access is revoked, and the system is flagged.

To maintain trust after execution begins, this verification should be extended with a **continuous heartbeat and session validation** mechanism. The daemon periodically collects selected runtime security signals from the eBPF monitoring pipeline and sends them to an external validation

server. If the heartbeat stops, arrives outside the expected cadence, or shows abnormal hook activity, the server can treat the session as potentially compromised and restrict access.

Together, these mechanisms can provide integrity checks across the full execution lifecycle: remote attestation verifies the trusted base, file verification protects user-space components, and heartbeat validation monitors the active session. Future work should focus on optimizing the frequency and scope of these checks so they remain effective without introducing unnecessary latency or overhead.

9.2.2 Analytical Evaluation

The current evaluation would also benefit from a more analytical approach to determine the relative importance of eBPF filters and to select them optimally. For this reason, future work should include an analytical research phase focused on active threats and common exploitation techniques on Linux.

This could involve collaborating with competitive game studios, cheat developers on forums, or security researchers specializing in cheat studies. By gaining direct insight into how attackers circumvent protections, we can refine the functional requirements to focus on the most impactful hooks rather than theoretical vulnerabilities. Furthermore, the coverage metrics would naturally expand as a result of this research. Structuring these findings into a classification of true positives, false positives, true negatives, and false negatives would provide a standardized baseline, making the results more comparable for future studies.

9.2.3 Game-specific Policy Profiles

As highlighted in the compatibility results (Section 8.3.1), our current security enforcement occasionally conflicts with legitimate game operations, leading to execution failures. To address these stability issues while maintaining strong security, we propose introducing a dynamic configuration system based on game-specific policy profiles. By allowing the game server to maintain settings for each individual game, we can strike an optimal balance between security and compatibility.

These policy profiles would function as a whitelist-based mechanism. For instance, the server could enforce a strict, game-specific manifest of required libraries, reducing attempts to load malicious code or bypass security checks via `LD_LIBRARY_PATH` injection.

Furthermore, these profiles provide the flexibility to tune security enforcement on a per-game basis. Developers could selectively modify eBPF hook behavior or adjust restrictions on memory permissions, such as conditionally allowing specific patterns of writable and executable memory segments when justified by the game's architecture. This approach directly enables support for titles that require specialized runtime behavior, such as *Unturned* or *Valheim*, without compromising the global security baseline. By moving from a "one-size-fits-all" model to a per-game basis policy engine, we can reduce the rate of false positives and broaden the range of compatible titles while ensuring that each game operates within its strictly defined security boundaries. Improving the user experience for developers interested in using this solution to protect their games.

9.2.4 Virtualized Execution Environments

A critical aspect not addressed in our current implementation is the detection of virtualized execution environments. Sophisticated attackers may execute the kernel and game environment within a **Virtual Machine (VM)** to gain **hypervisor-level** visibility, effectively shielding their host environment from our detection mechanisms. By operating at a higher privilege level, an attacker could intercept and manipulate the virtualized Linux kernel, facilitating the development and deployment of undetected cheats.

To counter this, we propose **anti-virtualization** and **anti-hypervisor** detection mechanisms. A primary defense is to use timing attacks to identify the latency overhead introduced by the virtualization layer. Since hypervisors must intercept certain CPU instructions, which causes delays relative to native execution, measuring the time required for specific operations can reveal the presence of an underlying virtual environment.

In addition to timing-based checks, the system should track environmental artifacts that are unique to virtualized hardware, such as virtualized device drivers, specific I/O port patterns, or deviations in CPU information. By integrating these detections into the daemon, it can potentially identify when the system is operating in an unauthorized virtualized context. Implementing these safeguards is essential for preventing "out-of-guest" analysis and ensuring that our integrity verification mechanisms interact with a bare-metal environment, thereby closing a gap in our current protection strategy.

9.2.5 Extension of Current Evaluation

The current evaluation provides preliminary evidence, but both compatibility testing and performance benchmarking require significant expansion. Compatibility testing should incorporate a wider variety of commercial game engines, including multiple versions of Unity, Unreal Engine, and Godot, as well as proprietary custom engines. This would help ensure that OdinSight remains compatible with the diverse ways these engines manage memory and system calls.

Performance benchmarks must also be expanded to account for hardware variability and resource constraints. Future testing should categorize performance impacts across low-end, mid-range, and high-end hardware, using virtualized environments to simulate restricted specifications. Furthermore, relying on Baldur's Gate 3 alone is insufficient to establish a reliable performance profile; this title requires repeated sampling to improve accuracy. Beyond this, repeated sampling should be conducted across a broader library of titles, with a specific focus on competitive, high-frequency games where anti-cheat requirements are most stringent. The benchmarking should include CPU-bound titles alongside GPU-heavy ones, as the overhead introduced by eBPF filtering may manifest differently depending on the game's primary computational bottleneck.

To refine the implementation of eBPF filters, we propose metrics to assess daemon responsiveness. Specifically, future work should measure the latency of each eBPF hook by tracking the interval from the initial system call trigger to the hook's completion. We also propose stress testing by intentionally flooding the system with high volumes of targeted system calls to evaluate the daemon's stability under load. By repeating these benchmarks in restricted environments, such as virtual machines with throttled CPU and limited memory resources, we can better understand OdinSight's performance. These refinements would provide a more comprehensive evaluation, accurately mapping OdinSight's security impact and performance cost across the spectrum of modern Linux gaming.

9.3 Conclusion

This project investigated the following problem statement:

How can a Linux client-side anti-cheat architecture use an eBPF-based solution to provide continuous monitoring and active enforcement across the game process lifecycle?

The project addressed this question by designing and implementing OdinSight, an eBPF-based anti-cheat prototype for Linux. OdinSight explores how anti-cheat protection can be prepared before the protected game process is launched, reducing reliance on protection that begins only after the process is already running. By combining a daemon-and-launcher architecture with cgroups, eBPF programs, and LSM hooks, OdinSight supports both monitoring and enforcement for the anti-cheat daemon and the protected game process.

Can the solution enforce protection from system initialization to post-execution of the game? OdinSight partially satisfies this goal. The system validates selected kernel integrity properties before enabling protection and initializes protection before the protected game process starts. It then maintains protection throughout the process lifespan. However, OdinSight does not provide complete system-wide integrity verification from the earliest boot stages, since remote attestation and full game file validation remain outside the implemented scope.

Can eBPF detect the required events and use LSM hooks to enforce the corresponding policies? The evaluation shows that eBPF can be used for more than passive monitoring when combined with LSM hooks. OdinSight can detect and block selected forms of debugging, process inspection, memory manipulation, library injection, cgroup modification, unauthorized eBPF loading, and attempts to interfere with the daemon. This demonstrates that eBPF and LSM hooks can support active enforcement of anti-cheat policies on Linux.

Can the LSM policies function with the game execution? The compatibility and performance evaluation indicate that the implemented policies can function with four of the six tested games. OdinSight was able to run with the selected games while enforcing its policies, and performance measurements showed no clear degradation in rendering performance across the evaluated workloads. However, compatibility depends on policy strictness and individual games' behavior, so policy design must balance security coverage with practical game compatibility.

Overall, the project shows that eBPF has potential as a foundation for Linux client-side anti-cheat protection. OdinSight demonstrates that protection can be established before game execution, maintained during runtime, and applied to both the anti-cheat daemon and the protected game process. However, OdinSight should be understood as a client-side foundation rather than a complete anti-cheat solution. A full system would require a complementary server-side component to validate client integrity, verify session state, and make trust decisions that cannot be fully guaranteed by the local machine alone.

Bibliography

- [1] A. Alangari and O. Alharbi, **A systematic review of technical defenses against software-based cheating in online multiplayer games**, Dec. 2025. DOI: 10.48550/arXiv.2512.21377.
- [2] S. F. J. J. Ankergård, E. Dushku, and N. Dragoni, “State-of-the-art software-based remote attestation: Opportunities and open issues for internet of things,” **Sensors**, vol. 21, no. 5, 2021. DOI: 10.3390/s21051598. [Online]. Available: <https://www.mdpi.com/1424-8220/21/5/1598>.
- [3] Arch Linux Wiki contributors, **Signed kernel modules**, https://wiki.archlinux.org/title/Signed_kernel_modules, Arch Wiki, accessed on 2026-01-07, Arch Linux, 2026.
- [4] S. Berger. “Today i learned – kernel.modules_disabled.” dfir.ch blog post, accessed on 2026-01-07. [Online]. Available: https://dfir.ch/posts/today_i_learned_lkm_kernel_modules_disabled/.
- [5] B. Borisov. “Linux falls hard on steam after record 5% milestone,” Accessed: Jun. 9, 2026. [Online]. Available: <https://linuxiac.com/linux-falls-hard-on-steam-after-record-5-milestone/>.
- [6] “Bpf maps.” Accessed 2026-04-21, Linux kernel documentation. [Online]. Available: <https://docs.kernel.org/bpf/maps.html>.
- [7] systemd developers, **Systemd(1) -- systemd system and service manager**, man7.org. Accessed: Jun. 6, 2026. [Online]. Available: <https://man7.org/linux/man-pages/man1/systemd.1.html>.
- [8] W. Dietz, J. Cranmer, N. Dautenhahn, and V. S. Adve, “Slipstream: Automatic interprocess communication optimization,” in **2015 USENIX Annual Technical Conference (USENIX ATC 15)**, USENIX Association, 2015, pp. 431–443. [Online]. Available: <https://www.usenix.org/system/files/conference/atc15/atc15-paper-dietz.pdf>.
- [9] H. Duh and V. Chen, “Cheating behaviors in online gaming,” vol. 5621, Jul. 2009, pp. 567–573. DOI: 10.1007/978-3-642-02774-1_61.
- [10] eBPF Docs, **BPF CO-RE**, <https://docs.ebpf.io/concepts/core/>, Accessed: 2026-06-10, 2025.
- [11] eBPF Docs, **eBPF on Linux**, <https://docs.ebpf.io/linux/>, Accessed: 2026-06-10, 2025.
- [12] “Ebpf syscall.” Accessed 2026-04-21, Linux kernel documentation. [Online]. Available: <https://docs.kernel.org/userspace-api/ebpf/syscall.html>.
- [13] F. Gadaleta, N. Nikiforakis, Y. Younan, and W. Joosen, **Hello rootkitty: A lightweight invariance-enforcing framework**, 2014. [Online]. Available: <https://arxiv.org/abs/1405.5651>.
- [14] J. Hagl, O. Mann, and M. Pirker, “Securing the linux boot process: From start to finish,” in **International Conference on Information Systems Security and Privacy**, 2021. [Online]. Available: <https://api.semanticscholar.org/CorpusID:232106536>.

- [15] R. Jain, **The art of computer systems performance analysis: techniques for experimental design, measurement, simulation, and modeling**. Wiley New York, 1991.
- [16] Java Design Patterns Project. “Reactor pattern in java: Mastering non-blocking event-driven architectures.” Accessed: 2026-04-21. [Online]. Available: <https://java-design-patterns.com/patterns/reactor/>.
- [17] **Kernel_lockdown(7) --- linux manual page**, Linux man-pages 6.16, accessed 2026-04-21, Linux man-pages project, May 2025. [Online]. Available: https://www.man7.org/linux/man-pages/man7/kernel_lockdown.7.html.
- [18] M. Kerrisk, **Bpf(2) --- linux manual page**, Accessed 2026-04-21, man7.org. [Online]. Available: <https://man7.org/linux/man-pages/man2/bpf.2.html>.
- [19] E. K. Larsen, F. Anulis, M. H. Mariager, T. J. S. Merstrand, and A. H. Paludan, “Tyrsecure - an eBPF-based anti-cheat solution for linux,” Aalborg Universitet, Tech. Rep., 2026, p. 49. [Online]. Available: https://kdbk-aub.primo.exlibrisgroup.com/permalink/45KBDK_AUB/3b147k/alma9922322310005762.
- [20] Linux Man-pages Project, **Cgroups(7) – linux manual page**, Accessed: 2026-04-21, 2024. [Online]. Available: <https://man7.org/linux/man-pages/man7/cgroups.7.html>.
- [21] Linux man-pages project, **Bash(1) -- linux manual page**, Available at: <https://man7.org/linux/man-pages/man1/bash.1.html>. Accessed: 2026-06-06, man7.org.
- [22] Linux man-pages project, **Epoll(7) --- i/o event notification facility**, man 7 epoll, Online; accessed 2026-04-14, 2025. [Online]. Available: <https://www.man7.org/linux/man-pages/man7/epoll.7.html>.
- [23] Linux man-pages project, **Poll, ppoll - wait for some event on a file descriptor**, man 7 poll, Online; accessed 2026-04-14, 2025. [Online]. Available: <https://www.man7.org/linux/man-pages/man2/poll.2.html>.
- [24] Linux man-pages project, **Unix(7) --- linux manual page**, Linux man-pages 6.15, kernel.org, 2025. [Online]. Available: <https://www.kernel.org/pub/linux/docs/man-pages/book/man-pages-6.15.pdf>.
- [25] LordShen. “Invisible power: Loadable kernel module (lkm).” Medium article, accessed 2026-04-21. [Online]. Available: <https://medium.com/@lordshen/invisible-power-loadable-kernel-module-lkm-30c7aa52d19d>.
- [26] R. Love, **Linux Kernel Development**, 3rd ed. 2010.
- [27] W. Maurer, **Professional Linux Kernel Architecture**. Wrox Press, 2008.
- [28] P. Menage, “Adding generic process containers to the linux kernel,” in **Proceedings of the Linux Symposium**, vol. 2, 2007, pp. 45–57. [Online]. Available: <https://www.kernel.org/doc/ols/2007/ols2007v2-pages-45-58.pdf>.
- [29] Microsoft. “Trusted platform module technology overview.” [Online]. Available: <https://learn.microsoft.com/en-us/windows/security/hardware-security/tpm/trusted-platform-module-overview>.
- [30] S. L. Narasimhan, “Remote attestation of user-space applications at runtime,” Master’s Programme in Security and Cloud Computing, Master’s thesis, Aalto University, Otaniemi, Finland, Sep. 2025. [Online]. Available: <https://aaltodoc.aalto.fi/items/3eab5e33-1d3e-409c-b70e-33536ac13060>.
- [31] L. Nelson, J. V. Geffen, E. Torlak, and X. Wang, “Specification and verification in the field: Applying formal methods to BPF just-in-time compilers in the linux kernel,” in **14th USENIX Symposium on Operating Systems Design and Implementation (OSDI 20)**, USENIX Association, Nov. 2020, pp. 41–61. [Online]. Available: <https://www.usenix.org/conference/osdi20/presentation/nelson>.

- [32] B. Parno, J. M. McCune, and A. Perrig, “Bootstrapping trust in commodity computers,” in **Proceedings of IEEE Symposium on Security and Privacy**, May 2010. [Online]. Available: <https://ieeexplore.ieee.org/document/5504802>.
- [33] “Program types (linux).” Accessed 2026-04-21, eBPF Docs. [Online]. Available: <https://docs.ebpf.io/linux/program-type/>.
- [34] L. man-pages project, **Syscalls(2) -- linux manual page**, Available at: <https://man7.org/linux/man-pages/man2/syscalls.2.html>. Accessed: 2026-06-06, man7.org.
- [35] S. P. Rao, G. Limonta, and J. Lindqvist, “Usability and security of trusted platform module (TPM) library apis,” in **Proceedings of the Eighteenth Symposium on Usable Privacy and Security (SOUPS 2022)**, Boston, MA: USENIX Association, Aug. 2022, pp. 213–232. [Online]. Available: <https://www.usenix.org/conference/soups2022/presentation/rao>.
- [36] R. Sailer, X. Zhang, T. Jaeger, and L. van Doorn, “Design and implementation of a tcb-based integrity measurement architecture,” in **Proceedings of the 13th Conference on USENIX Security Symposium - Volume 13**, ser. SSYM’04, San Diego, CA: USENIX Association, 2004, p. 16. [Online]. Available: <https://dl.acm.org/doi/10.5555/1251375.1251391>.
- [37] D. C. Schmidt, “Reactor: An object behavioral pattern for concurrent event demultiplexing and event handler dispatching,” in **Pattern languages of program design**, USA: ACM Press/Addison-Wesley Publishing Co., 1995, pp. 529–545. [Online]. Available: <https://dl.acm.org/doi/abs/10.5555/218662.218705>.
- [38] A. Silberschatz, G. Gagne, and P. Galvin, **Operating System Concepts**. Jan. 2002.
- [39] J. Stühn, J.-N. Hilgert, and M. Lambertz, “The hidden threat: Analysis of linux rootkit techniques and limitations of current detection tools,” **Digital Threats**, vol. 5, no. 3, Oct. 2024. DOI: 10.1145/3688808. [Online]. Available: <https://doi.org/10.1145/3688808>.
- [40] systemd, **systemd.special and bootup process documentation**, <https://www.freedesktop.org/software/systemd/man/latest/bootup.html>. Accessed: 2026-06-06, Accessed: 2026-06-06.
- [41] systemd project, **Daemon -- systemd manual page**, Available at: <https://www.freedesktop.org/software/systemd/man/daemon.html>. Accessed: 2026-06-06, freedesktop.org.
- [42] The kernel development community. “Linux Security Module Usage.” Linux Kernel Documentation, version 7.1.0-rc6, Accessed: Jun. 5, 2026. [Online]. Available: <https://docs.kernel.org/admin-guide/LSM/index.html>.
- [43] The Linux Kernel Documentation, **Overview of the linux virtual file system**, Available at: <https://docs.kernel.org/filesystems/vfs.html>. Accessed: 2026-06-06, kernel.org.
- [44] The Linux Kernel Organization, **Ramfs, rootfs and initramfs**, Available at: <https://www.kernel.org/doc/Documentation/filesystems/ramfs-rootfs-initramfs.rst>. Accessed: 2026-06-06, 2026.
- [45] The Wolf Den – A Space for Ideas, Takes, and Chaos. “Linux is finally a real option for gamers, Why i’m leaving windows without regret,” Accessed: Jun. 9, 2026. [Online]. Available: <https://medium.com/@wolfman1546/linux-is-finally-a-real-option-for-gamers-cc07a32e7480>.
- [46] L. Torvalds et al., **Linux kernel source: Fs/eventpoll.c**, version v7.1, Accessed: 2026-04-21, 2026. [Online]. Available: <https://github.com/torvalds/linux/blob/master/fs/eventpoll.c>.
- [47] UEFI Forum, **UEFI Specification**, Available at: <https://uefi.org/specifications>. Accessed: 2026-06-06.
- [48] **Verifier**, Accessed 2026-04-21, eBPF Docs, 2024. [Online]. Available: <https://docs.ebpf.io/linux/concepts/verifier/>.

- [49] R. Wilkins. "The chain of trust. "[Online]. Available: https://uefi.org/sites/default/files/resources/UEFI%20Forum%20White%20Paper%20-%20Chain%20of%20Trust%20Introduction_Final.pdf.
- [50] J. Yan and B. Randell, "A systematic classification of cheating in online games," Oct. 2005, pp. 1–9. DOI: 10.1145/1103599.1103606.

Appendices

Appendix A

Compatibility tests

Games were executed via the OdinSight launcher using the command `./OdinSight_launcher`. Upon initialization, a text-based user interface (TUI) presents a menu of available titles; the corresponding game ID is then entered to begin the test session.

To facilitate log analysis, console output is color-coded to indicate system status:

- **Green:** Signifies successful execution and operational stability.
- **Red:** Highlights errors, failed dependencies, or system-level rejections.

AssaultCube

The launch environment was configured using the following game directory:

```
~/games/AssaultCube/
```

The binary path was set to:

```
~/games/AssaultCube/bin/_unix/linux/_64/_client
```

The required libraries were already present in the host environment, so it was not needed to add any additional libraries to `LD_LIBRARY_PATH`

```
AssaultCube Logs
GAME LOG
init: sdl init: net init: world init: video: sdl init: video: mode init: video: misc
init: gl
...
init: cfg init: models init: curl init: docs init: localconnect
...
loaded textures (27 milliseconds) loaded mapmodels (18 milliseconds) loaded mapsounds
(93 milliseconds) game mode is "TDM" init: mainloop
BPF_TRACE_PRINTK:
-
```

Figure A.1

As shown in Figure A.1, the game launched successfully without encountering errors or eBPF-related security rejections. To verify network compatibility, two clients connected to a public server simultaneously, confirming that the OdinSight launch does not interfere multiplayer gameplay. Figure A.2 demonstrates a successful session where two clients are connected to a public server.



Figure A.2: Multiplayer session in AssaultCube

SuperTuxKart

The launch environment was configured using the following game directory:

```
~/.games/SuperTuxKart/
```

The binary path was set to:

```
~/.games/SuperTuxKart/bin/SuperTuxKart
```

Because the application depends on both 32-bit and 64-bit system libraries, as well as its own bundled binaries, the `LD_LIBRARY_PATH` environment variable was explicitly configured to include the required local and host library directories:

```
LD_LIBRARY_PATH=~/.games/SuperTuxKart/lib:/usr/lib:/usr/lib32:/lib:/lib32
```

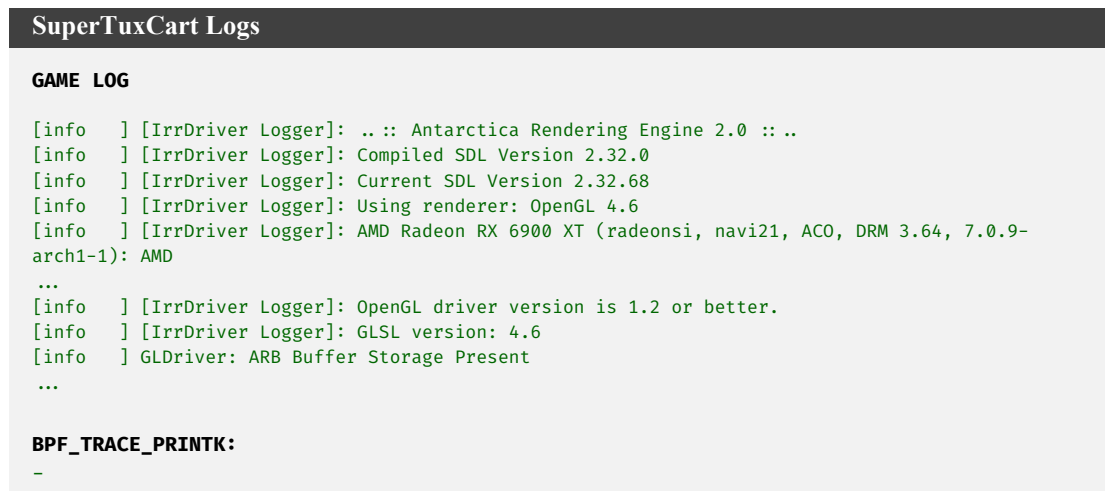


Figure A.3

As shown in Figure A.3, the game launched successfully without encountering errors or eBPF-related security rejections. To verify network compatibility, two clients connected to a public server simultaneously, confirming that the OdinSight launch does not interfere multiplayer gameplay. Figure A.4 demonstrates a successful session where two clients are connected to a public server.

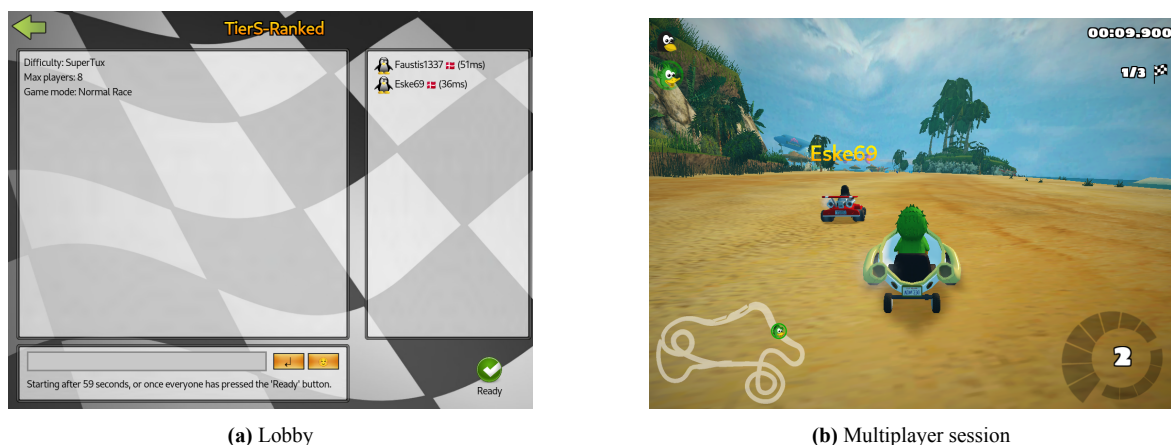


Figure A.4: Gameplay of SuperTuxCart

A Story About My Uncle

The launch environment was configured using the following game directory:

```
~/.local/share/Steam/steamapps/common/A Story About My Uncle/Binaries/linux-amd64/
```

The binary path was set to:

```
~/.local/share/Steam/steamapps/common/A Story About My Uncle/Binaries/linux-amd64/ASAMU
```

Because the application depends on both 32-bit and 64-bit system libraries, as well as its own

bundled binaries, the LD_LIBRARY_PATH environment variable was explicitly configured to include the required local and host library directories:

```
~/local/share/Steam/steamapps/common/A Story About My Uncle/Binaries/linux-  
amd64/:usr/lib:usr/lib32:lib:lib32
```

```
A Story About My Uncle Logs  
  
GAME LOG  
  
[S_API FAIL] SteamAPI_Init() failed;  
SteamAPI_IsSteamRunning() failed.  
[S_API FAIL] SteamAPI_Init() failed; unable to locate a running instance of Steam,  
or a local steamclient.so.  
[S_API FAIL] SteamAPI_Init() failed; SteamAPI_IsSteamRunning() failed.  
dlopen failed trying to load: steamclient.so with error:  
steamclient.so: cannot open shared object file: No such file or directory  
[S_API FAIL] SteamAPI_Init() failed; unable to locate a running  
instance of Steam, or a local steamclient.so.  
Connection to "Provider=sqloledb;Data Source=production-db;Initial  
Catalog=EngineTaskPerf;Trusted_Connection=Yes;Connection Timeout=2" or "10.1.20.20" failed  
  
Loading library: libPhysXCooking.so Result of loading library libPhysXCooking.so:  
0xfc8b530 Loading library: PhysXUpdateLoader.so Result of loading library  
PhysXUpdateLoader.so: 0xfc8bc90 Loading library: libPhysXCore.so Result of loading  
library libPhysXCore.so: 0xfc8c300  
  
BPF_TRACE_PRINTK:  
capable denied: [caller pid=165795 cgid=8061 cap=21 opts=0] [protected cgid=8061]
```

Figure A.5

As shown in Figure A.5, the game launch log reported several errors. These included SteamAPI initialization failures and a capability denial for capability 21, SYS_ADMIN. This capability was intentionally denied by the eBPF hook because it grants access to a broad set of privileged kernel operations, including namespace, mount, and other system-administration functions. Allowing it inside the protected game process would make the policy too permissive, as a compromised process could potentially use the capability to modify its execution environment or bypass parts of the isolation mechanism. However, these messages did not prevent the game from launching successfully. Figure A.6 shows a successful single-player session, confirming that the game was able to run despite the reported launch warnings.



Figure A.6: Singleplayer session in ASAMU

Baldur's Gate 3

The launch environment was configured using the following game directory:

```
~/.local/share/Steam/steamapps/common/Baldurs Gate 3/
```

The binary path was set to:

```
~/.local/share/Steam/steamapps/common/Baldurs Gate 3/bin/bg3
```

The application only depends on it's own bundled libraries, the LD_LIBRARY_PATH environment variable was configured to include the required local library directory:

```
~/.local/share/Steam/steamapps/common/Baldurs Gate 3/bin/
```

Baldur's Gate 3

GAME LOG

```
steam.sh[165251]: Running Steam on arch rolling 64-bit
steam.sh[165251]: STEAM_RUNTIME is enabled automatically
setup.sh[165283]: Steam runtime environment up-to-date!
steam.sh[165251]: Error: You are missing the following 32-bit libraries, and
Steam may not run: libc.so.6
steam-runtime-check-requirements[165726]: W: Child process exited with code 1:
bwrap: No permissions to creating new namespace, likely because the kernel does not allow non-
privileged user namespaces.

BPF_TRACE_PRINTK:
LD_PRELOAD detected before exec: [caller pid=165329 cgid=8061] [protected cgid=8061]
exec denied: [reason=LD_PRELOAD] [caller pid=165329 cgid=8061] [protected cgid=8061]
capable denied: [caller pid=165396 cgid=8061 cap=21 opts=0] [protected cgid=8061]
```

Figure A.7

As shown in Figure A.7, the game launch log reported several errors. These included Steam related failures from its use of LD_PRELOAD. While this mechanism is legitimate for library loading, it is also a common method for library injection attacks. Furthermore, the Steam runtime requires the CAP_SYS_ADMIN capability to create new namespaces. As previously stated, this capability is intentionally blocked by OdinSight.

Interestingly, when the Steam client is closed the game proceeds to run successfully without eBPF rejections. The gameplay of Baldur's Gate 3 is illustrated in Figure A.8, which depicts a successful multiplayer session with two clients connected, confirming that the game functions correctly despite the initial Steam-side initialization errors.

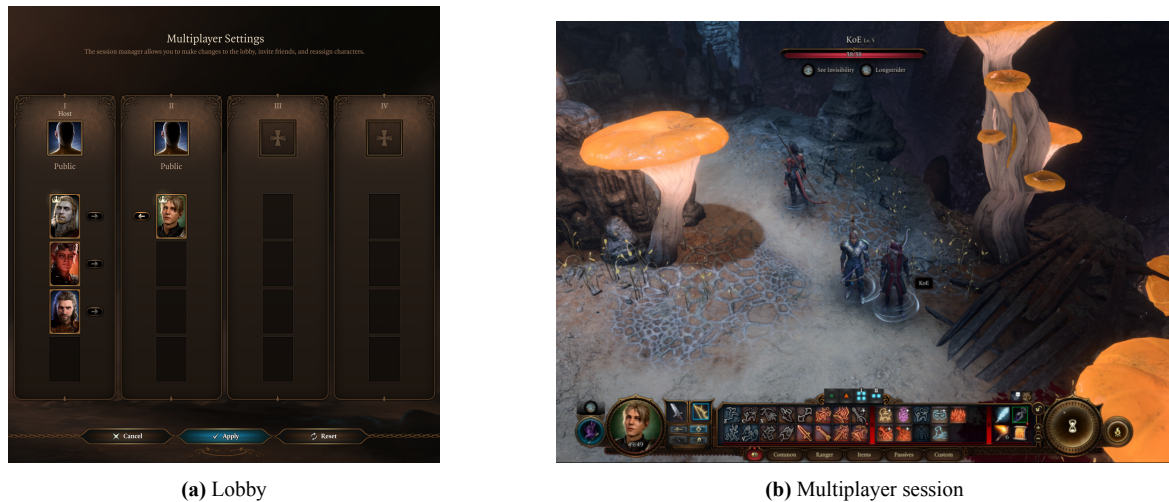


Figure A.8: Gameplay of Baldur's Gate 3

Valheim

The launch environment was configured using the following game directory:

```
~/.local/share/Steam/steamapps/common/Valheim/
```

The binary path was set to:

```
~/.local/share/Steam/steamapps/common/Valheim/valheim.x86_64
```

Because the application depends on its own bundled game libraries, the LD_LIBRARY_PATH environment variable was configured to include the required local game library directory:

```
LD_LIBRARY_PATH=~/.local/share/Steam/steamapps/common/Valheim/
```

```
LD_LIBRARY_PATH+=~/.local/share/Steam/steamapps/common/Valheim/valheim_Data/Plugins/
```

```
Valheim Logs

GAME LOG
[UnityMemory] Configuration Parameters - Can be set up in boot.config
"memorysetup-bucket-allocator-granularity=16"
...
"memorysetup-thread-allocator-block-size=16777216"
"memorysetup-gfx-main-allocator-block-size=16777216"
"memorysetup-gfx-thread-allocator-block-size=16777216"
"memorysetup-cache-allocator-block-size=4194304"
"memorysetup-typetree-allocator-block-size=2097152"
..
"memorysetup-profiler-editor-allocator-block-size=1048576"
"memorysetup-temp-allocator-size-main=4194304"
"memorysetup-job-temp-allocator-block-size=2097152"
...
"memorysetup-temp-allocator-size-background-worker=32768"
"memorysetup-temp-allocator-size-job-worker=262144"
"memorysetup-temp-allocator-size-preload-manager=262144"
"memorysetup-temp-allocator-size-nav-mesh-worker=65536"
"memorysetup-temp-allocator-size-audio-worker=65536"
"memorysetup-temp-allocaneighbourtor-size-cloud-worker=32768"
"memorysetup-temp-allocator-size-gfx=262144"

BPF_TRACE_PRINTK:
mmap_file denied: [caller pid=19566 cgid=13624] [reason=wx reqprot=7 prot=7 flags=98]
[protected cgid=13624]
mmap_file denied: [caller pid=19566 cgid=13624] [reason=wx reqprot=7 prot=7 flags=98]
[protected cgid=13624]
```

Figure A.9

As shown in Figure A.9, the launch of Valheim was unsuccessful with OdinSight. The game began initializing the Unity memory subsystem and printed its allocator configuration, but the process exited shortly afterwards. The eBPF trace log shows that OdinSight denied two `mmap_file` requests because they attempted to create memory with both write and execute permissions.

Mapping memory with both write and execute permissions is restricted to prevent code injection and runtime code modification. In this case, the denied mapping may be related to the Unity engine or one of Valheim's bundled native plugins, which can require runtime-generated executable memory during startup. However, because this operation violates the OdinSight memory protection policy, the launch was blocked and the game exited before reaching a playable state.

Unturned

The launch environment was configured using the following game directory:

```
~/local/share/Steam/steamapps/common/Unturned/
```

The binary path was set to:

```
~/local/share/Steam/steamapps/common/Unturned/Unturned.x86_64
```

The game depends on its own bundled game libraries and host libs, so the `LD_LIBRARY_PATH` environment variable was configured to include the required library directories:

LD_LIBRARY_PATH=~/.local/share/Steam/steamapps/common/Unturned

LD_LIBRARY_PATH+=~/.local/share/Steam/steamapps/common/Unturned/Unturned_Data/Plugins

```
Unturned Logs

GAME LOG
[UnityMemory] Configuration Parameters - Can be set up in boot.config
"memorysetup-bucket-allocator-granularity=16"
...
"memorysetup-thread-allocator-block-size=16777216"
"memorysetup-gfx-main-allocator-block-size=16777216"
"memorysetup-gfx-thread-allocator-block-size=16777216"
"memorysetup-cache-allocator-block-size=4194304"
"memorysetup-typetree-allocator-block-size=2097152"
...
"memorysetup-profiler-editor-allocator-block-size=1048576"
"memorysetup-temp-allocator-size-main=4194304"
"memorysetup-job-temp-allocator-block-size=2097152"
...
"memorysetup-temp-allocator-size-background-worker=32768"
"memorysetup-temp-allocator-size-job-worker=262144"
"memorysetup-temp-allocator-size-preload-manager=262144"
"memorysetup-temp-allocator-size-nav-mesh-worker=65536"
"memorysetup-temp-allocator-size-audio-worker=65536"
"memorysetup-temp-allocator-size-cloud-worker=32768"
"memorysetup-temp-allocator-size-gfx=262144"

BPF_TRACE_PRINTK:
mmap_file denied: [caller pid=27886 cgid=14456] [reason=wx reqprot=7 prot=7 flags=98]
[protected cgid=14456]
mmap_file denied: [caller pid=27886 cgid=14456] [reason=wx reqprot=7 prot=7 flags=98]
[protected cgid=14456]
```

Figure A.10

As shown in Figure A.10, the launch of Valheim was unsuccessful with OdinSight. Unturned had identical outputs to Valheim, which indicate that the design of the Unity Engine relies on memory mapping with execute and write permissions.

Appendix B

Coverage tests

To facilitate analysis, log outputs are color-coded:

- Green: Indicates successful execution and operational stability.
- Red: Indicates errors, failed dependencies, or system-level rejections.

Kernel Space

As it can be seen in Figure B.1, the KI-01 results show that OdinSight accepts the system state when the kernel is configured with Secure Boot, Kernel Lockdown and Module Signature Enforcement enabled and loading of unsigned kernel module is denied. However, the KI-02 to KI-05 results show that OdinSight exits immediately when any of these protections are disabled or an unsigned kernel module is allowed to be loaded into the system.

```

KI 01 to 05 Output
INPUT
$ sudo ./OdinSight_daemon
KI-01
OUTPUT: load unsigned kernel module denied by signature policy failed: Key was
rejected by service (129)
KI-02
OUTPUT: Invalid: Secure Boot - Disabled | load unsigned kernel module denied ... (129)
KI-03
OUTPUT: Invalid: Kernel lockdown - Disabled | load unsigned kernel module denied ...
(129)
KI-04
OUTPUT: Kernel module signature enforcement - Disabled | Operation not permitted (1)
KI-05
OUTPUT: Unsigned kernel module was loaded in
```

Figure B.1

Anti-cheat Software

Figure B.2 presents the result of the unauthorized eBPF loading test. OdinSight denies the BPF loading operation when the PID of the calling process does not match the PID of the OdinSight

daemon. The BPF trace printk output confirms that this policy prevents eBPF hooks from being loaded by processes outside the daemon.

```
DP-BPF-01 Output

INPUT
$ sudo bpftool prog load game_ebpf_protection.o /sys/fs/bpf/module_tp autoattach

OUTPUT
libbpf: Error in bpf_object__probe_loading(): -EPERM. Couldn't load trivial BPF
program. Make sure your kernel supports BPF (CONFIG_BPF_SYSCALL=y) and/or that
RLIMIT_MEMLOCK is set to big enough value. Error: failed to load object file

BPF_TRACE_PRINTK:
eBPF operation denied: [caller pid=18558] attempted cmd=5 on bpf object [protected
pid=18199]
```

Figure B.2

The IPC protection result is shown in Figure B.3. OdinSight denies the kill operation when the PID of the calling process does not match the PID of the OdinSight daemon. The BPF trace printk output confirms that the eBPF hook prevents termination signals from reaching the OdinSight daemon.

```
DP-IPC-01 Output

INPUT
$ sudo kill -9 15949

OUTPUT
sending signal to 15949 failed: Operation not permitted

BPF_TRACE_PRINTK:
Signal kill denied: [caller pid=16208] attempted to send signal 9 to daemon pid=15949
```

Figure B.3

Figure B.4 demonstrates the resource-control enforcement. OdinSight prevents unauthorized processes from modifying the daemon's execution parameters, including CPU priority through `renice`, scheduling policy through `chrt`, I/O priority through `ionice`, and file descriptor limits through `prlimit`. Each operation is denied with `Operation not permitted`, and the corresponding BPF trace output confirms that the denial is triggered by the relevant LSM hook. These results show that OdinSight protects the daemon from external attempts to degrade, starve, or destabilize its execution.

```
DP-RES 01 to 04 Output

RES-1
INPUT: $ sudo renice -n -10 -p 14677
OUTPUT: renice: failed to set priority for 14677 (process ID): Operation not permitted
TRACE: LSM Denied: task_setnice | Actor: renice (PID: 16307) → Target: 14677 | New
Nice: -10

RES-2
INPUT: $ sudo chrt -f -p 99 14677
OUTPUT: chrt: failed to set pid 14677's policy: Operation not permitted
TRACE: LSM Denied: task_setscheduler | Actor: chrt (PID: 15243) → Target: 14677

RES-3
INPUT: $ sudo ionice -c 1 -n 0 -p 14677
OUTPUT: ionice: ioprio_set failed: Operation not permitted
TRACE: LSM Denied: task_setioprio | Actor: ionice (PID: 16971) → Target: 14677 | IO
Prio: 8192

RES-4
INPUT: $ sudo prlimit --pid 14677 --nofile=100:100
OUTPUT: prlimit: failed to set the NOFILE resource limit: Operation not permitted
TRACE: LSM Denied: task_setrlimit | Actor: prlimit (PID: 17450) → Target: 14677 |
Resource: 7, Limit: 100
```

Figure B.4

The debugger-attachment test in Figure B.5 shows that OdinSight denies unauthorized `ptrace` access to the daemon. When an external process attempts to attach to the OdinSight daemon using `gdb`, the operation is rejected with `Operation not permitted`. The BPF trace `printk` output confirms that the unauthorized `ptrace_access_check` operation was blocked.

```
DP-DBG-01 Output

INPUT
$ sudo gdb -p 8100

OUTPUT
Attaching to process 8100 ptrace: Operation not permitted

BPF_TRACE_PRINTK:
Blocked unauthorized ptrace_access_check
```

Figure B.5

The `procfs` access-control test in Figure B.6 demonstrates that OdinSight protects daemon-specific process information without restricting unrelated system-wide `procfs` entries. In FS-01, an unauthorized attempt to access the daemon's process-specific `procfs` entry under `/proc` is denied with `Operation not permitted`. In contrast, FS-02 shows that access to unrelated system-wide `procfs` information, such as `/proc/cpuinfo`, remains allowed and does not generate a BPF trace. This confirms that OdinSight selectively restricts access to daemon-specific `procfs` entries rather than applying a blanket restriction to the entire `procfs` filesystem.

DP-FS 01 to 02 Output

FS-01

INPUT: \$ cat /proc/19392

OUTPUT: /proc/19392: Operation not permitted

TRACE: procfs access denied: [caller pid=19656] [target pid=19392 file=/proc/19392 mask=36]

FS-02

INPUT: \$ cat /proc/cpuinfo

OUTPUT: processor : 0 ... (Truncated)

TRACE: (No BPF trace generated)

Figure B.6

Game Assets and Process Lifecycle

Figure B.7 presents the debugging and process-inspection protections applied to the protected game process. In DBG-1, an external debugger attempts to attach to the game using gdb, but the ptrace request is denied. In DBG-2, scanmem attempts to inspect the protected process memory by reading process information exposed through /proc, but access to the protected procfs entry is blocked. In DBG-3, a custom injected library seen in figure 8.1, attempts to invoke PTRACE_TRACEME from inside the game process, but OdinSight denies the parent-based tracing attempt. Finally, DBG-4 shows that perf_event_open is denied, preventing performance-monitoring interfaces from being used to observe the protected game. These results confirm that OdinSight blocks multiple debugging and inspection surfaces.

GP-DBG 01 to 04 Output

DBG-1

INPUT: \$ sudo gdb -p 5088

OUTPUT: Attaching to process 5088

ptrace: Operation not permitted.

TRACE: ptrace access denied: [caller pid=5633 cgid=5503] [target pid=5088 cgid=5451 mode=18] [protected cgid=5451]

DBG-2

INPUT: \$ scanmem -p 48446

OUTPUT: error: failed to open maps file /proc/5088/maps.

error: sorry, there was a problem getting a list of regions to search.

warn: the pid may be invalid, or you don't have permission.

Enter the pid of the process to search using the "pid" command.

Enter "help" for other commands.

TRACE: procfs access denied: [caller pid=8346 cgid=5503] [target pid=5088 cgid=5451 file=/proc/5088/ mask=129] [protected cgid=5451]

DBG-3

INPUT: LD_PRELOAD=ptrace_me_library.so injected into the game.

TRACE: ptrace_traceme denied: [caller pid=33854 cgid=14989] [parent pid=33590 cgid=10650] [protected cgid=14989]

DBG-4

INPUT: \$ sudo perf stat

OUTPUT: Performance counter stats for 'system wide': <not supported> cpu-cycles ## nan GHz cycles_frequency 8.050426005 seconds time elapsed

TRACE: perf_event_open denied: [caller pid=21281 cgid=12411 type=0] [protected cgid=12463]

Figure B.7

Figure B.8 illustrates OdinSight's memory-protection enforcement for the protected game process. In

MEM-1, Valheim triggers a memory mapping request with both writable and executable permissions, which OdinSight denies as a wx violation. In MEM-2, the injected anonymous_exec_library.so seen in figure 8.2, attempts to create anonymous executable memory using mmap, but the request is denied because such mappings can be used to introduce runtime code into the protected process. In MEM-3, the injected wx_mprotect_library.so, seen in figure 8.3, first creates anonymous readable memory and then attempts to change it to writable and executable memory using mprotect. OdinSight blocks this transition through the file_mprotect hook. These results show that OdinSight prevents suspicious runtime memory permissions, including writable-executable mappings, anonymous executable memory, and permission changes that could enable injected code execution.

```

GP-MEM 01 to 03 Output

MEM-1
INPUT Launch Valheim through the OdinSight.
OUTPUT: [UnityMemory] Configuration Parameters - Can be set up in boot.config
"memorysetup-bucket-allocator-granularity=16"....
TRACE: mmap_file denied: [caller pid=28638 cgid=7184] [reason=wx reqprot=7 prot=7
flags=98] [protected cgid=7184]

MEM-2
INPUT: LD_PRELOAD=anonymous_exec_library.so injected into the game
TRACE: mmap_file denied: [caller pid=36865 cgid=15457] [reason=anonymous_exec
reqprot=5 prot=5 flags=34] [protected cgid=15457]

MEM-3
INPUT: LD_PRELOAD=wx_mprotect_library.so injected into the game
TRACE: file_mprotect denied: [caller pid=41886 cgid=16885] [reason=wx reqprot=6
prot=6] [protected cgid=16885]

```

Figure B.8

The injection test in Figure B.9 demonstrates OdinSight's pre-execution protection against LD_PRELOAD-based library injection. The daemon detects the LD_PRELOAD environment variable before the protected game process is executed and rejects the launch attempt. By preventing unauthorized shared objects from being loaded into the game process, OdinSight mitigates potential hooking, aimbot, wallhack, and memory-tampering techniques that rely on injected libraries.

```

GP-INJ-01 Output

INPUT
Launch Assault Cube through Odinsight with cheats by injecting
LD_PRELOAD=libc_rhack.so into the game

OUTPUT
[Error] Failed to launch game process: [Launcher::Runner] child_exec_prep failed:
Operation not permitted (1)

BPF_TRACE_PRINTK:
LD_PRELOAD detected before exec: [caller pid=22517 cgid=6300] [protected cgid=6300]

```

Figure B.9

Figure B.10 presents the filesystem-level restriction results for the protected game tests. In FS-01, an unauthorized attempt to access the protected game's process-specific procfs entry under /proc is denied with Operation not permitted. In FS-02, an unauthorized attempt to write to the game's cgroup directory is also denied. The BPF trace output confirms that OdinSight prevents external processes from accessing sensitive game process metadata and modifying files inside the protected game's cgroup.

GP-FS 01 to 02 Output

FS-01

INPUT: \$ cd /proc | sudo cat 6062

OUTPUT: cat: 6062: Operation not permitted

TRACE: procfs access denied: [caller pid=6154 cgid=6141] [target pid=6062 cgid=6089 file=/proc/6062/ mask=16] [protected cgid=6089]

FS-02

INPUT: \$ sudo touch /sys/fs/cgroup/OdinSight/game/test.txt

OUTPUT: touch: cannot touch ... : Operation not permitted

TRACE: cgroupfs access denied: [caller pid=44498 cgid=17158] [target cgid=18022 file=game mask=3] [protected cgid=18022]

Figure B.10

Figure B.11 shows OdinSight's mount-level protection for the protected game's cgroup path. In MNT-01, an attempt to move an external mount point into the protected game cgroup directory is denied. In MNT-02, OdinSight also denies an attempt to move the protected game cgroup directory out to another mount location. Together, these results show that OdinSight prevents external processes from replacing or relocating the protected game's cgroup filesystem path through mount operations.

GP-MNT 01 to 02 Output

MNT-01

INPUT: \$ sudo mount --move /tmp/mnt-from /sys/fs/cgroup/OdinSight/game

OUTPUT: permission denied.

TRACE: move_mount denied INTO protected cgroup: target cgid=12463

MNT-02

INPUT: \$ sudo mount --move /sys/fs/cgroup/OdinSight/game /tmp/mnt-out

OUTPUT: permission denied.

TRACE: move_mount denied FROM protected cgroup: target cgid=12463

Figure B.11

The capability test in Figure B.12 demonstrates that OdinSight restricts privileged capability use inside the protected game environment. In CAP-01, Baldur's Gate 3 is launched while Steam is open, and the Steam runtime attempts to create a new namespace using bwrap. This operation is denied, resulting in the reported namespace permission error. The BPF trace output confirms that OdinSight blocked capability 21, corresponding to CAP_SYS_ADMIN, for a process inside the protected cgroup. This shows that OdinSight restricts privileged administrative operations, such as namespace-related actions, when they are attempted within the protected game context.

GP-CAP-01 Output

INPUT

Launch Baldurs Gate 3 through OdinSight with Steam open.

OUTPUT

steam-runtime-check-requirements[36791]: W: Child process exited with code 1: bwrap: No permissions to creating new namespace, likely because the kernel does not allow non-privileged user namespaces. On e.g. debian this can be enabled with 'sysctl kernel.unprivileged_userns_clone=1'.

BPF_TRACE_PRINTK:

capable denied: [caller pid=36821 cgid=8120 cap=21 opts=0] [protected cgid=8120]

Figure B.12